
rwslib Documentation

Release 1.2.3

Ian Sparks

Nov 13, 2018

Contents

1	Getting started	3
1.1	Installation	3
1.2	Creating a connection to RWS	3
1.3	Making an RWS request	4
1.4	Overriding default domain name and virtual directory	4
1.5	Authentication	5
1.6	Timeouts	5
1.7	Retries	6
1.8	Getting more information from last_result	6
1.9	Getting the elapsed time of the request	6
1.10	Error Handling	7
1.11	Multi-byte encodings	7
2	rws_requests	9
2.1	VersionRequest()	9
2.2	BuildVersionRequest()	10
2.3	CodeNameRequest()	10
2.4	DiagnosticsRequest()	10
2.5	TwoHundredRequest()	10
2.6	CacheFlushRequest()	11
2.7	ConfigurableDatasetRequest()	11
3	Working with Clinical Data	13
3.1	Projectnames, Studynames and Environments	13
3.2	ClinicalStudiesRequest()	13
3.3	StudySubjectsRequest(self, project_name, environment_name)	14
4	ODM Clinical View Datasets	17
4.1	StudyDatasetRequest(project_name, environment_name)	18
4.2	SubjectDatasetRequest(project_name, environment_name, subjectkey)	19
4.3	VersionDatasetRequest(project_name, environment_name, version_oid)	20
4.4	X-MWS-CV-Last-Updated	21
5	Working with Architect Metadata	23
5.1	MetadataStudiesRequest()	23
5.2	StudyDraftsRequest(project_name)	24
5.3	StudyVersionsRequest(projectname)	25

5.4	StudyVersionRequest(projectname, oid)	25
5.5	PostMetadataRequest(projectname, data)	26
6	Working with Global Library Metadata	29
6.1	GlobalLibrariesRequest()	29
6.2	GlobalLibraryDraftsRequest(projectname)	29
6.3	GlobalLibraryVersionsRequest(projectname)	30
6.4	GlobalLibraryVersionRequest(projectname, version_oid)	30
7	Posting Clinical Data	33
7.1	PostDataRequest(odm_data)	33
7.2	Using Builders	34
8	Using Builders	37
8.1	Metadata Builders	39
9	Biostats Gateway Requests	43
9.1	CVMetaDataRequest(project_name, environment_name)	43
9.2	FormDataRequest(project_name, environment_name, dataset_type, form_oid)	45
9.3	MetaDataRequest()	46
9.4	ProjectMetaDataRequest(project_name)	49
9.5	ViewMetaDataRequest(view_name)	51
9.6	CommentDataRequest(project_name, environment_name)	53
9.7	ProtocolDeviationsRequest(project_name, environment_name)	54
9.8	DataDictionariesRequest(project_name, environment_name)	55
10	ODM Adapter Requests	57
10.1	AuditRecordsRequest(project_name, environment_name)	57
10.2	VersionFoldersRequest(project_name, environment_name)	58
10.3	SitesMetadataRequest()	60
10.4	UsersRequest(project_name, environment_name, location_oid=None)	61
10.5	SignatureDefinitionsRequest(project_name)	62
11	rwscmd	63
11.1	Usage	63
11.2	Examples	63
12	Class Reference	65
12.1	rwslib	65
12.2	Rave Web Services Request Objects	66
12.3	Rave Web Services Objects	71
12.4	Biostats Gateway	75
12.5	ODM Adapter	78
13	Class Reference for Builders	79
13.1	Metadata Builders	79
13.2	Clinical Data Builders	94
13.3	Medidata Extensions to ODM (MODM) Builders	100
13.4	Administrative Data Builders	107
14	Indices and tables	111
	Python Module Index	113

rwslib is a python library that provides a simple interface to Rave Web Services (RWS)

Contents:

CHAPTER 1

Getting started

Note that for any API calls that require authentication (anything useful) you will need access to a Rave environment. Medidata runs a program for integration access to this system, *Developer Central*. [Apply for access](#)

1.1 Installation

We strongly recommend working within a virtual environment with [Virtualenv](#).

Install with pip:

```
$ pip install rwslib
```

Or directly from github with:

```
$ pip install git+https://github.com/mdsol/rwslib.git
```

This will also install all required dependencies. Note that on Windows, lxml requires a binary installation.

1.2 Creating a connection to RWS

Before you can do any work with rwslib you must create a connection to a Rave instance. This is done through the RWSConnection object:

```
>>> from rwslib import RWSConnection
>>> rws = RWSConnection('innovate')
```

Note that the first parameter to the RWSConnection is the name of the url you wish to connect to. A url that does not start with “http” is assumed to be the sub-domain of mdsol.com. In the example above “innovate” is treated as `https://innovate.mdsol.com`.

If you wish to override this behaviour, supply a base URL that includes http or https at the start of the url:

```
>>> rws = RWSConnection('http://192.168.1.99')
```

It is important to understand that an `RWSConnection` is not a persistent connection to Rave, it is simply a convenience class for making calls to RWS endpoints.

1.3 Making an RWS request

Once you have an `RWSConnection`, you can use it to send messages to Rave and receive results back.

`rwslib` provides a set of request classes. To make a request, create an instance of that request type and pass it to the `RWSConnection` `send_request` method:

```
>>> from rwslib import RWSConnection
>>> rws = RWSConnection('innovate')
>>> from rwslib.rws_requests import VersionRequest
>>> rws.send_request(VersionRequest())
u'1.8.0'
```

The result you get back from `send_request` will depend on the request type since `Request` objects have the chance to process the text values returned from Rave. `VersionRequest()` returns a string value but other request types may return objects or collections (python lists) of objects.

All `Request` classes are descendants of the `RWSRequest` class. The API is designed this way so that new or custom requests can be easily added to the library. This also allows for easier versioning of requests and also subclassing of existing request types. For instance, `VersionRequest()` could be subclassed to return major, minor and patch-level values as a tuple of integers rather than as a string. This allows you to make specialized request classes for your integration.

`rwslib` provides several sets of standard request types arranged into python units:

- `rws_requests.py` contains the `RWSRequest` class and standard requests like `VersionRequest()`
- `rws_cv_requests.py` contains requests related to Rave Clinical Views and BioStat Gateway data extracts for Comments and Protocol Violations
- `odm_adapter_requests.py` contains requests related to the ODM Adapter datasets added in Rave 2013.3.0

1.4 Overriding default domain name and virtual directory

For convenience `rwslib` defaults the domain name to end with ‘`mdsol.com`’ and the virtual directory to be ‘`RaveWebServices`’:

```
>>> from rwslib import RWSConnection
>>> rws = RWSConnection('innovate')
>>> rws.base_url
'https://innovate.mdsol.com/RaveWebServices'
```

The default values will work for most Rave URLs but you can override them if necessary:

```
>>> from rwslib import RWSConnection
>>> rws = RWSConnection('http://10.0.1.20', virtual_dir='RWS')
>>> rws.base_url
'http://10.0.1.20/RWS'
```


1.5 Authentication

Most requests require authentication. Requests can be authenticated through Basic Authentication by providing a Rave (not iMedidata) username and password:

```
>>> from rwslib import RWSConnection
>>> from rwslib.rws_requests import MetadataStudiesRequest
>>> rws = RWSConnection('https://innovate.mdsol.com', "raveusername", "ravepassword")
>>>
>>> # Make an authenticated request to Rave
>>> rws.send_request(MetadataStudiesRequest())
```

Alternatively you can make a request using MAuth credentials. MAuth is Medidata's API authentication mechanism. MAuth credentials consist of an App UUID representing the application making the request and a Private Key, representing it's proof that it is who it says it is. These two are used with MAuth to sign requests.

Medidata provides the `requests_mauth` library which provides MAuth signing capabilities for accessing Medidata API's via MAuth:

```
>>> from requests_mauth import MAuth
>>> from rwslib import RWSConnection
>>> from rwslib.rws_requests import MetadataStudiesRequest
>>>
>>> app_id = '635r8aib-21e9-6b5f-867e-bk2358ub2784'
>>> key = open('private_key_file', 'r').read()
>>>
>>> rws = RWSConnection('https://innovate.mdsol.com', auth=MAuth(app_id, key))
>>>
>>> # Make an authenticated request to Rave
>>> rws.send_request(MetadataStudiesRequest())
```

A set of MAuth credentials are associated with a user in Rave just as with Basic Authentication, requests are performed in the context of this users rights and permissions. However, a user account associated with MAuth App ID does not have password expiry so MAuth is a better approach to long-term integrations with Rave URLs.

Note that an MAuth AppID can be associated with multiple Rave URLs but only one user per URL.

1.6 Timeouts

By default rwslib will not timeout. A timeout limit, in seconds, can be set on `send_request`, after which a Timeout exception will be thrown:

```
>>> from rwslib import RWSConnection
>>> rws = RWSConnection('innovate', 'my_username', 'my_password')
>>> #Get the rave version from rws
>>> rws.send_request(VersionRequest(), timeout=1)
```

In practice the timeout should be set to a value greater than any expected valid response time, which will vary depending upon the request types and volumes of data sent or received.

This timeout setting only applies to rwslib and does not alter timeouts in RWS itself or any other component in the network such as load balancers, etc.

1.7 Retries

By default rwslib will make a request only once. You can adjust the number of retries by setting the `retries` parameter to `send_request`:

```
>>> from rwslib import RWSConnection
>>> rws = RWSConnection('innovate', 'my_username', 'my_password')
>>> #Get the rave version from rws
>>> rws.send_request(VersionRequest(), retries=3)
```

Note that you should be very careful with retries when a request makes changes to data (e.g. POST requests) since in some situations errors can be returned by Rave and the request may still succeed.

1.8 Getting more information from `last_result`

Each time `RWSConnection` sends a request and receives results it keeps the result of the RWS call in it's `last_result` attribute. This is very useful for debugging RWS calls since it allows you to find out what headers were sent, what URL was called etc.

```
>>> from rwslib import RWSConnection
>>> from rwslib.rws_requests import VersionRequest
>>> rws = RWSConnection('innovate')
>>> #Get the rave version from rws
>>> rws.send_request(VersionRequest())
u'1.8.0'
>>> rws.last_result.url
https://innovate.mdsol.com/RaveWebServices/version
>>> rws.last_result.status_code
200
>>> rws.last_result.headers['content-type']
text/plain; charset=utf-8
>>> rws.last_result.text
u'1.8.0'
```

`last_result` is a `Requests` object. Please see that library for more information on all the properties that can be returned there.

Having access to `last_result` means that rwslib never hides it's workings from you. rwslib is intended to be a helper library to get your own integrations up and running, it tries not to hide implementation details from you.

1.9 Getting the elapsed time of the request

Each time `RWSConnection` sends a request and receives results it keeps the elapsed time, in seconds, of the RWS call in it's `request_time` attribute.

```
>>> from rwslib import RWSConnection
>>> from rwslib.rws_requests import VersionRequest
>>> rws = RWSConnection('innovate')
>>> #Get the rave version from rws
>>> rws.send_request(VersionRequest())
u'1.8.0'
>>> #Get the elapsed time in seconds to process the previous request
```

(continues on next page)

(continued from previous page)

```
>>> rws.request_time
0.760736942291
```

1.10 Error Handling

RWS returns a variety of error results depending on the type of request. `rwslib` packages these error types into `rwsobjects.RWSEException` exceptions which have an `rws_error` property. The `rws_error` property is populated with a different object type depending on the error type.

Where RWS returns an XML error response, `rwslib` will parse the error and return it in an `rwsobjects.RWSError` or `rwsobjects.RWSErrorResponse` object.

`RWSError` instances have an `errordescription` attribute while `RWSErrorResponse` have an `errordescription` and a `reasoncode`.

`RWSError` is used to parse ODM-formatted return messages like:

```
<?xml version="1.0" encoding="utf-8"?>
<ODM xmlns:mdsol="http://www.mdsol.com/ns/odm/metadata"
  FileType="Snapshot"
  CreationDateTime="2013-04-08T10:28:49.578-00:00"
  FileOID="4d13722a-ceb6-4419-a917-b6ad5d0bc30e"
  ODMVersion="1.3"
  mdsol:ErrorDescription="Incorrect login and password combination. [RWS00008]"
  xmlns="http://www.cdisc.org/ns/odm/v1.3" />
```

`RWSErrorResponse` parses simple XML return messages like:

```
<Response
  ReferenceNumber="0b47fe86-542f-4070-9e7d-16396a5ef08a"
  InboundODMFileOID="Not Supplied"
  IsTransactionSuccessful="0"
  ReasonCode="RWS00092"
  ErrorClientResponseMessage="CRF version not found">
</Response>
```

`RWSEException` also has a standard `message` attribute which the error description content from the RWS error is copied into. The purpose of this scheme is to make `rwslib` raise a standard exception type that surfaces the error message from the source RWS response but which also provides full access to the content of the original RWS error message.

1.11 Multi-byte encodings

In some cases the underlying `requests` library does not detect the correct encoding for the response; this is partly down to how Rave reports the *Content-Type* for different requests. To be able to safely encode the response for character sets beyond ‘UTF-8’, it is recommended the user subclasses the underlying `rwslib` Request object and force the encoding on the response object

As an example:

```
from rwslib.rws_requests import StudyDatasetRequest

class UTF8StudyDatasetRequest(StudyDatasetRequest):
    def result(self, response):
        """
        Process a result to create a custom output
        :param requests.models.Response response: returned response
        :return:
        """
        # By default return text
        response.encoding = "utf-8-sig"
        return response.text
```

The `rws_requests` module contains the core `RWSRequest` class and a set of `Request` classes that are standard in all versions of Rave Web Services for Rave.

Also part of this module:

- *Working with Architect Metadata*
- *Working with Global Library Metadata*
- *Working with Clinical Data*
- *Posting Clinical Data*

`rws_requests` also provides additional requests in sub-modules for additional features:

- *Biostats Gateway Requests*
- *ODM Adapter Requests*

2.1 VersionRequest()

Returns the text result of calling:

```
https://{ host }/RaveWebServices/version
```

Example:

```
>>> from rwslib import RWSConnection
>>> from rwslib.rws_requests import VersionRequest
>>> r = RWSConnection('innovate', 'username', 'password') #Authorization optional
>>> r.send_request(VersionRequest())
u'1.15.0'
```

2.2 BuildVersionRequest()

Returns the text result of calling:

```
https://{ host }/RaveWebServices/version/build
```

Returns a 200 response code and the internal build number.

Example:

```
>>> from rwslib import RWSConnection
>>> from rwslib.rws_requests import BuildVersionRequest
>>> r = RWSConnection('innovate', 'username', 'password') #Authorization optional
>>> r.send_request(BuildVersionRequest())
u'5.6.5.213'
```

2.3 CodeNameRequest()

Returns the text result of calling:

```
https://{ host }/RaveWebServices/version/codename
```

Returns a 200 response code and the internal code name of the RWS version.

Example:

```
>>> from rwslib import RWSConnection
>>> from rwslib.rws_requests import CodeNameRequest
>>> r = RWSConnection('innovate') #Authorization optional
>>> r.send_request(CodeNameRequest())
u'Uakari'
```

2.4 DiagnosticsRequest()

Returns the text result of calling:

```
https://{ host }/RaveWebServices/diagnostics
```

Returns a 200 response code and the text *OK* if RWS self-checks pass.

Example:

```
>>> from rwslib import RWSConnection
>>> from rwslib.rws_requests import DiagnosticsRequest
>>> r = RWSConnection('innovate', 'username', 'password') #Authorization optional
>>> r.send_request(DiagnosticsRequest())
u'OK'
```

2.5 TwoHundredRequest()

Returns the html result of calling:

```
https://{ host }/RaveWebServices/twohundred
```

Returns a 200 response code and a html document that contains information about the MAuth configuration of Rave Web Services on this url.

Example:

```
>>> from rwslib import RWSConnection
>>> from rwslib.rws_requests import TwoHundredRequest
>>> r = RWSConnection('innovate') #Authorization optional
>>> r.send_request(TwoHundredRequest())
u'<!DOCTYPE html>\r\n<html>\r\n<head><script>....
```

2.6 CacheFlushRequest()

Authorization is required for this method call.

Returns the text result of calling:

```
https://{ host }/RaveWebServices/websevice.aspx?CacheFlush
```

Warning: Calling CacheFlush unnecessarily causes RWS to re-load objects, causing additional resource utilization that could have a detrimental affect on Rave performance.

Flushes the RWS cache. Generally Rave and RWS manage their own caching. You should not need to use this method in normal operation. Returns a `rwsobjects.RWSResponse` object with a `istransactionsuccessful` attribute:

Example:

```
>>> from rwslib import RWSConnection
>>> from rwslib.rws_requests import CacheFlushRequest
>>> r = RWSConnection('innovate', 'username', 'password') #Authorization REQUIRED
>>> response = r.send_request(CacheFlushRequest())
>>> response.istransactionsuccessful
True
```

2.7 ConfigurableDatasetRequest()

Authorization is required for this method call.

Returns the text result of calling:

```
https://{ host }/RaveWebServices/datasets/{dataset_name}(.{dataset_format})?{params}
```

Sends a Configurable Dataset request to RWS. The `dataset_format` argument is optional and is only required if the corresponding configurable dataset requires it. The primary use case of this is as an abstract class that the user can subclass for their particular Configurable Dataset; the implemented class could such as validation of the requested `dataset_format` against the list of formats accepted by the configurable dataset or by overloading the `result` method to parse the raw response content (e.g. return a pre-parsed JSON response or a `csv.reader`). Returns a `rwsobjects.RWSResponse` object:

Example:

```
>>> from rwslib import RWSConnection
>>> from rwslib.rws_requests import ConfigurableDatasetRequest
>>> r = RWSConnection('innovate', 'username', 'password') #Authorization REQUIRED
>>> response = r.send_request(ConfigurableDatasetRequest('SomeRequest', dataset_
↳format='csv', params=dict(start='2012-02-01')))
>>> response.text
DataPageID,DataPointID,LastUpdated
1234,4321,2012-12-01T12:33:00
4334,1234,2012-12-02T12:33:00
...
```

Working with Clinical Data

The `rws_requests` module provides a number of standard requests which provide lists of clinical studies, extract clinical data (from Rave Clinical Views) and post clinical data to Rave.

3.1 Projectnames, Studynames and Environments

`rwslib` uses some standard definitions for studyname, protocol names and environments. It defines them as:

- Projectname - The name of the study without environment e.g. the Mediflex in **Mediflex** (Dev)
- Environment - The environment within the study e.g. the Dev in Mediflex (**Dev**)
- StudyName - The combination of projectname and environment **Mediflex** (**Dev**)

3.2 ClinicalStudiesRequest()

Authorization is required for this method.

Returns a `rwsobjects.RWSStudies` object which has ODM document attributes and a list of `rwsobjects.RWSStudyListItem` objects. These are the clinical studies that you have access to as an EDC user.

Calls:

```
https://{ host }/RaveWebServices/studies
```

Example:

```
>>> from rwslib import RWSConnection
>>> from rwslib.rws_requests import ClinicalStudiesRequest
>>> r = RWSConnection('https://innovate.mdsol.com', 'username', 'password')
↪ #Authorization required
>>> studies = r.send_request(ClinicalStudiesRequest())
```

(continues on next page)

(continued from previous page)

```

>>> studies.ODMVersion
1.3
>>> studies.fileoid
aa0dc756-b9f3-4bb2-ab4c-ee91e573e8fb
>>> studies.filetype
Snapshot
>>> studies.creationdatetime
2013-06-04T15:58:24.781-00:00
>>> len(studies)
1
>>> for study in studies:
...     print("OID", study.oid)
...     print("Name", study.studyname)
...     print("protocolname", study.protocolname)
...     print("IsProd?", study.isProd())
...
OID Mediflex (Dev)
Name Mediflex (Dev)
protocolname Mediflex
IsProd? False

```

3.3 StudySubjectsRequest(self, project_name, environment_name)

Authorization is required for this method.

Retrieves a `rwsobjects.RWSSubjects` object which is a list of `rwsobjects.RWSSubjectListItem` objects, each representing key information about a subject. This does not include clinical data for a subject.

Option	Description
<code>status={True False}</code>	If True, extracts status information at the subject level. Default = False
<code>links={True False}</code>	If True, includes “deep link”(s) (URL) to the subject page in Rave.Default = False
<code>in- clude={inactive deleted inactiveAndDeleted}</code>	Include inactive subjects, deleted subjects or both? By default these subjects are inactiveAndDeleted
<code>sub- ject_key_type={SubjectName SubjectID}</code>	Request that the Subject Name is in the SubjectKey field (<code>_SubjectName_</code>) or in the <code>SubjectID</code> field (with the Subject UUID in the SubjectKey field).

Calls:

```

https://{ host }/RaveWebServices/studies/{ projectname } ({ environment_name})/
↳subjects?{options}

```

Example:

```

>>> from rwslib import RWSSConnection
>>> from rwslib.rws_requests import StudySubjectsRequest
>>> r = RWSSConnection('https://innovate.mdsol.com', 'username', 'password')
↳#Authorization required
>>> subject_list = rave.send_request(StudySubjectsRequest("SIMPLESTUDY", "PROD")
>>> subject_list.ODMVersion
1.3
>>> for subject in subject_list:
...     print "Name: %s" % subject.subjectkey
Name: 1

```

(continues on next page)

(continued from previous page)

```

Name: 10
Name: 2
Name: 3
Name: 4
...
>>> r.last_url
https://innovate.mdsol.com/RaveWebServices/studies/SIMPLESTUDY(PROD)/subjects
>>> str(subject_list)
<ODM xmlns:mdsol="http://www.mdsol.com/ns/odm/metadata" xmlns:xlink="http://www.w3.
↳org/1999/xlink" xmlns="http://www.cdisc.org/ns/odm/v1.3" FileType="Snapshot"
↳FileOID="1af945c7-8334-4eb8-b7a9-735fb5c7db03" CreationDateTime="2013-09-
↳10T09:28:21.145-00:00" ODMVersion="1.3">
  <ClinicalData StudyOID="SIMPLESTUDY(TEST)" MetaDataVersionOID="1128">
    <SubjectData SubjectKey="1">
      <SiteRef LocationOID="TESTSITE"/>
    </SubjectData>
  </ClinicalData>
  <ClinicalData StudyOID="SIMPLESTUDY(TEST)" MetaDataVersionOID="1128">
    <SubjectData SubjectKey="10">
      <SiteRef LocationOID="TESTSITE"/>
    </SubjectData>
  </ClinicalData>
  <ClinicalData StudyOID="SIMPLESTUDY(TEST)" MetaDataVersionOID="1128">
    <SubjectData SubjectKey="2">
      <SiteRef LocationOID="TESTSITE"/>
    </SubjectData>
  </ClinicalData>
  <ClinicalData StudyOID="SIMPLESTUDY(TEST)" MetaDataVersionOID="1128">
    <SubjectData SubjectKey="3">
      <SiteRef LocationOID="TESTSITE"/>
    </SubjectData>
  </ClinicalData>
  <ClinicalData StudyOID="SIMPLESTUDY(TEST)" MetaDataVersionOID="1128">
    <SubjectData SubjectKey="4">
      <SiteRef LocationOID="TESTSITE"/>
    </SubjectData>
  </ClinicalData>
  <ClinicalData StudyOID="SIMPLESTUDY(TEST)" MetaDataVersionOID="1128">
    ...
>>> subject_list = rave.send_request(StudySubjectsRequest("SIMPLESTUDY", "PROD",
↳subject_key_type="SubjectUUID")
>>> str(subject_list)
<ODM xmlns:mdsol="http://www.mdsol.com/ns/odm/metadata" xmlns:xlink="http://www.w3.
↳org/1999/xlink" xmlns="http://www.cdisc.org/ns/odm/v1.3" FileType="Snapshot"
↳FileOID="1af945c7-8334-4eb8-b7a9-735fb5c7db03" CreationDateTime="2013-09-
↳10T09:28:21.145-00:00" ODMVersion="1.3">
  <ClinicalData StudyOID="SIMPLESTUDY(TEST)" MetaDataVersionOID="1128">
    <SubjectData SubjectKey="0C1F5F71-B136-4C95-8199-1397F4262B31"
↳mdsol:SubjectKeyType="SubjectUUID" mdsol:SubjectName="1">
      <SiteRef LocationOID="TESTSITE"/>
    </SubjectData>
  </ClinicalData>
  <ClinicalData StudyOID="SIMPLESTUDY(TEST)" MetaDataVersionOID="1128">
    <SubjectData SubjectKey="91F686CE-37A0-4A9D-BC3B-CFFC3C609ECC"
↳mdsol:SubjectKeyType="SubjectUUID" mdsol:SubjectName="10">
      <SiteRef LocationOID="TESTSITE"/>
    </SubjectData>

```

(continues on next page)

(continued from previous page)

```
</ClinicalData>
<ClinicalData StudyOID="SIMPLESTUDY(TEST)" MetaDataVersionOID="1128">
...
>>> for subject in subject_list:
...     print "Name: %s (%s)" % (subject.subject_name, subject.subjectkey)
Name: 1 (0C1F5F71-B136-4C95-8199-1397F4262B31)
Name: 10 (91F686CE-37A0-4A9D-BC3B-CFFC3C609ECC)
...
```

CHAPTER 4

ODM Clinical View Datasets

Important: Clinical Views must be active for these requests to work.

RWS allows extraction of clinical data from Rave's Clinical Views by 3 strata:

1. By Study
2. By Individual Subject
3. By MetaData Version

All 3 variants take the same options:

Option	Description
dataset_type={regular raw}	Request extracts to regular or raw data. Default is regular.
start={ISO 8601 date}	Request changes to this dataset since the start date. Note that CV's must be set to <i>Include Inactive</i> for this option to work.
rawsuf-fix={suffix}	e.g. .RAW means raw field definitions are suffixes with .RAW ex. AE_STD.T.RAW
for-moid={form oid}	Only extracts dataset information for the named form. If missing extracts for all forms.
version-item={version suffix}	Add MetaDataVersionOID="<<datetime>>" and additional itemlevel version ItemData element per itemgroup to identify last CV update date and CRF Version each itemgroup was entered under e.g. <ItemData ItemOID="AE.VERSION" Value="16" />
codelistsuffix={cl_suffix}	Add name of codelist as an item for each field with data dictionary. e.g. codelistsuffix="CL" results in <ItemData ItemDataOID="DM.SEX.CL" Value="GENDERS"/>
decodesuffix={decode_suffix}	Add user value of codelist entry as an item for each field with data dictionary. e.g. decodesuffix="DECODE" results in <ItemData ItemDataOID="DM.SEX.DECODE" Value="Male"/>
stdsuf-fix={decode_suffix}	Adds standard and unit data values to a full or incremental dataset, and identifies these values with (std) suffix.

4.1 StudyDatasetRequest(project_name, environment_name)

Authorization is required for this method.

Returns a unicode string of the ODM Clinical Data for the study / environment. Can be filtered by form (i.e. to return only data for a single form type) and by start date for an incremental dataset.

Warning: If not filtered by form this is a potentially very large download for an existing study.

Calls:

```
https://{ host }/RaveWebServices/studies/{ projectname } ({ environment_name})/
↳datasets/{ regular|raw }?{options}

or (form filtered)

https://{ host }/RaveWebServices/studies/{ projectname } ({ environment_name})/
↳datasets/{ regular|raw }/{ formoid }?{options}
```

Example:

```
>>> from rwslib import RWSConnection
>>> from rwslib.rws_requests import StudyDatasetRequest
>>> r = RWSConnection('innovate', 'username', 'password')
>>> r.send_request(StudyDatasetRequest('Mediflex', 'DEV'))
ï»¿<?xml version="1.0" encoding="utf-8"?>
  <ODM FileType="Snapshot" FileOID="6b967555-8e05-4890-afb2-c2d2f1496031"
    CreationDateTime="2013-06-07T13:13:52.883-00:00"
    ODMVersion="1.3" xmlns:mdsol="http://www.mdsol.com/ns/odm/metadata"
    xmlns:xlink="http://www.w3.org/1999/xlink" xmlns="http://www.cdisc.org"
    ....
  <ClinicalData StudyOID="Mediflex(Dev)" MetaDataVersionOID="16">
    <SubjectData SubjectKey="123 ABC">
      <SiteRef LocationOID="MDSOL" />
      <StudyEventData StudyEventOID="SUBJECT">
        <FormData FormOID="AE" FormRepeatKey="1">
          <ItemGroupData ItemGroupOID="AE_LOG_LINE" ItemGroupRepeatKey=
↳ "1">

              <ItemData ItemOID="AE.AEYN" Value="Y" />
              <ItemData ItemOID="AE.AETERM" Value="HEADACHE" />
              <ItemData ItemOID="AE.AESTDTC" Value="2008-01-01" />
              <ItemData ItemOID="AE.AESTDTC.RAW" Value="01 JAN 2008" />
              <ItemData ItemOID="AE.AEONG" Value="N" />
              <ItemData ItemOID="AE.AEENDTC" Value="2008-01-01" />
              <ItemData ItemOID="AE.AEENDTC.RAW" Value="01 JAN 2008" />
              ...
          </ItemGroupData>
        </FormData>
      </StudyEventData>
    </SubjectData>
  </ClinicalData>
```

Form Example:

```
>>> from rwslib import RWSConnection
>>> from rwslib.rws_requests import StudyDatasetRequest
>>> r = RWSConnection('innovate', 'username', 'password')
>>> r.send_request(StudyDatasetRequest('SimpleStudy', 'TEST', formoid='VITAL'))
ï»¿<?xml version="1.0" encoding="utf-8"?>
  <ODM xmlns="http://www.cdisc.org/ns/odm/v1.3" xmlns:mdsol="http://www.mdsol.com/ns/
↳ odm/metadata" xmlns:xlink="http://www.w3.org/1999/xlink" FileType="Snapshot"
↳ FileOID="f323dba3-b31b-4e61-8894-104353fac743" CreationDateTime="2013-10-08T08:33:25.811-00:00" ODMVersion="1.3">
```

(continues on next page)

(continued from previous page)

```

<ClinicalData StudyOID="SIMPLESTUDY(TEST)" MetaDataVersionOID="1128">
  <SubjectData SubjectKey="1">
    <SiteRef LocationOID="TESTSITE" />
    <StudyEventData StudyEventOID="SCREEN" StudyEventRepeatKey="1">
      <FormData FormOID="VITAL" FormRepeatKey="1">
        <ItemGroupData ItemGroupOID="VITAL_LOG_LINE">
          <ItemData ItemOID="VITAL.VDAT" Value="2013-02-01" />
          <ItemData ItemOID="VITAL.WEIGHT_KG" Value="132.0" />
          <ItemData ItemOID="VITAL.HEIGHT_CM" Value="174.5" />
        </ItemGroupData>
      </FormData>
    </StudyEventData>
  </SubjectData>
</ClinicalData>
<ClinicalData StudyOID="SIMPLESTUDY(TEST)" MetaDataVersionOID="1128">
  <SubjectData SubjectKey="2">
    <SiteRef LocationOID="TESTSITE" />
    ...
  </SubjectData>
</ClinicalData>
...
>>> r.last_url
https://innovate.mdsol.com/RaveWebServices/studies/SIMPLESTUDY(TEST)/datasets/regular/
↳ VITAL

```

Note: Note that the XML string returned by this method includes the Byte Order Mark (BOM) as sent by RWS and does not include line breaks as shown in the above example. Depending on how you intend to parse the XML returned you may need to strip the BOM.

4.2 SubjectDatasetRequest(project_name, environment_name, subjectkey)

Authorization is required for this method.

Extracts ODM data for a single subject.

Calls:

```

https://{ host }/RaveWebServices/studies/{ projectname } ({ environment_name})/
↳ subjects/{ subjectkey }/datasets/{ regular|raw }?{options}

or (form filtered)

https://{ host }/RaveWebServices/studies/{ projectname } ({ environment_name})/
↳ subjects/{ subjectkey }/datasets/{ regular|raw }/{ formoid }?{options}

```

Example:

```

>>> from rwslib import RWSConnection
>>> r = RWSConnection('innovate', 'username', 'password')
>>> from rwslib.rws_requests import SubjectDatasetRequest
>>> r.send_request(SubjectDatasetRequest('SIMPLESTUDY', 'TEST', '1', formoid='ENROL'))
i>>?<?xml version="1.0" encoding="UTF-8"?>
<ODM xmlns="http://www.cdisc.org/ns/odm/v1.3" xmlns:mdsol="http://www.mdsol.com/ns/
↳ odm/metadata" xmlns:xlink="http://www.w3.org/1999/xlink" FileType="Snapshot"
↳ FileOID="c850bb82-f08f-4f43-9c8c-fce2b5e80e79" CreationDateTime="2013-09-
↳ 10T15:23:22.395-00:00" ODMVersion="1.3">

```

(continues on next page)

(continued from previous page)

```

<ClinicalData StudyOID="SIMPLESTUDY(TEST)" MetaDataVersionOID="1128">
  <SubjectData SubjectKey="1">
    <SiteRef LocationOID="TESTSITE" />
    <StudyEventData StudyEventOID="SUBJECT">
      <FormData FormOID="ENROL" FormRepeatKey="1">
        <ItemGroupData ItemGroupOID="ENROL_LOG_LINE">
          <ItemData ItemOID="ENROL.SUBID" Value="1" />
          <ItemData ItemOID="ENROL.BIRTHDT" Value="1973-06-26" />
        </ItemGroupData>
      </FormData>
    </StudyEventData>
  </SubjectData>
</ClinicalData>
</ODM>
>>> r.last_url
https://innovate.mdsol.com/RaveWebServices/studies/SIMPLESTUDY(TEST)/subjects/1/
↳datasets/regular/ENROL

```

4.3 VersionDatasetRequest(project_name, environment_name, version_oid)

Authorization is required for this method.

Extracts ODM data for a single Rave study version across all subjects.

Calls:

```

https://{ host }/RaveWebServices/studies/{ projectname } ({ environment_name})/
↳versions/{ version_id }/datasets/{ regular|raw }?{options}

or (form filtered)

https://{ host }/RaveWebServices/studies/{ projectname } ({ environment_name})/
↳versions/{ version_id }/datasets/{ regular|raw }/{ formoid }?{options}

```

Example:

```

>>> from rwslib import RWSConnection
>>> r = RWSConnection('innovate', 'username', 'password')
>>> from rwslib.rws_requests import VersionDatasetRequest
>>> r.send_request(VersionDatasetRequest('SIMPLESTUDY', 'TEST', 1128, formoid='VITAL'))
i»¿<?xml version="1.0" encoding="UTF-8"?>
<ODM xmlns="http://www.cdisc.org/ns/odm/v1.3" xmlns:mdsol="http://www.mdsol.com/ns/
↳odm/metadata" xmlns:xlink="http://www.w3.org/1999/xlink" FileType="Snapshot"
↳FileOID="00d28b0e-df45-43a4-93dc-7e4dd3cf36e7" CreationDateTime="2013-09-
↳10T15:45:54.179-00:00" ODMVersion="1.3">
  <ClinicalData StudyOID="SIMPLESTUDY(TEST)" MetaDataVersionOID="1128">
    <SubjectData SubjectKey="1">
      <SiteRef LocationOID="TESTSITE" />
      <StudyEventData StudyEventOID="SCREEN" StudyEventRepeatKey="1">
        <FormData FormOID="VITAL" FormRepeatKey="1">
          <ItemGroupData ItemGroupOID="VITAL_LOG_LINE">
            <ItemData ItemOID="VITAL.VDAT" Value="2013-02-01" />
            <ItemData ItemOID="VITAL.WEIGHT_KG" Value="132.0" />
          </ItemGroupData>
        </FormData>
      </StudyEventData>
    </SubjectData>
  </ClinicalData>
</ODM>

```

(continues on next page)

(continued from previous page)

```

        <ItemData ItemOID="VITAL.HEIGHT_CM" Value="174.5" />
      </ItemGroupData>
    </FormData>
  </StudyEventData>
</SubjectData>
</ClinicalData>
<ClinicalData StudyOID="SIMPLESTUDY(TEST)" MetaDataVersionOID="1128">
  <SubjectData SubjectKey="2">
    <SiteRef LocationOID="TESTSITE" />
    <StudyEventData StudyEventOID="SCREEN" StudyEventRepeatKey="1">
      <FormData FormOID="VITAL" FormRepeatKey="1">
        <ItemGroupData ItemGroupOID="VITAL_LOG_LINE">
          <ItemData ItemOID="VITAL.VDAT" Value="2013-02-09" />
          .....
        </ItemGroupData>
      </FormData>
    </StudyEventData>
  </SubjectData>
</ClinicalData>

```

>>> r.last_url
 https://innovate.mdsol.com/RaveWebServices/studies/SIMPLESTUDY(TEST)/versions/1128/
 ↪ datasets/regular/VITAL

4.4 X-MWS-CV-Last-Updated

The Clinical View datasets return a header, X-MWS-CV-Last-Updated, which tells you the last time a Clinical View was updated.

This is especially important with Incremental calls using the `start` option to know whether the dataset has actually been updated since your last call.

You can get the value of X-MWS-CV-Last-Updated via the `last_result.headers` property:

```

>>> from rwslib import RWSConnection
>>> from rwslib.rws_requests import StudyDatasetRequest
>>> r = RWSConnection('innovate', 'username', 'password')
>>> xml = r.send_request(StudyDatasetRequest('Mediflex', 'DEV'))
>>> r.last_result.headers['X-MWS-CV-Last-Updated']
2013-05-02T19:11:46

```

Working with Architect Metadata

Access to Architect study designs relies on the credentials you supply having access to Rave Architect. See [Working with Global Library Metadata](#) for details on how to access study Metadata in Global Libraries.

5.1 MetadataStudiesRequest()

Authorization is required for this method.

Returns a `rwsobjects.RWSStudies` object which has `rwsobjects.ODMDoc` document attributes and a list of `rwsobjects.RWSStudyListItem` objects. These are the studies you have access to as an Architect user.

Use [ClinicalStudiesRequest\(\)](#) to get a list of studies that you can interact with as an EDC user.

Calls:

```
https://{ host }/RaveWebServices/metadata/studies
```

Example:

```
>>> from rwslib import RWSConnection
>>> from rwslib.rws_requests import MetadataStudiesRequest
>>> r = RWSConnection('innovate', 'username', 'password') #Authorization required
>>> studies = r.send_request(MetadataStudiesRequest())
>>> studies.ODMVersion
1.3
>>> studies.fileoid
4e7f5b37-3f77-4a80-8b91-c48978103bae
>>> studies.filetype
Snapshot
>>> studies.creationdatetime
2013-06-07T02:13:31
>>> len(studies)
1
>>> for study in studies:
```

(continues on next page)

(continued from previous page)

```

...     print("-----")
...     print("OID", study.oid)
...     print("Name", study.studyname)
...     print("protocolname", study.protocolname)
...     print("IsProd?", study.isProd())
...     print("ProjectType", study.projecttype)
...
-----
OID Bill's New Library
Name Bill's New Library
protocolname Bill's New Library
IsProd? False
ProjectType GlobalLibraryVolume
-----
OID Rave CDASH
Name Rave CDASH
protocolname Rave CDASH
IsProd? False
ProjectType GlobalLibraryVolume

```

Studies returned in the list from `metadata_studies()` have an empty environment attribute since they represent study designs, they do not have associated environments.

5.2 StudyDraftsRequest(project_name)

Authorization is required for this method.

Returns a `rwsobjects.RWSStudyMetadataVersions` object which inherits from `rwsobjects.RWSStudyListItem`. This has ODM document attributes, study attributes and a list of `rwsobjects.MetadataVersion` objects representing Rave Architect Drafts.

Calls:

```
https://{ host }/RaveWebServices/metadata/studies/{ projectname }/drafts
```

Example:

```

>>> from rwslib import RWSConnection
>>> from rwslib.rws_requests import StudyDraftsRequest
>>> r = RWSConnection('innovate', 'username', 'password') #Authorization required
>>> projectname = 'Mediflex'
>>> drafts = r.send_request(StudyDraftsRequest(projectname))
>>> drafts.fileoid
e88d622d-8ddd-476c-8978-ccfe23b26969

>>> drafts.study.studyname
Mediflex

>>> for draft in drafts:
...     print(draft.name, draft.oid)
Draft2 1006
Draft1 126

```

5.3 StudyVersionsRequest(projectname)

Authorization is required for this method.

Returns a `rwsobjects.RWSStudyMetadataVersions` object which inherits from `rwsobjects.RWSStudyListItem`. This has ODM document attributes, study attributes and a list of `rwsobjects.MetadataVersion` objects representing Rave Architect Versions

Calls:

```
https://{ host }/RaveWebServices/metadata/studies/{ projectname }/versions
```

Example:

```
>>> from rwslib import RWSConnection
>>> from rwslib.rws_requests import StudyVersionsRequest
>>> r = RWSConnection('https://innovate.mdsol.com', 'username', 'password')
>>> projectname = 'Mediflex'
>>> versions = r.send_request(StudyVersionsRequest(projectname))
>>> versions.fileoid
66567494-c76a-4b94-afbb-64f5c1b21cbb
>>> versions.study.studyname
Mediflex
>>> for version in versions:
...     print(version.name, version.oid)
v19 1015
v18 1007
v17 999
v16 481
```

5.4 StudyVersionRequest(projectname, oid)

Authorization is required for this method.

Returns a unicode string of the ODM Metadata for this study version.

Calls:

```
https://{ host }/RaveWebServices/metadata/studies/{ projectname }/versions/{ oid }
```

Example:

```
>>> from rwslib import RWSConnection
>>> from rwslib.rws_requests import StudyVersionRequest
>>> r = RWSConnection('innovate', 'username', 'password')

>>> projectname = 'Mediflex'
>>> version_oid = 1015
>>> r.send_request(StudyVersionRequest(projectname, version_oid))
<ODM FileType="Snapshot" Granularity="Metadata" CreationDateTime="2013-06-05T08:30:45.
↪900-00:00"
    FileOID="012d24dd-d7d8-44fe-997b-b287ae4faf7e" ODMVersion="1.3"
    xmlns:mdsol="http://www.mdsol.com/ns/odm/metadata" xmlns="http://www.cdisc.org/
↪ns/odm/v1.3">
    <Study OID="Mediflex">
        <GlobalVariables>
```

(continues on next page)

(continued from previous page)

```

    <StudyName>Mediflex</StudyName>
    <StudyDescription></StudyDescription>
    <ProtocolName>Mediflex</ProtocolName>
  </GlobalVariables>
  <BasicDefinitions>
    ....

```

rwslib does not automatically parse this xml for you (you may want to control this yourself) but rwslib does include some XML helper methods that make working with XML that is generated from web-services easier.

Note: At this time RWS does not provide a way retrieve the ODM for a Draft, only for a Version.

5.5 PostMetadataRequest(projectname, data)

Authorization is required for this method.

Creates a new study draft (or overwrites an existing one) with a new ODM definition.

Calls:

```
POST https://{ host }/RaveWebServices/metadata/studies/{ projectname }/drafts
```

Options:

Option	Description
headers={ 'Content-type': "text/xml" }	Set custom headers. May need to provide a Content-type if your RWS version is set to accept different standard content-type than default.

Example:

```

>>> from rwslib import RWSConnection
>>> from rwslib.rws_requests import PostMetadataRequest
>>> r = RWSConnection('innovate', 'username', 'password')

>>> projectname = 'TESTSTUDY'
>>> odm_definition = """<ODM FileType="Snapshot" Granularity="Metadata"
...   CreationDateTime="2013-06-18T15:09:54.843-00:00" FileOID="82370e27-a6a5-41dc-
↪8c07-829e489823df" ODMVersion="1.3"
...   xmlns:mdsol="http://www.mdsol.com/ns/odm/metadata" xmlns="http://www.cdisc.org/
↪ns/odm/v1.3">
...   <Study OID="TESTSTUDY">
...     <GlobalVariables>
...       <StudyName>TESTSTUDY</StudyName>
...       <StudyDescription></StudyDescription>
...       <ProtocolName>TESTSTUDY</ProtocolName>
...     </GlobalVariables>
...     <BasicDefinitions/>
...     <MetaDataVersion OID="1" Name="TINY STUDY" mdsol:PrimaryFormOID="ENROL"
↪mdsol:DefaultMatrixOID="DEFAULT" mdsol:SignaturePrompt="Sign this please.">
...       <Protocol>
...         <StudyEventRef StudyEventOID="SCREEN" OrderNumber="1" Mandatory="No" />
...       </Protocol>

```

(continues on next page)

(continued from previous page)

```

...     <StudyEventDef OID="SCREEN" Name="Screening" Type="Common" Repeating="Yes"
↳mdsol:OrderNumber="1">
...         <FormRef FormOID="VITAL" OrderNumber="2" Mandatory="No" />
...     </StudyEventDef>
...     <FormDef OID="ENROL" Name="Enrol" Repeating="No" mdsol:OrderNumber="1"
↳mdsol:ConfirmationStyle="None">
...         <ItemGroupRef ItemGroupOID="ENROL" Mandatory="Yes" />
...     </FormDef>
...     <FormDef OID="VITAL" Name="Vitals" Repeating="No" mdsol:OrderNumber="2">
...         <ItemGroupRef ItemGroupOID="VITAL" Mandatory="Yes" />
...     </FormDef>
...     <ItemGroupDef OID="ENROL" Name="ENROL" Repeating="No">
...         <ItemRef ItemOID="SUBID" OrderNumber="1" Mandatory="No" />
...         <ItemRef ItemOID="BIRTHDT" OrderNumber="2" Mandatory="No" />
...     </ItemGroupDef>
...     <ItemGroupDef OID="VITAL" Name="VITAL" Repeating="No">
...         <ItemRef ItemOID="WEIGHT_KG" OrderNumber="1" Mandatory="No" />
...         <ItemRef ItemOID="HEIGHT_CM" OrderNumber="2" Mandatory="No" />
...     </ItemGroupDef>
...     <ItemDef OID="SUBID" Name="SUBID" DataType="text" Length="10"
↳mdsol:ControlType="Text">
...         <Question>
...             <TranslatedText xml:lang="en">Subject ID</TranslatedText>
...         </Question>
...     </ItemDef>
...     <ItemDef OID="BIRTHDT" Name="BIRTHDT" DataType="date" mdsol:DateTimeFormat=
↳"yyyy MMM dd" mdsol:ControlType="DateTime">
...         <Question>
...             <TranslatedText xml:lang="en">Date of Birth</TranslatedText>
...         </Question>
...     </ItemDef>
...     <ItemDef OID="WEIGHT_KG" Name="WEIGHT_KG" DataType="float" Length="4"
↳SignificantDigits="1" mdsol:ControlType="Text">
...         <Question>
...             <TranslatedText xml:lang="en">Weight</TranslatedText>
...         </Question>
...     </ItemDef>
...     <ItemDef OID="HEIGHT_CM" Name="HEIGHT_CM" DataType="float" Length="4"
↳SignificantDigits="1" mdsol:ControlType="Text">
...         <Question>
...             <TranslatedText xml:lang="en">Height</TranslatedText>
...         </Question>
...     </ItemDef>
... </MetaDataVersion>
... </Study>
... </ODM>

>>> response = r.send_request(PPostMetadataRequest(projectname, odm_definition))
>>> print(str(response))
<Response ReferenceNumber="5b260cff-e136-4b44-9211-e473fa4d6048"
    InboundODMFileOID="82370e27-a6a5-41dc-8c07-829e489823df"
    IsTransactionSuccessful="1"
    SuccessStatistics="N/A" NewRecords="" DraftImported="1">
</Response>

```

Working with Global Library Metadata

rwslib provides access to metadata stored in Rave Global Library Volumes. You must provide credentials that have access to Global Library Volumes to read this metadata. See [Working with Architect Metadata](#) for information on working with Architect Study Designs.

6.1 GlobalLibrariesRequest()

Authorization is required for this method.

Returns a `rwobjects.RWSStudies` object which inherits from `RWSStudy`. This has ODM document attributes, study attributes and a list of `rwobjects.RWSStudyListItem` objects representing Rave Architect Global Library Drafts

Calls:

```
https://{ host }/RaveWebServices/metadata/libraries
```

Example:

```
>>> from rwslib import RWSConnection
>>> from rwslib.rws_requests import GlobalLibrariesRequest
>>> r = RWSConnection('https://innovate.mdsol.com', 'username', 'password')
>>> gl_studies = r.send_request(GlobalLibrariesRequest())
>>> print("There are %d global libraries" % len(gl_studies))
>>> for study in gl_studies:
...     print(study.studyname)
There are 1 global libraries
Mediflex
```

6.2 GlobalLibraryDraftsRequest(projectname)

Authorization is required for this method.

Returns a `rwsobjects.RWSStudyMetadataVersions` object which inherits from `RWSStudy`. This has ODM document attributes, study attributes and a list of `rwsobjects.RWSStudyListItem` objects representing Rave Architect Global Library Drafts

Calls:

```
https://{ host }/RaveWebServices/metadata/libraries/{ projectname }/drafts
```

Example:

```
>>> from rwslib import RWSConnection
>>> from rwslib.rws_requests import GlobalLibrariesRequest
>>> r = RWSConnection('https://innovate.mdsol.com', 'username', 'password')

>>> drafts = r.send_request(GlobalLibraryDraftsRequest('Rave CDASH'))
>>> print(drafts.fileoid)
be117ac3-e7ea-48fc-8bd8-373eb387703f
>>> for draft in drafts:
...     print("%s - %s" % (draft.name, draft.oid,))
Rave CDASH 01 - 397
```

6.3 GlobalLibraryVersionsRequest(projectname)

Authorization is required for this method.

Returns a `rwsobjects.RWSStudyMetadataVersions` object which inherits from `RWSStudy`. This has ODM document attributes, study attributes and a list of `rwsobjects.RWSStudyListItem` objects representing Rave Architect Global Library Versions.

Calls:

```
https://{ host }/RaveWebServices/metadata/libraries/{ projectname }/versions
```

Example:

```
>>> from rwslib import RWSConnection
>>> from rwslib.rws_requests import GlobalLibraryVersionsRequest
>>> r = RWSConnection('https://innovate.mdsol.com', 'username', 'password')
>>> versions = r.send_request(GlobalLibraryVersionsRequest('Rave CDASH'))
>>> versions.fileoid
50252a80-e233-4d30-9e69-b510e965e44a
>>> versions.study.studyname
Rave CDASH
>>> for version in versions:
...     print("%s - %s" % (version.name, version.oid,))
1.0 - 398
```

6.4 GlobalLibraryVersionRequest(projectname, version_oid)

Authorization is required for this method.

Returns a unicode string of the ODM Metadata for this study version from the global library.

Calls:

```
https://{ host }/RaveWebServices/metadata/libraries/{ projectname }/versions/{ version_oid }
```

Example:

```
>>> from rwslib import RWSConnection
>>> from rwslib.rws_requests import GlobalLibraryVersionRequest
>>> r = RWSConnection('innovate', 'username', 'password')
>>> r.send_request(GlobalLibraryVersionRequest("Rave CDASH", 395))
<ODM FileType="Snapshot" Granularity="Metadata" CreationDateTime="2013-06-07T15:36:23.
↳531-00:00" FileOID="f914dbf8-41fc-492b-bc8f-f4c98e471c38" ODMVersion="1.3"
↳xmlns:mdsol="http://www.mdsol.com/ns/odm/metadata" xmlns="http://www.cdisc.org/ns/
↳odm/v1.3">
  <Study OID="Rave CDASH" mdsol:ProjectType="GlobalLibraryVolume">
    <GlobalVariables>
      <StudyName>Rave CDASH</StudyName>
      <StudyDescription></StudyDescription>
      <ProtocolName>Rave CDASH</ProtocolName>
    </GlobalVariables>
    <BasicDefinitions />
    <MetaDataVersion OID="398" Name="1.0" mdsol:PrimaryFormOID="SUBJECT"
↳mdsol:DefaultMatrixOID="PRIMARY" mdsol:SignaturePrompt="I hereby confirm that all
↳data is accurate to the best of my knowledge.">
      <Protocol>
        ...
```

Note: At this time RWS does not provide a way retrieve the ODM for a Draft, only for a Version.

Posting Clinical Data

Creating or updating subject data in Rave is managed via the `post_data` method to RWS.

7.1 PostDataRequest(odm_data)

Authorization is required for this method.

Returns a `rwsobjects.RWSPostResponse` object which has attributes that report on the changes made and new counts of subjects in the study. On failure, raises an exception with a `RWSPostErrorResponse` attached in the `error` attribute that reports on the full source of the error as reported by RWS.

Calls:

```
https://{ host }/RaveWebServices/webservice.aspx?PostODMClinicalData
```

Options:

Option	Description
<code>headers={'Content-type': "text/xml"}</code>	Set custom headers. May need to provide a Content-type if your RWS version is set to accept different standard content-type than default.

Note that the content-type header that RWS will accept can be varied by configuration. The standard, default setting is `text/xml`. You should not need to specify `content_type` in the normal case.

Example:

```
>>> from rwslib import RWSEConnection
>>> from rwslib.rws_requests import PostDataRequest
>>> r = RWSEConnection('https://innovate.mdsol.com', 'username', 'password')
↪ #Authorization required
>>> data = "<?xml version='1.0' encoding='utf-8' ?>
... <ODM CreationDateTime='2013-06-17T17:03:29'
...     FileOID='3b9fea8b-e825-4e5f-bdc8-1464bdd7a664' FileType='Transactional'
...     "
```

(continues on next page)

(continued from previous page)

```

...     ODMVersion="1.3" Originator="test system"
...     xmlns="http://www.cdisc.org/ns/odm/v1.3"
...     xmlns:mdsol="http://www.mdsol.com/ns/odm/metadata">
...     <ClinicalData MetaDataVersionOID="1" StudyOID="Mediflex (DEV)">
...     <SubjectData SubjectKey="New Subject" TransactionType="Insert">
...     <SiteRef LocationOID="MDSOL" />
...     <StudyEventData StudyEventOID="SUBJECT">
...     <FormData FormOID="EN" FormRepeatKey="1" TransactionType="Update">
...     <ItemGroupData ItemGroupOID="EN" mdsol:Submission="SpecifiedItemsOnly">
...     <ItemData ItemOID="SUBJID" Value="1" />
...     <ItemData ItemOID="SUBJINIT" Value="AAA" />
...     </ItemGroupData>
...     </FormData>
...     </StudyEventData>
...     </SubjectData>
...     </ClinicalData>
... </ODM>""
>>> resp = r.send_request(PostDataRequest(data))
>>> resp.istransactionssuccessful
True
>>> resp.fields_touched
2
>>> str(resp)
<Response ReferenceNumber="ebb3dfc7-fca6-4872-84b4-f0942cd66ce7"
InboundODMFileOID="3b9fea8b-e825-4e5f-bdc8-1464bdd7a664"
→IsTransactionSuccessful="1"
SuccessStatistics="Rave objects touched: Subjects=1; Folders=0; Forms=0;
→Fields=2; LogLines=0"
NewRecords=""
SubjectNumberInStudy="1103"
SubjectNumberInStudySite="55"/>

```

7.2 Using Builders

Creating ODM strings can be error prone, especially when dealing with XML entities such as `>`, `<`, `&`; etc. `rwslib` provides the `builders` module to help build ODM documents using python syntax.

The ODM from the example above could be generated using the builder vocabulary:

```

>>> from rwslib.builders import *
>>> odm = ODM("test system") (
...     ClinicalData("Mediflex", "DEV") (
...         SubjectData("MDSOL", "New Subject", "Insert") (
...             StudyEventData("Subject") (
...                 FormData("EN", transaction_type="Update") (
...                     ItemGroupData() (
...                         ItemData("SUBJINIT", "AAA"),
...                         ItemData("SUBJID", 001)
...                     )
...                 )
...             )
...         )
...     )
... )

```

(continues on next page)

(continued from previous page)

```
>>> str(odm) #Returns the string representation of the odm object and all it's ↵  
↵children.
```

See *Using Builders* for examples of using rwslib builder objects to create ODM messages.

CHAPTER 8

Using Builders

Creating ODM messages by hand can be difficult. The builders package provides a simple API for generating these ODM documents.

Here is an example of an ODM message to create a subject and update two values in the Enrollment default form for this study.

```
<?xml version="1.0" encoding="utf-8" ?>
<ODM CreationDateTime="2013-06-17T18:49:28" FileOID="45b854a7-e170-4ff6-8ce5-
↪f511d72688cb"
  FileType="Transactional" ODMVersion="1.3"
  Originator="test system"
  xmlns="http://www.cdisc.org/ns/odm/v1.3" xmlns:mdsol="http://www.mdsol.com/ns/
↪odm/metadata">
  <ClinicalData MetaDataVersionOID="1" StudyOID="Mediflex (DEV)">
    <SubjectData SubjectKey="New Subject" TransactionType="Insert">
      <SiteRef LocationOID="MDSOL" />
      <StudyEventData StudyEventOID="SUBJECT">
        <FormData FormOID="EN" TransactionType="Update">
          <ItemGroupData ItemGroupOID="EN" mdsol:Submission="SpecifiedItemsOnly">
            <ItemData ItemOID="SUBJID" Value="1" />
            <ItemData ItemOID="SUBJINIT" Value="AAA" />
          </ItemGroupData>
        </FormData>
      </StudyEventData>
    </SubjectData>
  </ClinicalData>
</ODM>
```

This ODM file can be created using builders with this python code:

```
>>> from rwslib.builders import *
>>> odm = ODM("test system") (
...   ClinicalData("Mediflex", "DEV") (
...     SubjectData("MDSOL", "New Subject", "Insert") (
```

(continues on next page)

(continued from previous page)

```

...     StudyEventData("Subject") (
...         FormData("EN", transaction_type="Update") (
...             ItemGroupData() (
...                 ItemData("SUBJINIT", "AAA"),
...                 ItemData("SUBJID", 001)
...             )
...         )
...     )
... )
... )
... )
>>> # Get an lxml document from the ODM object for further manipulation
>>> root = odm.getroot()
>>>
>>> # Print a string representation of the ODM document
>>> print(str(odm))
<?xml version="1.0" encoding="utf-8" ?>
<ODM CreationDateTime="2013-06-17T18:49:28" FileOID="45b854a7-e170-4ff6-8ce5-
↪f511d72688cb"
    FileType="Transactional" ODMVersion="1.3" Originator="test system"
    xmlns="http://www.cdisc.org/ns/odm/v1.3" xmlns:mdsol="http://www.mdsol.com/ns/
↪odm/metadata">
  <ClinicalData MetaDataVersionOID="1" StudyOID="Mediflex (DEV)">
    <SubjectData SubjectKey="New Subject" TransactionType="Insert">
      <SiteRef LocationOID="MDSOL" />
      <StudyEventData StudyEventOID="SUBJECT">
        <FormData FormOID="EN" TransactionType="Update">
          <ItemGroupData ItemGroupOID="EN" mdsol:Submission="SpecifiedItemsOnly">
            <ItemData ItemOID="SUBJID" Value="1" />
            <ItemData ItemOID="SUBJINIT" Value="AAA" />
          </ItemGroupData>
        </FormData>
      </StudyEventData>
    </SubjectData>
  </ClinicalData>
</ODM>

```

Builders also allow you to create the elements in stages and “pipeline” them together to create the document. The following example creates the same document as above:

```

>>> from rwslib.builders import *
>>>
>>> # Make a root ODM element with originator system
>>> odm = ODM("test system")
>>>
>>> # Study and environment
>>> clinical_data = ClinicalData("Mediflex", "DEV")
>>>
>>> # Subject Site, Subject Name and the transaction type
>>> subject_data = SubjectData("MDSOL", "New Subject", "Insert")
>>>
>>> # The special "SUBJECT" event represents subject-level forms
>>> event_data = StudyEventData("SUBJECT")
>>>
>>> # We want to update this form that will be created automatically when subject_
↪created

```

(continues on next page)

(continued from previous page)

```

>>> form_data = FormData("EN", transaction_type="Update")
>>>
>>> # We need an ItemGroupData element
>>> itemgroup = ItemGroupData()
>>>
>>> # Push itemdata elements into the itemgroup
>>> itemgroup << ItemData("SUBJINIT", "AAA")
>>> itemgroup << ItemData("SUBJID", 001)
>>>
>>> # Now we put it all together
>>> odm << clinical_data << subject_data << event_data << form_data << itemgroup
>>>
>>> # Get an lxml document from the ODM object for further manipulation
>>> root = odm.getroot()
>>>
>>> # Print a string representation of the ODM document
>>> print(str(odm))
<?xml version="1.0" encoding="utf-8" ?>
<ODM CreationDateTime="2013-06-17T18:49:28" FileOID="45b854a7-e170-4ff6-8ce5-
↪f511d72688cb"
    FileType="Transactional" ODMVersion="1.3" Originator="test system"
    xmlns="http://www.cdisc.org/ns/odm/v1.3" xmlns:mdsol="http://www.mdsol.com/ns/
↪odm/metadata">
  <ClinicalData MetaDataVersionOID="1" StudyOID="Mediflex (DEV)">
    <SubjectData SubjectKey="New Subject" TransactionType="Insert">
      <SiteRef LocationOID="MDSOL" />
      <StudyEventData StudyEventOID="SUBJECT">
        <FormData FormOID="EN" TransactionType="Update">
          <ItemGroupData ItemGroupOID="EN" mdsol:Submission="SpecifiedItemsOnly">
            <ItemData ItemOID="SUBJID" Value="1" />
            <ItemData ItemOID="SUBJINIT" Value="AAA" />
          </ItemGroupData>
        </FormData>
      </StudyEventData>
    </SubjectData>
  </ClinicalData>
</ODM>

```

The builder creates a number of ODM properties including CreationDateTime, FileOID (a random identifier), FileType and all namespace declarations.

8.1 Metadata Builders

Builders also exist for creating Metadata ODM files:

```

from rwslib.builders import *

odm = ODM("SYSTEM_NAME", filetype=ODM.FILETYPE_SNAPSHOT)

study_name = 'MyStudy'
draft_name = 'Draft 1'

study = Study(study_name, project_type=Study.PROJECT)

```

(continues on next page)

(continued from previous page)

```

# Push study element into odm
odm << study

# Create global variables and set them into study.
study << GlobalVariables(study_name) # Expected that protocol name will match the_
↪Study OID.

# Create some basic definitions
bd = BasicDefinitions()

# Add some measurement units to the basic definitions. This time using the call ()_
↪syntax:
bd(
  MeasurementUnit("KG", "Kilograms") (
    Symbol() (TranslatedText("Kilograms"))
  ),
  MeasurementUnit("CM", "Centimeters") (
    Symbol() (TranslatedText("Centimeters"))
  )
)

# Add basic definitions to study
study << bd

# Now metadata which will contain all our form and field defs eventually
meta = MetaDataVersion('META1', draft_name)
study << meta

# Protocol contains StudyEventRefs
protocol = Protocol()
# Add some StudyEventRefs
protocol << StudyEventRef("FLDR1", 1, True) # Order 1, Mandatory
# protocol << StudyEventRef("FLDR2", 2, False) # Order 2, Not Mandatory
# protocol << StudyEventRef("AE", 3, True)

meta << protocol

# Add Study Event Defs with some child FormRefs
fldr1 = StudyEventDef("FLDR1", "Folder 1", False, StudyEventDef.SCHEDULED)

fldr1 << FormRef("DM", 1, True)
fldr1 << FormRef("VS", 2, True)

meta << fldr1

meta << StudyEventDef("FLDR2", "Folder 2", False, StudyEventDef.UNSCHEDULED) (
  FormRef("VS", 1, True)
)

meta << StudyEventDef("AE", "Adverse Events", False, StudyEventDef.COMMON) (
  FormRef("AE", 1, False)
)

dm_form = FormDef("DM", "Demography")
dm_form << MdsolHelpText("en", "Some help text for Demography form")
dm_form << MdsolViewRestriction('Data Manager')
dm_form << MdsolEntryRestriction('Batch Upload')

```

(continues on next page)

(continued from previous page)

```

dm_form << ItemGroupRef("DM_IG1", 1)
dm_form << ItemGroupRef("DM_IG2", 2)

# Add to metadata
meta << dm_form

# Define item group
meta << ItemGroupDef("DM_IG1", "DM Item Group 1") (
  MdsolLabelRef("LBL1", 1),
  ItemRef("SEX", 2),
  ItemRef("RACE", 3),
  ItemRef("RACE_OTH", 4),
  ItemRef("DOB", 5),
  ItemRef("AGE", 6)
)

# Add the ItemDefs
meta << ItemDef("SEX", "Gender", DATATYPE_TEXT, 1, control_type=ItemDef.CONTROLTYPE_
↳RADIOBUTTON
  ) (
    Question() (TranslatedText("Gender at Birth")),
    CodeListRef("CL_SEX")
  )
meta << ItemDef("RACE", "Race", DATATYPE_TEXT, 2,
  control_type=ItemDef.CONTROLTYPE_RADIOBUTTON_VERTICAL
  ) (
    Question() (TranslatedText("Race")),
    CodeListRef("CL_RACE")
  )
meta << ItemDef("RACE_OTH", "RaceOther", DATATYPE_TEXT, 20) \
  << Question() << TranslatedText("If Race Other, please specify")

meta << ItemDef("DOB", "DateOfBirth", DATATYPE_DATE, 10,
  control_type=ItemDef.CONTROLTYPE_DATETIME,
  date_time_format="dd/mm/yyyy"
  ) (
    Question() (TranslatedText("Date of Birth")),
    MdsolHelpText("en", "If month unknown, enter January")
  )

meta << ItemDef("AGE", "Age in Years", DATATYPE_INTEGER, 4, significant_digits=3,
↳control_type=ItemDef.CONTROLTYPE_TEXT
  ) (
    Question() (TranslatedText("Age in Years")),
    RangeCheck(RangeCheck.GREATER_THAN_EQUAL_TO, RangeCheck.SOFT) (
      CheckValue("18")
    ),
    RangeCheck(RangeCheck.LESS_THAN_EQUAL_TO, RangeCheck.SOFT) (
      CheckValue("65")
    )
  )

# Add a Label
meta.add(MdsolLabelDef("LBL1", "Label1") (TranslatedText("Please answer all questions.
↳")))

# As well as () and << you can use add()

```

(continues on next page)

(continued from previous page)

```
meta.add(
    CodeList("CL_SEX", "SEX", datatype=DATATYPE_TEXT) (
        CodeListItem("M").add(
            Decode().add(
                TranslatedText("Male"))
        ),
        CodeListItem("F").add(
            Decode().add(
                TranslatedText("Female"))
        ),
    ),
    CodeList("CL_RACE", "RACE", datatype=DATATYPE_TEXT) (
        CodeListItem("Y") (Decode() (TranslatedText("Yes"))),
        CodeListItem("N") (Decode() (TranslatedText("No"))),
    )
)

# Get an lxml document from the ODM object for further manipulation
root = odm.getroot()

# Print a string representation of the ODM document
print(str(odm))
```

Biostats Gateway Requests

`rwslib` provides a module, `rws_requests.biostats_gateway` which provides datasets designed to assist in the creation of SDTM datasets including comments and protocol violations.

Many of the Biostats gateway requests pull data from the Rave Clinical Views. They will not return any data unless Clinical Views are active in Rave.

Read more about Biostats Gateway in the [Rave Web Services documentation](#)

9.1 CVMetaDataRequest(project_name, environment_name)

Authorization is required for this method.

Returns an ODM string representing the metadata of clinical view columns for this study and environment. Note that the structure of clinical views are influenced by all the study versions active for a particular study and environment. If a field is defined as numeric in one version and text in another, the most permissive type (in this case text) will result in the structure of the clinical views.

Calls:

```
https://{ host }/RaveWebServices/studies/{ project name }(environment_name) /  
→ datasets/metadata/regular/?{options}
```

Options:

Option	Description
rawsuf- fix={suffix}	e.g. .RAW means raw field definitions are suffixes with .RAW ex. AE_STDT.RAW
version- item={version suffix}	Add MetaDataVersionOID="<<datetime>>" and additional itemlevel version ItemData element per itemgroup to identify last CV update date and CRF Version each itemgroup was entered under e.g. <ItemData ItemOID="AE.VERSION" Value="16" />
codelistsuf- fix={cl_suffix}	Add name of codelist as an item for each field with data dictionary. e.g. codelistsuffix="CL" results in <ItemData ItemDataOID="DM.SEX.CL" Value="GENDERS"/>
decodesuf- fix={decode suffix}	Add user value of codelist entry as an item for each field with data dictionary. e.g. decode suffix="DECODE" results in <ItemData ItemDataOID="DM.SEX.DECODE" Value="Male"/>

Example:

```
>>> from rwslib import RWSConnection
>>> from rwslib.rws_requests.biostats_gateway import *
>>> r = RWSConnection('https://innovate.mdsol.com', 'username', 'password')
↪ #Authorization required
>>> cv_metadata_odm = r.send_request(CVMetaDataRequest('SIMPLESTUDY', 'TEST',
↪ rawsuffix='RAW'))
>>> print(cv_metadata_odm)
<?xml version="1.0" encoding="utf-8"?><ODM FileType="Snapshot" Granularity="Metadata"
  CreationDateTime="2013-10-07T13:39:33.334-00:00"
  FileOID="0299d82e-8b76-4363-bd3e-2e6985999ce4" ODMVersion="1.3"
  xmlns="http://www.cdisc.org/ns/odm/v1.3">
  <Study OID="SIMPLESTUDY (TEST)">
    <GlobalVariables>
      <StudyName>SIMPLESTUDY (TEST)</StudyName>
      <StudyDescription></StudyDescription>
      <ProtocolName>SIMPLESTUDY</ProtocolName>
    </GlobalVariables>
    <MetaDataVersion OID="2013-06-24T09:47:04.000-00:00" Name="2013-06-
↪ 24T09:47:04.000-00:00">
      <FormDef OID="ENROL" Name="V_SIMPLESTUDY_ENROL" Repeating="Yes">
        <ItemGroupRef ItemGroupOID="ENROL_LOG_LINE" Mandatory="No" />
      </FormDef>
      <FormDef OID="VITAL" Name="V_SIMPLESTUDY_VITAL" Repeating="Yes">
        <ItemGroupRef ItemGroupOID="VITAL_LOG_LINE" Mandatory="No" />
      </FormDef>
      <ItemGroupDef OID="ENROL_LOG_LINE" Name="ENROL_LOG_LINE" Repeating="Yes
↪ ">
        <ItemRef ItemOID="ENROL.SUBID" Mandatory="No" />
        <ItemRef ItemOID="ENROL.BIRTHDT" Mandatory="No" />
        <ItemRef ItemOID="ENROL.BIRTHDTRAW" Mandatory="No" />
      </ItemGroupDef>
      <ItemGroupDef OID="VITAL_LOG_LINE" Name="VITAL_LOG_LINE" Repeating="Yes
↪ ">
        <ItemRef ItemOID="VITAL.VDAT" Mandatory="No" />
        <ItemRef ItemOID="VITAL.VDATRAW" Mandatory="No" />
        <ItemRef ItemOID="VITAL.WEIGHT_KG" Mandatory="No" />
        <ItemRef ItemOID="VITAL.WEIGHT_KGRAW" Mandatory="No" />
        <ItemRef ItemOID="VITAL.HEIGHT_CM" Mandatory="No" />
        <ItemRef ItemOID="VITAL.HEIGHT_CMRAW" Mandatory="No" />
      </ItemGroupDef>
      <ItemDef OID="ENROL.SUBID" Name="SUBID" DataType="text" Length="10" />
      <ItemDef OID="ENROL.BIRTHDT" Name="BIRTHDT" DataType="datetime" />
```

(continues on next page)

(continued from previous page)

```

        <ItemDef OID="ENROL.BIRTHDTRAW" Name="BIRTHDTRAW" DataType="text"
↪Length="11" />
        <ItemDef OID="VITAL.VDAT" Name="VDAT" DataType="datetime" />
        <ItemDef OID="VITAL.VDATRAW" Name="VDATRAW" DataType="text" Length="11"
↪/>
        <ItemDef OID="VITAL.WEIGHT_KG" Name="WEIGHT_KG" DataType="float" Length=
↪"4" SignificantDigits="1" />
        <ItemDef OID="VITAL.WEIGHT_KGRAW" Name="WEIGHT_KGRAW" DataType="text"
↪Length="6" />
        <ItemDef OID="VITAL.HEIGHT_CM" Name="HEIGHT_CM" DataType="float" Length=
↪"4" SignificantDigits="1" />
        <ItemDef OID="VITAL.HEIGHT_CMRAW" Name="HEIGHT_CMRAW" DataType="text"
↪Length="6" />
    </MetaDataVersion>
</Study>
</ODM>

```

9.2 FormDataRequest(project_name, environment_name, dataset_type, form_oid)

Authorization is required for this method.

Retrieve data from Clinical Views for a single form. Data can be extracted from raw or regular views and can be formatted in XML or CSV. If CSV is selected (the default) then the first line contains heading information and the last line of the results contain the string “EOF”. The EOF marker allows you to know that you received the full dataset before any RWS timeout cut off the data stream.

Calls:

```

https://{ host }/RaveWebServices/studies/{ project name }(environment_name)/
↪datasets/{dataset_type}/{form_oid[.csv]}/?{options}

```

Note that dataset_type can be ‘regular’ or ‘raw’. When called with a dataset type of “csv” a .csv is appended to the end of the form oid in the calling URL. When left off, XML will be returned.

Options:

Option	Description
start={ISO 8601 date}	Request changes to this dataset since the start date. Note that CV’s must be set to <i>Include Inactive</i> for this option to work.
dataset_format={csv xml}	Determine the format returned by the request object. CSV is the default, but can also return XML in a simple format.

Example:

```

>>> from rwslib import RWSConnection
>>> from rwslib.rws_requests.biostats_gateway import FormDataRequest
>>> r = RWSConnection('https://innovate.mdsol.com', 'username', 'password')
↪#Authorization required
>>> vital_csv_data = r.send_request(FormDataRequest('SIMPLESTUDY', 'TEST', 'REGULAR',
↪'VITAL', dataset_format="csv"))
>>> print(vital_csv_data)
userid,projectid,project,studyid,environmentName,subjectId,StudySiteId,Subject,siteid,
↪Site,SiteNumber,SiteGroup,instanceId,InstanceName,InstanceRepeatNumber,folderid,
↪Folder,FolderName,FolderSeq,TargetDays,DataPageId,DataPageName,PageRepeatNumber,
↪RecordDate,RecordId,recordposition,RecordActive,SaveTs,MinCreated,MaxUpdated,VDAT,
↪WEIGHT_KG,WEIGHT_KGRAW,HEIGHT_CM,HEIGHT_CM_RAW

```

(continued from previous page)

```

"457","85","SIMPLESTUDY","95","TEST","32112","143","1","120","TESTSITE","TESTSITE",
↪"World","192310","Screening","0","5791","SCREEN","Screening","1.0","", "662502",
↪"Vitals","0","", "1346659","0","1","2013-06-24T09:52:52","2013-06-24T09:52:10","2013-
↪06-24T09:52:10","2013-02-01T00:00:00","2013 Feb 01","2013-02-01T00:00:00","2013","2
↪","1","132.0","132","174.5","174.5"
"457","85","SIMPLESTUDY","95","TEST","32113","143","2","120","TESTSITE","TESTSITE",
↪"World","192311","Screening","0","5791","SCREEN","Screening","1.0","", "662504",
↪"Vitals","0","", "1346661","0","1","2013-06-24T09:52:52","2013-06-24T09:52:11","2013-
↪06-24T09:52:11","2013-02-09T00:00:00","2013 Feb 09","2013-02-09T00:00:00","2013","2
↪","9","82.5","82.5","173.0","173"
"457","85","SIMPLESTUDY","95","TEST","32114","143","3","120","TESTSITE","TESTSITE",
↪"World","192312","Screening","0","5791","SCREEN","Screening","1.0","", "662506",
↪"Vitals","0","", "1346663","0","1","2013-06-24T09:52:52","2013-06-24T09:52:12","2013-
↪06-24T09:52:12","2013-03-14T00:00:00","2013 Mar 14","2013-03-14T00:00:00","2013","3
↪","14","95.2","95.2","152.0","152"
"457","85","SIMPLESTUDY","95","TEST","32115","143","4","120","TESTSITE","TESTSITE",
↪"World","192313","Screening","0","5791","SCREEN","Screening","1.0","", "662508",
↪"Vitals","0","", "1346665","0","1","2013-06-24T09:52:52","2013-06-24T09:52:13","2013-
↪06-24T09:52:13","2013-03-16T00:00:00","2013 Mar 16","2013-03-16T00:00:00","2013","3
↪","16","67.7","67.7","178.0","178"
"457","85","SIMPLESTUDY","95","TEST","32116","143","5","120","TESTSITE","TESTSITE",
↪"World","192314","Screening","0","5791","SCREEN","Screening","1.0","", "662510",
↪"Vitals","0","", "1346667","0","1","2013-06-24T09:52:52","2013-06-24T09:52:15","2013-
↪06-24T09:52:15","2013-03-19T00:00:00","2013 Mar 19","2013-03-19T00:00:00","2013","3
↪","19","81.5","81.5","158.0","158"
"457","85","SIMPLESTUDY","95","TEST","32117","143","6","120","TESTSITE","TESTSITE",
↪"World","192315","Screening","0","5791","SCREEN","Screening","1.0","", "662512",
↪"Vitals","0","", "1346669","0","1","2013-06-24T09:52:52","2013-06-24T09:52:16","2013-
↪06-24T09:52:16","2013-03-24T00:00:00","2013 Mar 24","2013-03-24T00:00:00","2013","3
↪","24","73.9","73.9","180.5","180.5"
"457","85","SIMPLESTUDY","95","TEST","32118","143","7","120","TESTSITE","TESTSITE",
↪"World","192316","Screening","0","5791","SCREEN","Screening","1.0","", "662514",
↪"Vitals","0","", "1346671","0","1","2013-06-24T09:52:52","2013-06-24T09:52:17","2013-
↪06-24T09:52:17","2013-04-06T00:00:00","2013 Apr 06","2013-04-06T00:00:00","2013","4
↪","6","", "175.0","175"
"457","85","SIMPLESTUDY","95","TEST","32119","143","8","120","TESTSITE","TESTSITE",
↪"World","192317","Screening","0","5791","SCREEN","Screening","1.0","", "662516",
↪"Vitals","0","", "1346673","0","1","2013-06-24T09:52:52","2013-06-24T09:52:18","2013-
↪06-24T09:52:18","2013-04-11T00:00:00","2013 Apr 11","2013-04-11T00:00:00","2013","4
↪","11","114.8","114.8","190.0","190"
"457","85","SIMPLESTUDY","95","TEST","32120","143","9","120","TESTSITE","TESTSITE",
↪"World","192318","Screening","0","5791","SCREEN","Screening","1.0","", "662518",
↪"Vitals","0","", "1346675","0","1","2013-06-24T09:52:52","2013-06-24T09:52:19","2013-
↪06-24T09:52:19","2013-04-16T00:00:00","2013 Apr 16","2013-04-16T00:00:00","2013","4
↪","16","68.8","68.8","184.0","184"
"457","85","SIMPLESTUDY","95","TEST","32121","143","10","120","TESTSITE","TESTSITE",
↪"World","192319","Screening","0","5791","SCREEN","Screening","1.0","", "662520",
↪"Vitals","0","", "1346677","0","1","2013-06-24T09:52:52","2013-06-24T09:52:20","2013-
↪06-24T09:52:20","2013-04-26T00:00:00","2013 Apr 26","2013-04-26T00:00:00","2013","4
↪","26","92.7","92.7","175.0","175"
EOF

```

9.3 MetaDataRequest()

Authorization is required for this method.

Returns metadata for all Clinical Views that you have access to in XML or CSV format.

Warning: This could be a large download if you have access to several studies or to a large study. It may be better to request data only for a single study using `ProjectMetadataRequest`

Options:

Option	Description
<code>dataset_format={csv xml}</code>	Determine the format returned by the request object. CSV is the default, but can also return XML in a simple format.

Calls:

```
https://{ host }/RaveWebServices/datasets/ClinicalViewMetadata[.csv]
```

Example:

```
>>> from rwslib import RWSConnection
>>> from rwslib.rws_requests.biostats_gateway import MetaDataRequest
>>> r = RWSConnection('https://innovate.mdsol.com', 'username', 'password')
↳ #Authorization required
>>> all_csv_meta = rave.send_request(MetaDataRequest(dataset_format='xml'))
>>> print(all_csv_meta)
<?xml version="1.0" encoding="UTF-8"?>
<datasets>
  <dataset>
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_ENROL" ordinal="1"
↳ varname="userid" vartype="num" varlength="8" varformat="10." varlabel="Internal id
↳ for the user" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_ENROL" ordinal="2"
↳ varname="projectid" vartype="num" varlength="8" varformat="10." varlabel="projectid
↳ " />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_ENROL" ordinal="3"
↳ varname="project" vartype="char" varlength="255" varformat="$255." varlabel="project
↳ " />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_ENROL" ordinal="4"
↳ varname="studyid" vartype="num" varlength="8" varformat="10." varlabel="Internal id
↳ for the study" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_ENROL" ordinal="5"
↳ varname="environmentName" vartype="char" varlength="20" varformat="$20." varlabel=
↳ "Environment" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_ENROL" ordinal="6"
↳ varname="subjectId" vartype="num" varlength="8" varformat="10." varlabel="Internal
↳ id for the subject" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_ENROL" ordinal="7"
↳ varname="StudySiteId" vartype="num" varlength="8" varformat="10." varlabel=
↳ "Internal id for study site" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_ENROL" ordinal="8"
↳ varname="Subject" vartype="char" varlength="50" varformat="$50." varlabel="Subject
↳ name or identifier" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_ENROL" ordinal="9"
↳ varname="siteid" vartype="num" varlength="8" varformat="10." varlabel="Internal id
↳ for the site" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_ENROL" ordinal="10"
↳ varname="Site" vartype="char" varlength="255" varformat="$255." varlabel="Site name
↳ " />
```

(continues on next page)

(continued from previous page)

```

    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_ENROL" ordinal="11"
↪varname="SiteNumber" vartype="char" varlength="50" varformat="$50." varlabel=
↪"SiteNumber" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_ENROL" ordinal="12"
↪varname="SiteGroup" vartype="char" varlength="40" varformat="$40." varlabel=
↪"SiteGroup" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_ENROL" ordinal="13"
↪varname="instanceId" vartype="num" varlength="8" varformat="10." varlabel="Internal
↪id for the instance" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_ENROL" ordinal="14"
↪varname="InstanceName" vartype="char" varlength="255" varformat="$255." varlabel=
↪"Folder instance name" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_ENROL" ordinal="15"
↪varname="InstanceRepeatNumber" vartype="num" varlength="8" varformat="10." varlabel=
↪"InstanceRepeatNumber" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_ENROL" ordinal="16"
↪varname="folderid" vartype="num" varlength="8" varformat="10." varlabel="Internal
↪id for the folder" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_ENROL" ordinal="17"
↪varname="Folder" vartype="char" varlength="50" varformat="$50." varlabel="Folder OID
↪" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_ENROL" ordinal="18"
↪varname="FolderName" vartype="char" varlength="255" varformat="$255." varlabel=
↪"Folder name" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_ENROL" ordinal="19"
↪varname="FolderSeq" vartype="num" varlength="8" varformat="12.1" varlabel="Folder
↪sequence number" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_ENROL" ordinal="20"
↪varname="TargetDays" vartype="num" varlength="8" varformat="10." varlabel="Target
↪days from study start" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_ENROL" ordinal="21"
↪varname="DataPageId" vartype="num" varlength="8" varformat="10." varlabel="Internal
↪id for data page" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_ENROL" ordinal="22"
↪varname="DataPageName" vartype="char" varlength="255" varformat="$255." varlabel=
↪"eCRF page name" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_ENROL" ordinal="23"
↪varname="PageRepeatNumber" vartype="num" varlength="8" varformat="10." varlabel=
↪"Sequence number of eCRF page in folder" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_ENROL" ordinal="24"
↪varname="RecordDate" vartype="num" varlength="8" varformat="datetime22.3" varlabel=
↪"Clinical date of record (ex: visit date)" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_ENROL" ordinal="25"
↪varname="RecordId" vartype="num" varlength="8" varformat="10." varlabel="Internal
↪id for the record" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_ENROL" ordinal="26"
↪varname="recordposition" vartype="num" varlength="8" varformat="10." varlabel=
↪"Record number" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_ENROL" ordinal="27"
↪varname="RecordActive" vartype="num" varlength="8" varformat="1." varlabel="Is
↪record active" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_ENROL" ordinal="28"
↪varname="SaveTs" vartype="num" varlength="8" varformat="datetime22.3" varlabel=
↪"Timestamp of last save in clinical views" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_ENROL" ordinal="29"
↪varname="MinCreated" vartype="num" varlength="8" varformat="datetime22.3" varlabel=
↪"Earliest data creation time" />

```

(continues on next page)

(continued from previous page)

```

    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_ENROL" ordinal="30"
    ↪varname="MaxUpdated" vartype="num" varlength="8" varformat="datetime22.3" varlabel=
    ↪"Latest data update time" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_ENROL" ordinal="31"
    ↪varname="SUBID" vartype="char" varlength="10" varformat="$10." varlabel="SUBID" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_ENROL" ordinal="32"
    ↪varname="BIRTHDT" vartype="num" varlength="8" varformat="datetime22.3" varlabel=
    ↪"BIRTHDT" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_ENROL" ordinal="33"
    ↪varname="BIRTHDT_RAW" vartype="char" varlength="11" varformat="$11." varlabel=
    ↪"BIRTHDT (Character)" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_ENROL" ordinal="34"
    ↪varname="BIRTHDT_INT" vartype="num" varlength="8" varformat="datetime22.3" varlabel=
    ↪"BIRTHDTInterpolated" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_ENROL" ordinal="35"
    ↪varname="BIRTHDT_YYYY" vartype="num" varlength="8" varformat="4." varlabel=
    ↪"BIRTHDTYear" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_ENROL" ordinal="36"
    ↪varname="BIRTHDT_MM" vartype="num" varlength="8" varformat="2." varlabel=
    ↪"BIRTHDTMonth" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_ENROL" ordinal="37"
    ↪varname="BIRTHDT_DD" vartype="num" varlength="8" varformat="2." varlabel="BIRTHDTDay
    ↪" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_ENROL_RAW" ordinal=
    ↪"1" varname="userid" vartype="num" varlength="8" varformat="10." varlabel="Internal
    ↪id for the user" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_ENROL_RAW" ordinal=
    ↪"2" varname="projectid" vartype="num" varlength="8" varformat="10." varlabel=
    ↪"projectid" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_ENROL_RAW" ordinal=
    ↪"3" varname="project" vartype="char" varlength="255" varformat="$255." varlabel=
    ↪"project" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_ENROL_RAW" ordinal=
    ↪"4" varname="studyid" vartype="num" varlength="8" varformat="10." varlabel=
    ↪"Internal id for the study" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_ENROL_RAW" ordinal=
    ↪"5" varname="environmentName" vartype="char" varlength="20" varformat="$20."
    ↪varlabel="Environment" />
    ....
  </dataset>
</datasets>

```

9.4 ProjectMetaDataRequest(project_name)

Authorization is required for this method.

Returns metadata for all Clinical Views related to a single project in XML or CSV format.

Options:

Option	Description
dataset_format={csv xml}	Determine the format returned by the request object. CSV is the default, but can also return XML in a simple format.

Calls:

```
https://{ host }/RaveWebServices/datasets/ClinicalViewMetadata[.csv]?ProjectName=
↳{project_name}
```

Example:

```
>>> from rwslib import RWSEConnection
>>> from rwslib.rws_requests.biostats_gateway import MetaDataRequest
>>> r = RWSEConnection('https://innovate.mdsol.com', 'username', 'password')
↳#Authorization required
>>> simplestudy_csv_meta = r.send_request(ProjectMetaDataRequest('SIMPLESTUDY'))
>>> print(simplestudy_csv_meta)
projectname,viewname,ordinal,varname,vartype,varlength,varformat,varlabel
"SIMPLESTUDY","V_SIMPLESTUDY_ENROL","1","userid","num","8","10.,""Internal id for the_
↳user"
"SIMPLESTUDY","V_SIMPLESTUDY_ENROL","2","projectid","num","8","10.,""projectid"
"SIMPLESTUDY","V_SIMPLESTUDY_ENROL","3","project","char","255","$255.,""project"
"SIMPLESTUDY","V_SIMPLESTUDY_ENROL","4","studyid","num","8","10.,""Internal id for_
↳the study"
"SIMPLESTUDY","V_SIMPLESTUDY_ENROL","5","environmentName","char","20","$20.,"
↳"Environment"
"SIMPLESTUDY","V_SIMPLESTUDY_ENROL","6","subjectId","num","8","10.,""Internal id for_
↳the subject"
"SIMPLESTUDY","V_SIMPLESTUDY_ENROL","7","StudySiteId","num","8","10.,""Internal id_
↳for study site"
"SIMPLESTUDY","V_SIMPLESTUDY_ENROL","8","Subject","char","50","$50.,""Subject name or_
↳identifier"
"SIMPLESTUDY","V_SIMPLESTUDY_ENROL","9","siteid","num","8","10.,""Internal id for the_
↳site"
"SIMPLESTUDY","V_SIMPLESTUDY_ENROL","10","Site","char","255","$255.,""Site name"
"SIMPLESTUDY","V_SIMPLESTUDY_ENROL","11","SiteNumber","char","50","$50.,""SiteNumber"
"SIMPLESTUDY","V_SIMPLESTUDY_ENROL","12","SiteGroup","char","40","$40.,""SiteGroup"
"SIMPLESTUDY","V_SIMPLESTUDY_ENROL","13","instanceId","num","8","10.,""Internal id_
↳for the instance"
"SIMPLESTUDY","V_SIMPLESTUDY_ENROL","14","InstanceName","char","255","$255.,""Folder_
↳instance name"
"SIMPLESTUDY","V_SIMPLESTUDY_ENROL","15","InstanceRepeatNumber","num","8","10.,"
↳"InstanceRepeatNumber"
"SIMPLESTUDY","V_SIMPLESTUDY_ENROL","16","folderid","num","8","10.,""Internal id for_
↳the folder"
"SIMPLESTUDY","V_SIMPLESTUDY_ENROL","17","Folder","char","50","$50.,""Folder OID"
"SIMPLESTUDY","V_SIMPLESTUDY_ENROL","18","FolderName","char","255","$255.,""Folder_
↳name"
"SIMPLESTUDY","V_SIMPLESTUDY_ENROL","19","FolderSeq","num","8","12.1","Folder_
↳sequence number"
"SIMPLESTUDY","V_SIMPLESTUDY_ENROL","20","TargetDays","num","8","10.,""Target days_
↳from study start"
"SIMPLESTUDY","V_SIMPLESTUDY_ENROL","21","DataPageId","num","8","10.,""Internal id_
↳for data page"
"SIMPLESTUDY","V_SIMPLESTUDY_ENROL","22","DataPageName","char","255","$255.,""eCRF_
↳page name"
"SIMPLESTUDY","V_SIMPLESTUDY_ENROL","23","PageRepeatNumber","num","8","10.,""Sequence_
↳number of eCRF page in folder"
"SIMPLESTUDY","V_SIMPLESTUDY_ENROL","24","RecordDate","num","8","datetime22.3",
↳"Clinical date of record (ex: visit date)"
"SIMPLESTUDY","V_SIMPLESTUDY_ENROL","25","RecordId","num","8","10.,""Internal id for_
↳the record"
"SIMPLESTUDY","V_SIMPLESTUDY_ENROL","26","recordposition","num","8","10.,""Record_
↳number"
```

(continues on next page)

(continued from previous page)

```
"SIMPLESTUDY", "V_SIMPLESTUDY_ENROL", "27", "RecordActive", "num", "8", "1.", "Is record_
↪active"
...many more lines
EOF
```

9.5 ViewMetaDataRequest(view_name)

Authorization is required for this method.

Returns metadata for a single clinical view in XML or CSV format. A clinical view name will have the format:

```
V_{projectname}_{formoid}
```

for standard views and:

```
prod.V_{projectname}_{formoid}
```

for production-only views if these are set to be created by the Rave Clinical View Settings.

Options:

Option	Description
dataset_format={csv xml}	Determine the format returned by the request object. CSV is the default, but can also return XML in a simple format.

Calls:

```
https://{ host }/RaveWebServices/datasets/ClinicalViewMetadata[.csv]?ViewName={view_
↪name}
```

Example:

```
>>> from rwslib import RWSConnection
>>> from rwslib.rws_requests.biostats_gateway import MetaDataRequest
>>> r = RWSConnection('https://innovate.mdsol.com', 'username', 'password')
↪#Authorization required
>>> ss_vital_meta = r.send_request(ViewMetaDataRequest("V_SIMPLESTUDY_VITAL", dataset_
↪format='xml'))
>>> print(ss_vital_meta)
<?xml version="1.0" encoding="UTF-8"?>
<datasets>
  <dataset>
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_VITAL" ordinal="1"
↪varname="userid" vartype="num" varlength="8" varformat="10." varlabel="Internal id_
↪for the user" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_VITAL" ordinal="2"
↪varname="projectid" vartype="num" varlength="8" varformat="10." varlabel="projectid
↪" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_VITAL" ordinal="3"
↪varname="project" vartype="char" varlength="255" varformat="$255." varlabel="project
↪" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_VITAL" ordinal="4"
↪varname="studyid" vartype="num" varlength="8" varformat="10." varlabel="Internal id_
↪for the study" />
```

(continues on next page)

(continued from previous page)

```

    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_VITAL" ordinal="5"
    ↪varname="environmentName" vartype="char" varlength="20" varformat="$20." varlabel=
    ↪"Environment" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_VITAL" ordinal="6"
    ↪varname="subjectId" vartype="num" varlength="8" varformat="10." varlabel="Internal
    ↪id for the subject" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_VITAL" ordinal="7"
    ↪varname="StudySiteId" vartype="num" varlength="8" varformat="10." varlabel=
    ↪"Internal id for study site" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_VITAL" ordinal="8"
    ↪varname="Subject" vartype="char" varlength="50" varformat="$50." varlabel="Subject
    ↪name or identifier" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_VITAL" ordinal="9"
    ↪varname="siteid" vartype="num" varlength="8" varformat="10." varlabel="Internal id
    ↪for the site" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_VITAL" ordinal="10"
    ↪varname="Site" vartype="char" varlength="255" varformat="$255." varlabel="Site name
    ↪" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_VITAL" ordinal="11"
    ↪varname="SiteNumber" vartype="char" varlength="50" varformat="$50." varlabel=
    ↪"SiteNumber" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_VITAL" ordinal="12"
    ↪varname="SiteGroup" vartype="char" varlength="40" varformat="$40." varlabel=
    ↪"SiteGroup" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_VITAL" ordinal="13"
    ↪varname="instanceId" vartype="num" varlength="8" varformat="10." varlabel="Internal
    ↪id for the instance" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_VITAL" ordinal="14"
    ↪varname="InstanceName" vartype="char" varlength="255" varformat="$255." varlabel=
    ↪"Folder instance name" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_VITAL" ordinal="15"
    ↪varname="InstanceRepeatNumber" vartype="num" varlength="8" varformat="10." varlabel=
    ↪"InstanceRepeatNumber" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_VITAL" ordinal="16"
    ↪varname="folderid" vartype="num" varlength="8" varformat="10." varlabel="Internal
    ↪id for the folder" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_VITAL" ordinal="17"
    ↪varname="Folder" vartype="char" varlength="50" varformat="$50." varlabel="Folder OID
    ↪" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_VITAL" ordinal="18"
    ↪varname="FolderName" vartype="char" varlength="255" varformat="$255." varlabel=
    ↪"Folder name" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_VITAL" ordinal="19"
    ↪varname="FolderSeq" vartype="num" varlength="8" varformat="12.1" varlabel="Folder
    ↪sequence number" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_VITAL" ordinal="20"
    ↪varname="TargetDays" vartype="num" varlength="8" varformat="10." varlabel="Target
    ↪days from study start" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_VITAL" ordinal="21"
    ↪varname="DataPageId" vartype="num" varlength="8" varformat="10." varlabel="Internal
    ↪id for data page" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_VITAL" ordinal="22"
    ↪varname="DataPageName" vartype="char" varlength="255" varformat="$255." varlabel=
    ↪"eCRF page name" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_VITAL" ordinal="23"
    ↪varname="PageRepeatNumber" vartype="num" varlength="8" varformat="10." varlabel=
    ↪"Sequence number of eCRF page in folder" />

```

(continues on next page)

(continued from previous page)

```

    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_VITAL" ordinal="24"
    ↪varname="RecordDate" vartype="num" varlength="8" varformat="datetime22.3" varlabel=
    ↪"Clinical date of record (ex: visit date)" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_VITAL" ordinal="25"
    ↪varname="RecordId" vartype="num" varlength="8" varformat="10." varlabel="Internal
    ↪id for the record" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_VITAL" ordinal="26"
    ↪varname="recordposition" vartype="num" varlength="8" varformat="10." varlabel=
    ↪"Record number" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_VITAL" ordinal="27"
    ↪varname="RecordActive" vartype="num" varlength="8" varformat="1." varlabel="Is
    ↪record active" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_VITAL" ordinal="28"
    ↪varname="SaveTs" vartype="num" varlength="8" varformat="datetime22.3" varlabel=
    ↪"Timestamp of last save in clinical views" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_VITAL" ordinal="29"
    ↪varname="MinCreated" vartype="num" varlength="8" varformat="datetime22.3" varlabel=
    ↪"Earliest data creation time" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_VITAL" ordinal="30"
    ↪varname="MaxUpdated" vartype="num" varlength="8" varformat="datetime22.3" varlabel=
    ↪"Latest data update time" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_VITAL" ordinal="31"
    ↪varname="VDAT" vartype="num" varlength="8" varformat="datetime22.3" varlabel="VDAT"
    ↪/>
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_VITAL" ordinal="32"
    ↪varname="VDAT_RAW" vartype="char" varlength="11" varformat="$11." varlabel=
    ↪"VDAT (Character)" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_VITAL" ordinal="33"
    ↪varname="VDAT_INT" vartype="num" varlength="8" varformat="datetime22.3" varlabel=
    ↪"VDATInterpolated" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_VITAL" ordinal="34"
    ↪varname="VDAT_YYYY" vartype="num" varlength="8" varformat="4." varlabel="VDATYear" /
    ↪>
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_VITAL" ordinal="35"
    ↪varname="VDAT_MM" vartype="num" varlength="8" varformat="2." varlabel="VDATMonth" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_VITAL" ordinal="36"
    ↪varname="VDAT_DD" vartype="num" varlength="8" varformat="2." varlabel="VDATDay" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_VITAL" ordinal="37"
    ↪varname="WEIGHT_KG" vartype="num" varlength="8" varformat="5.1" varlabel="WEIGHT_KG
    ↪" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_VITAL" ordinal="38"
    ↪varname="WEIGHT_KG_RAW" vartype="char" varlength="6" varformat="$6." varlabel=
    ↪"WEIGHT_KG (Character)" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_VITAL" ordinal="39"
    ↪varname="HEIGHT_CM" vartype="num" varlength="8" varformat="5.1" varlabel="HEIGHT_CM
    ↪" />
    <record projectname="SIMPLESTUDY" viewname="V_SIMPLESTUDY_VITAL" ordinal="40"
    ↪varname="HEIGHT_CM_RAW" vartype="char" varlength="6" varformat="$6." varlabel=
    ↪"HEIGHT_CM (Character)" />
  </dataset>
</datasets>

```

9.6 CommentDataRequest(project_name, environment_name)

Provides all comments from Rave in CSV or XML format.

Options:

Option	Description
dataset_format={csv xml}	Determine the format returned by the request object. CSV is the default, but can also return XML in a simple format.

Calls:

```
https://{ host }/RaveWebServices/datasets/SDTMComments[.csv]?studyid={project_name} (
↳ {environment_name})
```

Example:

```
>>> from rwslib import RWSConnection
>>> from rwslib.rws_requests.biostats_gateway import CommentDataRequest
>>> r = RWSConnection('https://innovate.mdsol.com', 'username', 'password')
↳ #Authorization required
>>> r.send_request(CommentDataRequest("SIMPLESTUDY", "TEST"))
ProjectName, EnvironmentName, SiteNumber, SubjectName, SubjectID, InstanceName, InstanceID,
↳ InstanceRepeatNumber, DataPageName, DatapageID, PageRepeatNumber, Datapointid, FormOID,
↳ RecordID, RecordPosition, FieldOID, Text, Updated, DataActive, RecordActive
SIMPLESTUDY, TEST, TESTSITE, 3, 32114, "", "", "", Enrol, 662507, 0, 2289018,
↳ ENROL, 1346664, 0, BIRTHDT, This subject was late, 10/7/2013 3:50:18 PM, True
↳ , True
SIMPLESTUDY, TEST, TESTSITE, 8, 32119, Screening, 192317, 0, Vitals, 662516
↳ , 0, 2289046, VITAL, 1346673, 0, HEIGHT_CM, This weight was not taken during
↳ the visit. It was called in by the subject., 10/7/2013 3:50:46 PM, True, True
EOF
```

9.7 ProtocolDeviationsRequest(project_name, environment_name)

Provides all ProtocolDeviations from Rave in CSV or XML format.

Options:

Option	Description
dataset_format={csv xml}	Determine the format returned by the request object. CSV is the default, but can also return XML in a simple format.

Calls:

```
https://{ host }/RaveWebServices/datasets/SDTMPProtocolDeviations[.csv]?studyid=
↳ {project_name} ({environment_name})
```

Example:

```
>>> from rwslib import RWSConnection
>>> from rwslib.rws_requests.biostats_gateway import ProtocolDeviationsRequest
>>> r = RWSConnection('https://innovate.mdsol.com', 'username', 'password')
↳ #Authorization required
>>> r.send_request(ProtocolDeviationsRequest("SIMPLESTUDY", "TEST"))
ProjectName, EnvironmentName, SiteNumber, SubjectName, SubjectID, InstanceName, InstanceID,
↳ InstanceRepeatNumber, DataPageName, DatapageID, PageRepeatNumber, Datapointid, FormOID,
↳ RecordID, RecordPosition, FieldOID, Text, Updated, PDClass, PDCode, DataActive, RecordActive
```

(continues on next page)

(continued from previous page)

```
"SIMPLESTUDY","TEST","TESTSITE","8","32119","","","Enrol","662517","0","2289043",
↳ "ENROL","1346674","0","BIRTHDT","Inc/Exc criteria do not seem to be met for this_
↳ subject. Too young.","10/7/2013 3:16:29 PM","Incl/Excl Criteria not met","Deviation
↳ ","True","True"
"SIMPLESTUDY","TEST","TESTSITE","8","32119","Screening","192317","0","Vitals","662516
↳ ","0","2289046","VITAL","1346673","0","HEIGHT_CM","Height/Weight outside range!",
↳ "10/7/2013 3:21:01 PM","Incl/Excl Criteria not met","violation","True","True"
EOF
```

9.8 DataDictionariesRequest(project_name, environment_name)

Provides all Data Dictionaries from a study in CSV or XML format.

Options:

Option	Description
dataset_format={csv xml}	Determine the format returned by the request object. CSV is the default, but can also return XML in a simple format.

Calls:

```
https://{ host }/RaveWebServices/datasets/SDTMDDataDictionaries[.csv]?studyid=
↳ {project_name}({environment_name})
```

Example:

```
>>> from rwslib import RWSConnection
>>> from rwslib.rws_requests.biostats_gateway import DataDictionariesRequest
>>> r = RWSConnection('https://innovate.mdsol.com', 'username', 'password')
↳ #Authorization required
>>> r.send_request(DataDictionariesRequest("CDASH Forms", "PROD", dataset_format="xml
↳ "))
<?xml version="1.0" encoding="UTF-8"?>
<datasets>
  <dataset>
    <record studyid="CDASH Forms(Prod)" ProjectName="CDASH Forms" EnvironmentName=
↳ "Prod" CRFVersionID="18" DictionaryName="AE Action Taken" CodedValue="8" UserValue=
↳ "Other (specify)" userid="457" />
    <record studyid="CDASH Forms(Prod)" ProjectName="CDASH Forms" EnvironmentName=
↳ "Prod" CRFVersionID="18" DictionaryName="AE Action Taken" CodedValue="7" UserValue=
↳ "Unknown" userid="457" />
    <record studyid="CDASH Forms(Prod)" ProjectName="CDASH Forms" EnvironmentName=
↳ "Prod" CRFVersionID="18" DictionaryName="AE Action Taken" CodedValue="6" UserValue=
↳ "Not Applicable" userid="457" />
    <record studyid="CDASH Forms(Prod)" ProjectName="CDASH Forms" EnvironmentName=
↳ "Prod" CRFVersionID="18" DictionaryName="AE Action Taken" CodedValue="5" UserValue=
↳ "Drug Withdrawn" userid="457" />
    <record studyid="CDASH Forms(Prod)" ProjectName="CDASH Forms" EnvironmentName=
↳ "Prod" CRFVersionID="18" DictionaryName="AE Action Taken" CodedValue="4" UserValue=
↳ "Drug Interrupted" userid="457" />
    <record studyid="CDASH Forms(Prod)" ProjectName="CDASH Forms" EnvironmentName=
↳ "Prod" CRFVersionID="18" DictionaryName="AE Action Taken" CodedValue="3" UserValue=
↳ "Dose Reduced" userid="457" />
```

(continues on next page)

(continued from previous page)

```
<record studyid="CDASH Forms (Prod)" ProjectName="CDASH Forms" EnvironmentName=
↪ "Prod" CRFVersionID="18" DictionaryName="AE Action Taken" CodedValue="2" UserValue=
↪ "Dose not Changed" userid="457" />
  <record studyid="CDASH Forms (Prod)" ProjectName="CDASH Forms" EnvironmentName=
↪ "Prod" CRFVersionID="18" DictionaryName="AE Action Taken" CodedValue="1" UserValue=
↪ "Dose Increased" userid="457" />
    <record studyid="CDASH Forms (Prod)" ProjectName="CDASH Forms" EnvironmentName=
↪ "Prod" CRFVersionID="18" DictionaryName="AE Severity" CodedValue="3" UserValue=
↪ "Severe" userid="457" />
      <record studyid="CDASH Forms (Prod)" ProjectName="CDASH Forms" EnvironmentName=
↪ "Prod" CRFVersionID="18" DictionaryName="AE Severity" CodedValue="2" UserValue=
↪ "Moderate" userid="457" />
        <record studyid="CDASH Forms (Prod)" ProjectName="CDASH Forms" EnvironmentName=
↪ "Prod" CRFVersionID="18" DictionaryName="AE Severity" CodedValue="1" UserValue="Mild
↪ " userid="457" />
        ....
      </dataset>
    </datasets>
```

CHAPTER 10

ODM Adapter Requests

The ODM Adapter module provides Request implementations for the Rave Web Service ODM Adapter URLs. These include:

- Clinical Audit Records Dataset
- Version Folders Dataset
- Sites Dataset
- Users Dataset
- Signature Definitions Dataset

Read more about ODM Adapter in the [Rave Web Services documentation](#)

10.1 AuditRecordsRequest(project_name, environment_name)

Authorization is required for this request.

Returns audit data in ODM format. Since the audit trail is a large table which cannot be downloaded in a single request, this dataset returns headers that tell you what the next page of data to request is. This allows you to make further requests until all data from the dataset has been received.

Calls:

```
https://{ host }/RaveWebServices/datasets/ClinicalAuditRecords.odm/?studyoid=
→ {project_name} ({environment_name}) &startid={startid} &per_page={per_page}
```

Options:

Option	Description
startid=1	The audit ID to start on. Defaults to 1. The first Audit ID
per_page=100	How many audits to return per request. Default is 100.

Example:

```
>>> from rwslib import RWSConnection
>>> from rwslib.rws_requests.odm_adapter import *
>>> r = RWSConnection('https://innovate.mdsol.com', 'username', 'password')
>>> audits = r.send_request(AuditRecordsRequest('MEDIFLEX', 'DEV'))
>>> r.last_request.headers
{....'Link': '<https://innovate.mdsol.com/RaveWebServices/datasets/
↳ClinicalAuditRecords.odm?studyoid=MEDIFLEX%28DEV%29&startid=3842&per_page=1000>;
↳rel="next"',...., 'Content-Type': 'text/xml'}
```

Note that the `audit_event` example in the `extras` package of `rwslib` provides a parser for the content of audit records and a class to simplify the consumption of this web service. See the README for that project in the `extras/audit_event` package.

10.2 VersionFoldersRequest(project_name, environment_name)

Authorization is required for this request.

Returns a dataset in ODM format which represents the folders in-use. This is useful because the standard ODM Metadata can only return the primary Matrix (folder structure) of Rave. `VersionFoldersRequest` provides all possible folders.

Calls:

```
https://{ host }/RaveWebServices/datasets/VersionFolders.odm/?studyoid={project_
↳name}({environment_name})
```

Example:

```
>>> from rwslib import RWSConnection
>>> from rwslib.rws_requests.odm_adapter import *
>>> r = RWSConnection('https://innovate.mdsol.com', 'username', 'password')
>>> r.send_request(VersionFoldersRequest('MEDIFLEX', 'DEV'))
<?xml version="1.0" encoding="UTF-8"?>
<ODM xmlns="http://www.cdisc.org/ns/odm/v1.3" xmlns:mdsol="http://www.mdsol.com/ns/
↳odm/metadata" ODMVersion="1.3" Granularity="Metadata" FileType="Snapshot" FileOID=
↳"2f4f7fdf-f5db-4150-bf41-79060d4b5ffb" CreationDateTime="2016-04-13T13:53:04.000-
↳00:00">
  <Study OID="Mediflex (Dev)">
    <GlobalVariables>
      <StudyName>Mediflex (Dev)</StudyName>
      <StudyDescription />
      <ProtocolName>Mediflex</ProtocolName>
    </GlobalVariables>
    <MetaDataVersion OID="16" Name="1" mdsol:PrimaryFormOID="EN">
      <Protocol>
        <StudyEventRef StudyEventOID="YEAR1" OrderNumber="7" Mandatory="No"
↳mdsol:StudyEventDefName="Year 01" mdsol:StudyEventDefType="Common"
↳mdsol:StudyEventDefRepeating="No" />
      </Protocol>
    </MetaDataVersion>
    <MetaDataVersion OID="16" Name="1" mdsol:MatrixOID="ADDCYCLE"
↳mdsol:PrimaryFormOID="EN">
      <Protocol>
        <StudyEventRef StudyEventOID="ADDCYCLE" OrderNumber="11" Mandatory="No"
↳mdsol:StudyEventDefName="Additional Cycle" mdsol:StudyEventDefType="Common"
↳mdsol:StudyEventDefRepeating="No" />
```

(continues on next page)

(continued from previous page)

```

        </Protocol>
    </MetaDataVersion>
    <MetaDataVersion OID="16" Name="1" mdsol:DefaultMatrixOID="BASE"
↪mdsol:PrimaryFormOID="EN">
        <Protocol>
            <StudyEventRef StudyEventOID="SCREEN" OrderNumber="1" Mandatory="No"
↪mdsol:StudyEventDefName="Screening" mdsol:StudyEventDefType="Common"
↪mdsol:StudyEventDefRepeating="No" />
            <StudyEventRef StudyEventOID="VISIT01" OrderNumber="2" Mandatory="No"
↪mdsol:StudyEventDefName="Visit 01" mdsol:StudyEventDefType="Common"
↪mdsol:StudyEventDefRepeating="No" />
            <StudyEventRef StudyEventOID="VISIT02" OrderNumber="3" Mandatory="No"
↪mdsol:StudyEventDefName="Visit 02" mdsol:StudyEventDefType="Common"
↪mdsol:StudyEventDefRepeating="No" />
            <StudyEventRef StudyEventOID="VISIT03" OrderNumber="4" Mandatory="No"
↪mdsol:StudyEventDefName="Visit 03" mdsol:StudyEventDefType="Common"
↪mdsol:StudyEventDefRepeating="No" />
            <StudyEventRef StudyEventOID="VISIT04" OrderNumber="5" Mandatory="No"
↪mdsol:StudyEventDefName="Visit 04" mdsol:StudyEventDefType="Common"
↪mdsol:StudyEventDefRepeating="No" />
            <StudyEventRef StudyEventOID="CYCLE1" OrderNumber="8" Mandatory="No"
↪mdsol:StudyEventDefName="Cycle 01" mdsol:StudyEventDefType="Common"
↪mdsol:StudyEventDefRepeating="No" />
            <StudyEventRef StudyEventOID="CYCLE2" OrderNumber="9" Mandatory="No"
↪mdsol:StudyEventDefName="Cycle 02" mdsol:StudyEventDefType="Common"
↪mdsol:StudyEventDefRepeating="No" />
        </Protocol>
    </MetaDataVersion>
    <MetaDataVersion OID="16" Name="1" mdsol:MatrixOID="EXTCYCLE"
↪mdsol:PrimaryFormOID="EN">
        <Protocol>
            <StudyEventRef StudyEventOID="EXTCYCLE" OrderNumber="10" Mandatory="No"
↪mdsol:StudyEventDefName="Extended Cycle" mdsol:StudyEventDefType="Common"
↪mdsol:StudyEventDefRepeating="No" />
        </Protocol>
    </MetaDataVersion>
    <MetaDataVersion OID="16" Name="1" mdsol:MatrixOID="UNSCHEDULED"
↪mdsol:PrimaryFormOID="EN">
        <Protocol>
            <StudyEventRef StudyEventOID="UNSCHEDULED" OrderNumber="6" Mandatory="No"
↪mdsol:StudyEventDefName="Visit" mdsol:StudyEventDefType="Common"
↪mdsol:StudyEventDefRepeating="No" />
        </Protocol>
    </MetaDataVersion>
    <MetaDataVersion OID="16" Name="1" mdsol:MatrixOID="VISITS"
↪mdsol:PrimaryFormOID="EN">
        <Protocol>
            <StudyEventRef StudyEventOID="VISIT01" OrderNumber="2" Mandatory="No"
↪mdsol:StudyEventDefName="Visit 01" mdsol:StudyEventDefType="Common"
↪mdsol:StudyEventDefRepeating="No" />
            mdsol:MatrixOID="VISITS"
            <StudyEventRef StudyEventOID="VISIT02" OrderNumber="3" Mandatory="No"
↪mdsol:StudyEventDefName="Visit 02" mdsol:StudyEventDefType="Common"
↪mdsol:StudyEventDefRepeating="No" />
            mdsol:MatrixOID="VISITS"
            <StudyEventRef StudyEventOID="VISIT03" OrderNumber="4" Mandatory="No"
↪mdsol:StudyEventDefName="Visit 03" mdsol:StudyEventDefType="Common"
↪mdsol:StudyEventDefRepeating="No" />

```

(continues on next page)

(continued from previous page)

```

        </Protocol>
    </MetaDataVersion>
    <MetaDataVersion OID="23" Name="2" mdsol:PrimaryFormOID="EN">
        <Protocol>
            <StudyEventRef StudyEventOID="YEAR1" OrderNumber="7" Mandatory="No"
↳ mdsol:StudyEventDefName="Year 01" mdsol:StudyEventDefType="Common"
↳ mdsol:StudyEventDefRepeating="No" />
        </Protocol>
    </MetaDataVersion>
    <MetaDataVersion OID="23" Name="2" mdsol:MatrixOID="ADDCYCLE"
↳ mdsol:PrimaryFormOID="EN">
        <Protocol>
            <StudyEventRef StudyEventOID="ADDCYCLE" OrderNumber="11" Mandatory="No"
↳ mdsol:StudyEventDefName="Additional Cycle" mdsol:StudyEventDefType="Common"
↳ mdsol:StudyEventDefRepeating="No" />
        </Protocol>
    </MetaDataVersion>
    ....
</Study>
</ODM>

```

10.3 SitesMetadataRequest()

Authorization is required for this request.

Returns an ODM AdminData document which lists all sites along with their metadata versions and effective dates. Optionally can take a project name and an environment to filter the list only to that study/environment.

To find the current active metadata version for a study/site you will need to sort the metadata versions for the site by the effective date and take the latest one.

Calls:

```

https://{ host }/RaveWebServices/datasets/Sites.odm/[?studyoid={project_name} (
↳ {environment_name})]

```

Options:

Option	Description
project_name={projectname}	Project to filter the result set to (recommended)
environment_name={environment}	Environment to filter the result set to

If used, the project_name and environmen_namet must both be supplied or an error will result.

Example:

```

>>> from rwslib import RWSEConnection
>>> from rwslib.rws_requests.odm_adapter import *
>>> r = RWSEConnection('https://innovate.mdsol.com', 'username', 'password')
>>> r.send_request(SitesMetadataRequest('Mediflex','DEV'))
<?xml version="1.0" encoding="UTF-8"?>
<ODM xmlns="http://www.cdisc.org/ns/odm/v1.3" xmlns:mdsol="http://www.mdsol.com/ns/
↳ odm/metadata" ODMVersion="1.3" FileType="Snapshot" FileOID="b4e9560d-0e67-4788-aa04-
↳ 9b9dfe5d740b" CreationDateTime="2016-04-13T13:59:25">

```

(continues on next page)

(continued from previous page)

```

    <AdminData>
      <Location OID="4567" Name="Uxbridge Medical Centre" LocationType="Site"
↳mdsol:Active="Yes">
        <MetaDataVersionRef StudyOID="Mediflex (Dev)" MetaDataVersionOID="23"
↳EffectiveDate="2009-04-20" mdsol:StudySiteNumber="" />
      </Location>
      <Location OID="MEDI0001" Name="Medidata" LocationType="Site" mdsol:Active="Yes">
        <MetaDataVersionRef StudyOID="Mediflex (Dev)" MetaDataVersionOID="1015"
↳EffectiveDate="2013-05-02" mdsol:StudySiteNumber="" />
      </Location>
    </AdminData>
  </ODM>

```

10.4 UsersRequest(project_name, environment_name, location_oid=None)

Authorization is required for this request.

Returns an ODM AdminData document listing all users associated with a study with optional filtering to a single location.

Calls:

```

https://{host}/RaveWebServices/datasets/Users.odm/?studyoid={project_name} (
↳{environment_name}) [&locationoid={locationoid}]

```

Options:

Option	Description
locationoid	A site number from Rave that uniquely identifies a site

Example:

```

>>> from rwslib import RWSConnection
>>> from rwslib.rws_requests.odm_adapter import *
>>> r = RWSConnection('https://innovate.mdsol.com', 'username', 'password')
>>> r.send_request(UsersRequest('SIMPLESTUDY', 'TEST'))
<?xml version="1.0" encoding="UTF-8"?>
<ODM xmlns="http://www.cdsc.org/ns/odm/v1.3" xmlns:mdsol="http://www.mdsol.com/ns/
↳odm/metadata" ODMVersion="1.3" FileType="Snapshot" FileOID="7c2ef3a2-8df5-405c-bacc-
↳c3ae220ed2bd" CreationDateTime="2016-04-13T14:20:01">
  <AdminData>
    <User OID="isparks_other_account" UserType="Other" mdsol:Active="Yes"
↳mdsol:UserGroup="Test" mdsol:SiteGroup="World" mdsol:UserRole="Batch Upload">
      <LoginName>isparks_other_account</LoginName>
      <DisplayName>Ian Sparks</DisplayName>
      <FullName>Ian Sparks</FullName>
      <FirstName>Ian</FirstName>
      <LastName>Sparks</LastName>
      <Address />
      <Email>isparks@mdsol.com</Email>
      <Fax />
      <Phone />

```

(continues on next page)

(continued from previous page)

```

        <LocationRef LocationOID="TESTSITE" />
        <LocationRef LocationOID="TESTSITE2" />
    </User>
    <!-- More Users here -->
</AdminData>
</ODM>

```

10.5 SignatureDefinitionsRequest(project_name)

Authorization is required for this request.

Returns an ODM AdminData document listing the definition of all signatures for this study (across all environments). This allows you to match signature audits to their definitions and know in what context a signature was being made.

Calls:

```
https://{ host }/RaveWebServices/datasets/Signatures.odm/?studyid={project_name}
```

Example:

```

>>> from rwslib import RWSConnection
>>> from rwslib.rws_requests.odm_adapter import *
>>> r = RWSConnection('https://innovate.mdsol.com', 'username', 'password')
>>> r.send_request(SignatureDefinitionsRequest('SIMPLESTUDY'))
<?xml version="1.0" encoding="UTF-8"?>
<ODM xmlns="http://www.cdisc.org/ns/odm/v1.3" xmlns:mdsol="http://www.mdsol.com/ns/
->odm/metadata" ODMVersion="1.3"
  FileType="Snapshot" FileOID="1d885ac2-ffc7-4b10-a2ab-bfc056a1d57e"
->CreationDateTime="2016-04-13T16:08:44">
  <AdminData>
    <SignatureDef Methodology="Electronic" OID="2866" mdsol:Study="SIMPLESTUDY">
      <Meaning>Approval</Meaning>
      <LegalReason>I hereby confirm that all data is accurate to the best of my
->knowledge.</LegalReason>
    </SignatureDef>
    <SignatureDef Methodology="Electronic" OID="2867" mdsol:Study="SIMPLESTUDY">
      <Meaning>Approval</Meaning>
      <LegalReason>I hereby confirm that all data is accurate to the best of my
->knowledge.</LegalReason>
    </SignatureDef>
    <SignatureDef Methodology="Electronic" OID="2919" mdsol:Study="SIMPLESTUDY">
      <Meaning>Approval</Meaning>
      <LegalReason>I hereby confirm that all data is accurate to the best of my
->knowledge.</LegalReason>
    </SignatureDef>
    <SignatureDef Methodology="Electronic" OID="2976" mdsol:Study="SIMPLESTUDY">
      <Meaning>Approval</Meaning>
      <LegalReason>I hereby confirm that all data is accurate to the best of my
->knowledge.</LegalReason>
    </SignatureDef>
  </AdminData>
</ODM>

```

CHAPTER 11

rwscmd

rwscmd is a command-line tool providing convenient access to Rave WebServices, via rwslib.

11.1 Usage

```
rwscmd [OPTIONS] URL COMMAND [ARGS]
Options:
  -u, --username TEXT           Rave login
  -p, --password TEXT           Rave password
  --virtual_dir TEXT            RWS virtual directory, defaults to RaveWebServices
  --raw / --list                Display raw xml response from RWS or human-readable
  ↪list, defaults to list
  -v, --verbose / -s, --silent
  -o, --output FILENAME         Write output to file
  --help                        Show this message and exit.

Commands:
  autofill  Request enterable data for a subject,...
  data      List EDC data for [STUDY] [ENV] [SUBJECT]
  direct    Make direct call to RWS, bypassing rwslib
  metadata  List metadata for [PROJECT] [VERSION]
  post      Post ODM clinical data
  version   Display RWS version
```

11.2 Examples

```
$ rwscmd innovate version
  Username: anewbigging
  Password:
  1.15.0
```

(continues on next page)

(continued from previous page)

```
$ export RWSCMD_USERNAME=anewbigging
$ export RWSCMD_PASSWORD=*****

$ rwscmd innovate version
  1.15.0

$ rwscmd innovate data
  ATN01 (Prod)
  Medidata (Prod)
  Mediflex (Prod)
  Mediflex (Dev)

$ rwscmd innovate data Mediflex Prod
  0004-bbc-003
  001 aaa
  001 ADS
```

12.1 rwslib

exception `rwslib.AuthorizationException`

Raised if a request requires authorization but no authorization header is provided

class `rwslib.RWSConnection` (*domain*, *username=None*, *password=None*, *auth=None*, *virtual_dir='RaveWebServices'*)

A connection to RWS

Create a connection to Rave

Parameters

- **domain** (*str*) – Rave URL Name
- **username** (*str*) – Rave User Login
- **password** (*str*) – Rave User password
- **auth** (*str*) – Authentication tuple (usually something like (*username*, *password*))
- **virtual_dir** (*str*) – Name of the Rave Web Services prefix (usually *RaveWebServices*, but can be customised)

Note: If the *domain* does not start with `http` then it is assumed to be the name of the Medidata url and `https://` will be added as a prefix and `.mdsol.com` will be added as a postfix.

- `innovate` => `https://innovate.mdsol.com`
 - `http://mytest` => `http://mytest`
-

send_request (*request_object*, *timeout=None*, *retries=1*, ***kwargs*)

Send request to RWS endpoint. The request object passed provides the URL endpoint and the HTTP method. Takes the text response from RWS and allows the request object to modify it for return. This allows the request object to return text, an XML document object, a CSV file or anything else that can

be generated from the text response from RWS. A timeout, in seconds, can be optionally passed into `send_request`.

12.2 Rave Web Services Request Objects

Core request classes that implement calls available in all versions of RWS.

RWSRequest subclasses represent URL endpoints. They can be passed to a RWSConnection get or post method as appropriate.

TODO: Note that might want to make the Request objects responsible for deciding whether they are get, post, patch etc.

```
class rwslib.rws_requests.BuildVersionRequest
```

Return the RWS build version number

```
class rwslib.rws_requests.CacheFlushRequest
```

Calls RWS cache-flush

result (*response*)

Return RWSResponse object for success :param requests.models.Response response: request response

```
class rwslib.rws_requests.ClinicalStudiesRequest
```

Return the list of clinical studies as a `RWSSStudies` object. Clinical studies are the studies that you have access to as an EDC user.

result (*response*)

Return RWSResponse object for success :param requests.models.Response response: request response

```
class rwslib.rws_requests.CodeNameRequest
```

Return the RWS version codename

[illegible]

```

Create a new Configurable Dataset Request :param dataset_name: Name for the dataset :type dataset_name: str
:param dataset_format: Format for the dataset :type dataset_format: str :param params: set of parameters to
pass to URL :type params: dict

```

dataset

Qualify the dataset_name with the dataset_format if supplied :return: dataset name :rtype: str

```
class rwslib.rws_requests.DiagnosticsRequest
```

Return the RWS build version number

```
class rwslib.rws_requests.GlobalLibrariesRequest
```

Return the list of global libraries as a RWSSStudies object. `metadata_libraries` are the list of libraries that you have access to as an Architect Global Library Volume user

result (*response*)

Return RWSResponse object for success :param requests.models.Response response: request response

```
class rwslib.rws_requests.GlobalLibraryDraftsRequest(project_name)
```

Return the list of global library drafts

Parameters `project_name` (*str*) – Project Name

result (*response*)

Return RWSResponse object for success :param requests.models.Response response: request response

class rwslib.rws_requests.**GlobalLibraryVersionRequest** (*project_name, oid*)

Return a global library version as a string

Parameters

- **project_name** (*str*) – Project Name
- **oid** (*int*) – Metadata version OID (CRF Draft Number)

class rwslib.rws_requests.**GlobalLibraryVersionsRequest** (*project_name*)

Return the list of global library versions

Parameters **project_name** (*str*) – Project Name

result (*response*)

Return RWSResponse object for success :param requests.models.Response response: request response

class rwslib.rws_requests.**MetadataStudiesRequest**

Return the list of metadata studies as a RWSSudies object. metadata_studies are the list of studies that you have access to as an Architect user.

result (*response*)

Return RWSResponse object for success :param requests.models.Response response: request response

class rwslib.rws_requests.**ODMDatasetBase**

class rwslib.rws_requests.**PostDataRequest** (*data, headers={'Content-type': 'text/xml'}*)

Post an ODM data transaction to Rave, get back an RWSResponse object

Post an ODM to a RWS endpoint :param bytes data: Data to POST :param dict headers: Headers to add to request

args ()

Return additional args here as dict (only for post data requests)

result (*response*)

Return RWSPostResponse object for success :param requests.models.Response response: request response

class rwslib.rws_requests.**PostMetadataRequest** (*project_name, data, headers={'Content-type': 'text/xml'}*)

Post an ODM data transaction to Rave, get back an RWSResponse object

Parameters

- **project_name** (*str*) – Project Name
- **data** (*str*) – Data to dispatch
- **headers** (*dict*) – Headers to pass to client

args ()

Return additional args here as dict (only for post data requests)

result (*response*)

Return RWSPostResponse object for success :param requests.models.Response response: request response

class rwslib.rws_requests.**QueryOptionGetRequest**

Manages requests that have known query string options

class rwslib.rws_requests.**RWSAuthorizedGetRequest**

class rwslib.rws_requests.**RWSAuthorizedPostRequest**

class rwslib.rws_requests.**RWSGetRequest**

class rwslib.rws_requests.**RWSPostRequest**

class `rwslib.rws_requests.RWSRequest`

Base class for all RWS Requests

args ()

Return additional args here as dict

result (*response*)

Process a result to create a custom output :param requests.models.Response response: returned response
:return:

class `rwslib.rws_requests.StudyDatasetRequest` (*project_name*, *environment_name*,
dataset_type='regular', *start*=None,
rawsuffix=None, *formoid*=None, *versionitem*=None, *codelistsuffix*=None,
decodesuffix=None, *stdsuffix*=None)

Return the text of the full datasets listing as an ODM string.

Parameters

- **project_name** (*str*) – Name of the Rave Study
- **environment_name** (*str*) – Name of the Rave Study Environment
- **dataset_type** (*str*) – Dataset type, one of 'regular' or 'raw'
- **start** (*str*) – Start date for the Report (iso8601)
- **rawsuffix** (*str*) – Adds raw data values to a full or incremental dataset, and identifies these values with {rawsuffix}.
- **formoid** (*str*) – FormOID for which to report
- **versionitem** (*str*) – Adds the subject's CRF version to the dataset, and identifies it with {version-item}
- **codelistsuffix** (*str*) – Adds code list OIDS for fields that use a code list to the dataset, and identifies these values with {codelistsuffix}.
- **decodesuffix** (*str*) – Add decoded values of items that have an associated code list to the dataset, and identifies these values with {decodesuffix}.
- **stdsuffix** (*str*) – Adds standard data values to a full or incremental dataset, and identifies these values with {stdsuffix}.

class `rwslib.rws_requests.StudyDraftsRequest` (*project_name*)

Return the list of study drafts

Parameters **project_name** (*str*) – Project Name

result (*response*)

Return RWSResponse object for success :param requests.models.Response response: request response

class `rwslib.rws_requests.StudySubjectsRequest` (*project_name*, *environment_name*,
status=False, *include*=None, *subject_key_type*='SubjectName',
links=False)

Return the list of study subjects, defaults to the PROD environment

Parameters

- **project_name** (*str*) – Project/Study Name
- **environment_name** (*str*) – Study Environment Name (eg *Prod*)
- **status** (*bool*) – Add subject level workflow status

- **include** (*str*) – Query option to add to *include* parameter (see INCLUDE_OPTIONS for allowed values)
- **subject_key_type** (*str*) – Type of SubjectKey to have in the response (one of *SubjectName* or *SubjectUUID*)
- **links** (*bool*) – Add Deep Links to Output ODM

Note: If status == True then ?status=all If include then include parameter is also added to query string

result (*response*)

Return RWSSubjects object for success :param requests.models.Response response: request response

class rwslib.rws_requests.**StudyVersionRequest** (*project_name, oid*)

Return a study version as a string

Parameters

- **project_name** (*str*) – Project Name
- **oid** (*int*) – Metadata version OID (CRF Draft Number)

class rwslib.rws_requests.**StudyVersionsRequest** (*project_name*)

Return the list of study versions

Parameters **project_name** (*str*) – Project Name

result (*response*)

Return RWSResponse object for success :param requests.models.Response response: request response

class rwslib.rws_requests.**SubjectDatasetRequest** (*project_name, environment_name, subjectkey, dataset_type='regular', start=None, rawsuffix=None, formoid=None, versionitem=None, codelistsuffix=None, decodesuffix=None, stdsuffix=None*)

Return the text of the full datasets for a version as an ODM string. By supplying formoid, will be filtered to just that formoid data

Parameters

- **project_name** (*str*) – Name of the Rave Study
- **environment_name** (*str*) – Name of the Rave Study Environment
- **dataset_type** (*str*) – Dataset type, one of 'regular' or 'raw'
- **start** (*str*) – Start date for the Report (iso8601)
- **rawsuffix** (*str*) – Adds raw data values to a full or incremental dataset, and identifies these values with {rawsuffix}.
- **formoid** (*str*) – FormOID for which to report
- **versionitem** (*str*) – Adds the subject's CRF version to the dataset, and identifies it with {version-item}
- **codelistsuffix** (*str*) – Adds code list OIDS for fields that use a code list to the dataset, and identifies these values with {codelistsuffix}.
- **decodesuffix** (*str*) – Add decoded values of items that have an associated code list to the dataset, and identifies these values with {decodesuffix}.

- **stdsuffix** (*str*) – Adds standard data values to a full or incremental dataset, and identifies these values with {stdsuffix}.
- **subjectkey** (*str*) – Subject Key for subject for which to report

class rwslib.rws_requests.**TwoHundredRequest**
Return RWS MAuth information

class rwslib.rws_requests.**VersionDatasetRequest** (*project_name*, *environment_name*,
version_oid, *dataset_type*=*'regular'*,
start=*None*, *rawsuffix*=*None*, *formoid*=*None*, *versionitem*=*None*,
codelistsuffix=*None*, *decodesuffix*=*None*, *stdsuffix*=*None*)

Return the text of the full datasets for a version as an ODM string. By supplying formoid, will be filtered to just that formoid data

Parameters

- **project_name** (*str*) – Name of the Rave Study
- **environment_name** (*str*) – Name of the Rave Study Environment
- **dataset_type** (*str*) – Dataset type, one of 'regular' or 'raw'
- **start** (*str*) – Start date for the Report (iso8601)
- **rawsuffix** (*str*) – Adds raw data values to a full or incremental dataset, and identifies these values with {rawsuffix}.
- **formoid** (*str*) – FormOID for which to report
- **versionitem** (*str*) – Adds the subject's CRF version to the dataset, and identifies it with {version-item}
- **codelistsuffix** (*str*) – Adds code list OIDS for fields that use a code list to the dataset, and identifies these values with {codelistsuffix}.
- **decodesuffix** (*str*) – Add decoded values of items that have an associated code list to the dataset, and identifies these values with {decodesuffix}.
- **stdsuffix** (*str*) – Adds standard data values to a full or incremental dataset, and identifies these values with {stdsuffix}.
- **version_oid** (*str*) – version for which to report

class rwslib.rws_requests.**VersionRequest**
Get RWS Version number

class rwslib.rws_requests.**VersionRequestBase** (*project_name*, *oid*)
Base class for study and library metadata version requests

Parameters

- **project_name** (*str*) – Project Name
- **oid** (*int*) – Metadata version OID (CRF Draft Number)

result (*response*)

Return RWSResponse object for success :param requests.models.Response response: request response

rwslib.rws_requests.**check_dataset_type** (*dataset_type*)
Datasets may only be regular or raw :type dataset_type: str

`rwslib.rws_requests.format_date_argument(date_element)`

Take a date as either a `datetime.date/datetime` or a string and return it as a iso8601 formatted value :param Union[datetime.date, datetime.datetime] date_element: passed argument :rtype str :return:

`rwslib.rws_requests.make_url(*args, **kwargs)`

Makes a URL from component parts

12.3 Rave Web Services Objects

class `rwslib.rwsobjects.MetadataVersion`

A single MetadataVersion instance

```
<MetadataVersion OID="1203" Name="Webservice Outbound"/>
```

classmethod `fromElement(elem)`

Read properties from a MetadataVersion element

Parameters `elem` (`lxml.etree._Element`) – Source etree Element

class `rwslib.rwsobjects.ODMDoc(xml)`

A base ODM document

Abstract Doc Class :param bytes xml: Input content

class `rwslib.rwsobjects.RWSError(xml)`

Extends ODMDoc, inheriting attributes like filetype, creationdatetime etc.

Parses XML of the form:

```
<?xml version="1.0" encoding="utf-8"?>
<ODM xmlns:mdsol="http://www.mdsol.com/ns/odm/metadata"
  FileType="Snapshot"
  CreationDateTime="2013-04-08T10:28:49.578-00:00"
  FileOID="4d13722a-ceb6-4419-a917-b6ad5d0bc30e"
  ODMVersion="1.3"
  mdsol:ErrorDescription="Incorrect login and password combination. [RWS00008]"
  xmlns="http://www.cdisc.org/ns/odm/v1.3" />
```

class `rwslib.rwsobjects.RWSErrorResponse(xml)`

Parses messages of the form:

```
<Response
  ReferenceNumber="0b47fe86-542f-4070-9e7d-16396a5ef08a"
  InboundODMFileOID="Not Supplied"
  IsTransactionSuccessful="0"
  ReasonCode="RWS00092"
  ErrorClientResponseMessage="CRF version not found">
</Response>
```

exception `rwslib.rwsobjects.RWSException(msg, rws_error)`

RWS Exception. Usual to attach the error response object

Parameters

- `msg` (`str`) – Error message (base)
- `rws_error` (`str`) – RWS error message

class rwslib.rwsobjects.**RWSPostErrorResponse** (*xml*)

Responses to Clinical data post messages have additional Attributes to normal RWS Response messages:

```
<Response
  ReferenceNumber="5b1fa9a3-0cf3-46b6-8304-37c2e3b7d04f"
  InboundODMFileOID="1"
  IsTransactionSuccessful = "0"
  ReasonCode="RWS00024"
  ErrorOriginLocation="/ODM/ClinicalData[1]/SubjectData[1]"
  SuccessStatistics="Rave objects touched: Subjects=0; Folders=0; Forms=0;
↪Fields=0; LogLines=0"
  ErrorClientResponseMessage="Subject already exists.">
</Response>
```

Parameters *xml* (*str*) – Error response

class rwslib.rwsobjects.**RWSPostResponse** (*xml*)

Parses responses from PostODMClinicalData messages with the format:

```
<Response ReferenceNumber="82e942b0-48e8-4cf4-b299-51e2b6a89a1b"
  InboundODMFileOID=""
  IsTransactionSuccessful="1"
  SuccessStatistics="Rave objects touched: Subjects=0; Folders=0; Forms=0;
↪ Fields=0; LogLines=0" NewRecords=""
  SubjectNumberInStudy="1103" SubjectNumberInStudySite="55">
</Response>
```

class rwslib.rwsobjects.**RWSResponse** (*xml*)

Parses messages of the form:

```
<Response ReferenceNumber="82e942b0-48e8-4cf4-b299-51e2b6a89a1b"
  InboundODMFileOID=""
  IsTransactionSuccessful="1"
  SuccessStatistics="Rave objects touched: Subjects=0; Folders=0; Forms=0;
↪Fields=0; LogLines=0" NewRecords="">
</Response>
```

class rwslib.rwsobjects.**RWSStudies** (*xml*)

Represents a list of studies. Extends the list class and adds a couple of extra properties:

```
<ODM FileType="Snapshot" FileOID="767a1f8b-7b72-4d12-adbe-37d4d62ba75e"
  CreationDateTime="2013-04-08T10:02:17.781-00:00"
  ODMVersion="1.3"
  xmlns:mdsol="http://www.mdsol.com/ns/odm/metadata"
  xmlns:xlink="http://www.w3.org/1999/xlink"
  xmlns="http://www.cdisc.org/ns/odm/v1.3">
  <Study OID="Fixitol (Dev)" >
    <GlobalVariables>
      <StudyName>Fixitol (Dev)</StudyName>
      <StudyDescription/>
      <ProtocolName>Fixitol</ProtocolName>
    </GlobalVariables>
  </Study>
  <Study OID="IANTEST (Prod)" >
    <GlobalVariables>
      <StudyName>IANTEST</StudyName>
      <StudyDescription/>
```

(continues on next page)

(continued from previous page)

```

        <ProtocolName>IANTEST</ProtocolName>
    </GlobalVariables>
</Study>
</ODM>

```

```

class rwslib.rwsobjects.RWSStudyListItem(oid=None,      studyname=None,      protocol-
                                         name=None,      environment=None,      project-
                                         type=None)

```

An item in the RWS Study List response

Parameters

- **oid** (*str*) – Study OID
- **studyname** (*str*) – Study Name
- **protocolname** (*str*) – Protocol Name
- **environment** (*str*) – Study Environment
- **projecttype** (*str*) – Project Type

classmethod fromElement (*elem*)

Read properties from an XML Element to build a StudyList Item

Parameters **elem** (*lxml.etree.Element*) – The source Study XML Element

```

<Study OID="Fixitol (Dev)" mdsol:ProjectType="GlobalLibraryVolume">
  <GlobalVariables>
    <StudyName>Fixitol (Dev)</StudyName>
    <StudyDescription/>
    <ProtocolName>Fixitol</ProtocolName>
  </GlobalVariables>
</Study>

```

isProd ()

Is production if environment is empty

Return type bool

```

class rwslib.rwsobjects.RWSStudyMetadataVersions(xml)

```

Parses responses from MetaDataVersions request:

```

<ODM ODMVersion="1.3" Granularity="Metadata" FileType="Snapshot" FileOID=
→ "d26b4d33-376d-4037-9747-684411190179" CreationDateTime=" 2013-04-08T01:29:13 "
→ xmlns="http://www.cdisc.org/ns/odm/v1.3" xmlns:mdsol="http://www.mdsol.com/ns/
→ odm/metadata">
  <Study OID="IANTEST">
    <GlobalVariables>
      <StudyName>IANTEST</StudyName>
      <StudyDescription></StudyDescription>
      <ProtocolName>IANTEST</ProtocolName>
    </GlobalVariables>
    <MetaDataVersion OID="1203" Name="Webservice Outbound" />
    <MetaDataVersion OID="1195" Name="JC_Demo_Draft1" />
    <MetaDataVersion OID="1165" Name="Initial" />
  </Study>
</ODM>

```

```

class rwslib.rwsobjects.RWSSubjectListItem

```

Parses response of Subject List request:

```
<ClinicalData StudyOID="Fixitol (Dev) " MetaDataVersionOID="1111">
  <SubjectData SubjectKey="1111">
    <SiteRef LocationOID="335566"/>
  </SubjectData>
</ClinicalData>
```

Optionally ClinicalData may include status:

```
<SubjectData SubjectKey="1111" mdsol:Overdue="No"
  mdsol:Touched="Yes"
  mdsol:Empty="No"
  mdsol:Incomplete="No"
  mdsol:NonConformant="No"
  mdsol:RequiresSecondPass="No"
  mdsol:RequiresReconciliation="No"
  mdsol:RequiresVerification="No"
  mdsol:Verified="No"
  mdsol:Frozen="No"
  mdsol:Locked="No"
  mdsol:RequiresReview="No"
  mdsol:PendingReview="No"
  mdsol:Reviewed="No"
  mdsol:RequiresAnswerQuery="No"
  mdsol:RequiresPendingCloseQuery="No"
  mdsol:RequiresCloseQuery="No"
  mdsol:StickyPlaced="No"
  mdsol:Signed="No"
  mdsol:SignatureCurrent="No"
  mdsol:RequiresTranslation="No"
  mdsol:RequiresCoding="No"
  mdsol:RequiresPendingAnswerQuery="No"
  mdsol:RequiresSignature="No"
  mdsol:ReadyForFreeze="No"
  mdsol:ReadyForLock="Yes">
```

The SubjectKey can be either a Subject ID or a UUID depending on the value of SubjectKeyType:

```
<ClinicalData StudyOID="Fixitol (Dev) " MetaDataVersionOID="1111">
  <SubjectData SubjectKey="EC82F1AB-D463-4930-841D-36FC865E63B2" mdsol:SubjectName=
    ↪ "1" mdsol:SubjectKeyType="SubjectUUID">
    <SiteRef LocationOID="335566"/>
  </SubjectData>
</ClinicalData>
```

The Response may include links:

```
<ClinicalData StudyOID="Fixitol (Dev) " MetaDataVersionOID="1111">
  <SubjectData SubjectKey="1111">
    <SiteRef LocationOID="335566"/>
    <mdsol:Link xlink:type="simple" xlink:href="http://innovate.mdsol.com/
    ↪ MedidataRAVE/HandleLink.aspx?page=SubjectPage.aspx?ID=849" />
  </SubjectData>
</ClinicalData>
```

The ODM message has a ClinicalData element with a single SubjectData and SiteRef elements nested within. I collapse into a single object

classmethod **fromElement** (*elem*)

Read properties from an XML Element

subject_name

Get the subject name consistently :rtype str :return: The Subject ID for the subject

Note:

- If the *SubjectKeyType* is *SubjectUUID* then the subject name lives in the *mdsol:SubjectName* attribute
- If the *SubjectKeyType* is *SubjectName* then the subject name lives in the *SubjectKey* attribute

class rwslib.rwsobjects.**RWSSubjects** (*xml*)

Represents a list of subjects:

```
<ODM FileType="Snapshot"
  FileOID="770f1758-db33-4ab2-af72-38db863734aa"
  CreationDateTime="2013-04-08T14:08:06.875-00:00"
  ODMVersion="1.3">

  <ClinicalData StudyOID="Fixitol (Dev)" MetadataVersionOID="1111">
    <SubjectData SubjectKey="000001">
      <SiteRef LocationOID="BP001"/>
      <mdsol:Link xlink:type="simple" xlink:href="http://innovate.mdsol.com/
↪MedidataRAVE/HandleLink.aspx?page=SubjectPage.aspx?ID=849" />
    </SubjectData>
  </ClinicalData>

  <ClinicalData StudyOID="Fixitol (Dev)" MetadataVersionOID="1111">
    <SubjectData SubjectKey="1111">
      <SiteRef LocationOID="335566"/>
      <mdsol:Link xlink:type="simple" xlink:href="http://innovate.mdsol.com/
↪MedidataRAVE/HandleLink.aspx?page=SubjectPage.aspx?ID=849" />
    </SubjectData>
  </ClinicalData>
</ODM>
```

class rwslib.rwsobjects.**XMLRepr** (*xml*)

Classes that represent objects passed back from RWS as XML

Parameters *xml* (*str*) – XML returned from RWS

rwslib.rwsobjects.**getEnvironmentFromNameAndProtocol** (*studyname*, *protocolname*)

Extract environment name using studyname and protocolname to guide :param str studyname: Name of the study (including Env) :param str protocolname: Name of the study

rwslib.rwsobjects.**parseXMLString** (*xml*)

Parse XML string, return root :param str: Passed in XML

12.4 Biostats Gateway

Views related to clinical views and their metadata, Biostats Gateway/Adapter

http://rws-webhelp.s3.amazonaws.com/WebHelp_ENG/solutions/01_biostat_adapter.html#biostat-adapter

```
class rwslib.rws_requests.biostats_gateway.CVMetaDataRequest (project_name,  
                                                             environment_name,  
                                                             versionitem=None,  
                                                             rawsuffix=None,  
                                                             codelistsuf-  
                                                             fix=None,          de-  
                                                             codesuffix=None)
```

Return Clinical View Metadata as ODM string

Parameters

- **project_name** (*str*) – Project Name
- **environment_name** (*str*) – Environment Name
- **versionitem** (*str*) – Adds the subject's CRF version to the dataset, and identifies it with *versionitem*
- **rawsuffix** (*str*) – Adds raw data values to a full or incremental dataset, and identifies these values with *rawsuffix*.
- **codelistsuffix** (*str*) – Adds code list OIDS for fields that use a code list to the dataset, and identifies these values with *codelistsuffix*
- **decodesuffix** (*str*) – Add decoded values of items that have an associated code list to the dataset, and identifies these values with *decodesuffix*.

```
studyname_environment ()
```

Combine the Study Name and Environment

Return type *str*

```
class rwslib.rws_requests.biostats_gateway.CommentDataRequest (project_name, en-  
                                                             vironment_name,  
                                                             dataset_format='csv')
```

Return Comments from Rave as CSV or XML

Parameters

- **project_name** (*str*) – Project Name
- **dataset_format** (*str*) – Specify format of the Datasets (either *csv* or *xml*)
- **environment_name** (*str*) – Environment Name

```
class rwslib.rws_requests.biostats_gateway.DataDictionariesRequest (project_name,  
                                                             environ-  
                                                             ment_name,  
                                                             dataset_format='csv')
```

Retrieve Data Dictionaries from Rave

Parameters

- **project_name** (*str*) – Project Name
- **dataset_format** (*str*) – Specify format of the Datasets (either *csv* or *xml*)
- **environment_name** (*str*) – Environment Name

```
class rwslib.rws_requests.biostats_gateway.FormDataRequest (project_name,      en-  
                                                             vironment_name,  
                                                             dataset_type,  
                                                             form_oid, start=None,  
                                                             dataset_format='csv')
```

Return CV Form Data as CSV or XML

Parameters

- **project_name** (*str*) – Project Name
- **environment_name** (*str*) – Environment Name
- **dataset_type** (*str*) – Type of dataset (either *regular* or *raw*)
- **form_oid** (*str*) – OID for the Form of interest
- **start** (*str*) – Start Date for the dataset pull (should be an iso8601 formatted date)
- **dataset_format** (*str*) – Specify format of the Datasets (either *csv* or *xml*)

class rwslib.rws_requests.biostats_gateway.**MetaDataRequest** (*dataset_format='csv'*)
 Return Metadata for Clinical Views in CSV or XML format

Parameters **dataset_format** (*str*) – Specify format of the Datasets (either *csv* or *xml*)

class rwslib.rws_requests.biostats_gateway.**ProjectMetaDataRequest** (*project_name*,
dataset_format='csv')
 Return Metadata for Clinical Views in CSV or XML format for a Project

Parameters

- **project_name** (*str*) – Project Name
- **dataset_format** (*str*) – Specify format of the Datasets (either *csv* or *xml*)

class rwslib.rws_requests.biostats_gateway.**ProtocolDeviationsRequest** (*project_name*,
envi-
ron-
ment_name,
dataset_format='csv')

Retrieve Protocol Deviation Information from Rave

Parameters

- **project_name** (*str*) – Project Name
- **dataset_format** (*str*) – Specify format of the Datasets (either *csv* or *xml*)
- **environment_name** (*str*) – Environment Name

class rwslib.rws_requests.biostats_gateway.**ViewMetaDataRequest** (*view_name*,
dataset_format='csv')

Return Metadata for Clinical Views in CSV format for a single View

Parameters

- **view_name** (*str*) – Clinical View of interest
- **dataset_format** (*str*) – Specify format of the Datasets (either *csv* or *xml*)

rwslib.rws_requests.biostats_gateway.**check_dataset_format** (*ds_format*)
 Ensure dataset format is XML or CSV

Parameters **ds_format** (*str*) – Format of the Dataset (expected to be one of *csv* or *xml*)

rwslib.rws_requests.biostats_gateway.**dataset_format_to_extension** (*ds_format*)
 Get the preferred Dataset format extension

Parameters **ds_format** (*str*) – Format of the Dataset (expected to be one of *csv* or *xml*)

Return type *str*

12.5 ODM Adapter

Requests related to the ODM Adapter

http://rws-webhelp.s3.amazonaws.com/WebHelp_ENG/solutions/clinical_data_audits/index.html#odm-adapter

```
class rwslib.rws_requests.odm_adapter.AuditRecordsRequest (project_name, environ-  
ment_name, startid=1,  
per_page=100)
```

Clinical Audit Records Dataset

Parameters

- **project_name** (*str*) – Project Name
- **environment_name** (*str*) – Environment Name
- **startid** (*int*) – Starting Audit
- **per_page** (*int*) – Page Size

```
class rwslib.rws_requests.odm_adapter.SignatureDefinitionsRequest (project_name)  
Return signature definitions for all versions of the study
```

Parameters **project_name** (*str*) – Project Name

```
class rwslib.rws_requests.odm_adapter.SitesMetadataRequest (project_name=None,  
environ-  
ment_name=None)
```

List all sites in a study along with their StudyVersions

Parameters

- **project_name** (*str*) – Project Name
- **environment_name** (*str*) – Environment Name

```
class rwslib.rws_requests.odm_adapter.UsersRequest (project_name, environment_name,  
location_oid=None)
```

Return list of users for study (can be filtered by location)

Parameters

- **project_name** (*str*) – Project Name
- **environment_name** (*str*) – Environment Name
- **location_oid** (*str*) – Study Site Name to filter with

```
class rwslib.rws_requests.odm_adapter.VersionFoldersRequest (project_name, envi-  
ronment_name)
```

Identify all folders in use in study

Parameters

- **project_name** (*str*) – Project Name
- **environment_name** (*str*) – Environment Name

13.1 Metadata Builders

Note: Any Class with the Prefix **Mdsol** represents a Medidata Rave specific extension

class `rwslib.builders.metadata.Alias` (*context*, *name*)

An **Alias** provides an additional name for an element. The Context attribute specifies the application domain in which this additional name is relevant.

Parameters

- **context** (*str*) – Context attribute specifies the application domain
- **name** (*str*) – Name

class `rwslib.builders.metadata.BasicDefinitions`

Container for *MeasurementUnit*

measurement_units = **None**

Collection of *MeasurementUnit*

class `rwslib.builders.metadata.CheckValue` (*value*)

A value in a *RangeCheck*

Parameters **value** (*str*) – Value for a *RangeCheck*

class `rwslib.builders.metadata.CodeList` (*oid*, *name*, *datatype*, *sas_format_name=None*)

Defines a discrete set of permitted values for an item.

Note:

- Equates to a Rave Dictionary
- Does not support ExternalCodeList

Parameters

- **oid** (*str*) – CodeList OID
- **name** (*str*) – Name of CodeList
- **datatype** (*DataType*) – DataType restricts the values that can appear in the CodeList whether internal or external (*integer* | *float* | *text* | *string*)
- **sas_format_name** (*str*) – SASFormatName must be a legal SAS format for CodeList

aliases = None

Collection of *Alias*

codelist_items = None

Collection of *CodeListItem*

class `rwslib.builders.metadata.CodeListItem` (*coded_value*, *order_number=None*, *specify=False*)

Defines an individual member value of a *CodeList* including display format. The actual value is given, along with a set of print/display-forms.

Parameters

- **coded_value** (*str*) – Coded Value for CodeListItem
- **order_number** (*int*) – OrderNumber for the CodeListItem - Note: this is a Medidata Rave Extension, but upstream ODM has been updated to include the OrderNumber attribute
- **specify** (*bool*) – Does this have a Specify? option? - *Rave Specific Attribute*

aliases = None

Collection of *Alias*

class `rwslib.builders.metadata.CodeListRef` (*oid*)

A reference to a *CodeList* definition.

Parameters **oid** – OID for *CodeList*

class `rwslib.builders.metadata.Decode`

The displayed value relating to the CodedValue

translations = None

Collection of Translation for the Decode

class `rwslib.builders.metadata.FormDef` (*oid*, *name*, *repeating=False*, *order_number=None*, *active=True*, *template=False*, *signature_required=False*, *log_direction='Portrait'*, *double_data_entry='MustNotDDE'*, *confirmation_style='NoLink'*, *link_study_event_oid=None*, *link_form_oid=None*)

A FormDef describes a type of form that can occur in a study.

Parameters

- **oid** (*str*) – OID for FormDef
- **name** (*str*) – Name for FormDef
- **repeating** (*bool*) – Is this a repeating Form?
- **order_number** (*int*) – OrderNumber for the FormDef

- **active** (*bool*) – Indicates that the form is available to end users when you publish and push the draft to Rave EDC - *Rave Specific Attribute*
- **template** (*bool*) – Indicates that the form is a template form in Rave EDC - *Rave Specific Attribute*
- **signature_required** (*bool*) – Select to ensure that the form requires investigator signature for all submitted data points - *Rave Specific Attribute*
- **log_direction** (*str*) – Set the display mode of a form, (*Landscape* or *Portrait*) - *Rave Specific Attribute*
- **double_data_entry** (*str*) – Indicates if the form is used to collect data in Rave Double Data Entry (DDE), (*Always*, *Never* or *As Per Site*) - *Rave Specific Attribute*
- **confirmation_style** – Style of Confirmation, (*None*, *NotLink*, *LinkNext* or *LinkCustom*) - *Rave Specific Attribute*
- **link_study_event_oid** – OID for *StudyEvent* target for Link - *Rave Specific Attribute*
- **link_form_oid** – OID for *FormRef* target for Link - *Rave Specific Attribute*

aliases = *None*

Collection of *Alias* for Form

entry_restrictions = *None*

Collection of *EntryRestriction* for Form - *Rave Specific Attribute*

helptexts = *None*

Collection of *HelpText* for Form (Cardinality not clear) - *Rave Specific Attribute*

itemgroup_refs = *None*

Collection of *ItemGroupRef* for Form

view_restrictions = *None*

Collection of *ViewRestriction* for Form - *Rave Specific Attribute*

class rwslib.builders.metadata.**FormRef** (*oid*, *order_number*, *mandatory*)

A reference to a *FormDef* as it occurs within a specific *StudyEventDef*. The list of *FormRef* identifies the types of forms that are allowed to occur within this type of study event. The *FormRef* within a single *StudyEventDef* must not have duplicate FormOIDs nor OrderNumbers.

Parameters

- **oid** (*str*) – Set the *FormDef* OID for the *FormRef*
- **order_number** (*int*) – Define the OrderNumber for the *FormRef* within the containing *StudyEventDef*
- **mandatory** (*bool*) – Is this Form Mandatory?

class rwslib.builders.metadata.**GlobalVariables** (*protocol_name*, *name=None*, *description=""*)

GlobalVariables includes general summary information about the *Study*.

Note: Name and description are not important. *protocol_name* maps to the Rave project name

Parameters

- **protocol_name** (*str*) – Protocol Name
- **name** (*str*) – Study Name

- **description** (*str*) – Study Description

```
class rwslib.builders.metadata.ItemDef(oid, name, datatype, length=None, significant_digits=None, sas_field_name=None, sds_var_name=None, origin=None, comment=None, active=True, control_type=None, acceptable_file_extensions=None, indent_level=0, source_document_verify=False, default_value=None, sas_format=None, sas_label=None, query_future_date=False, visible=True, translation_required=False, query_non_conformance=False, other_visits=False, can_set_item_group_date=False, can_set_form_date=False, can_set_study_event_date=False, can_set_subject_date=False, visual_verify=False, does_not_break_signature=False, date_time_format=None, field_number=None, variable_oid=None)
```

An ItemDef describes a type of item that can occur within a study. Item properties include name, datatype, measurement units, range or codelist restrictions, and several other properties.

Parameters

- **oid** (*str*) – OID for ItemDef
- **name** (*str*) – Name for ItemDef
- **datatype** (*DataType*) – Datatype for ItemDef
- **length** (*int*) – Max. Length of content expected in Item Value
- **significant_digits** (*int*) – Max. Number of significant digits in Item Value
- **sas_field_name** (*str*) – SAS Name for the ItemDef
- **sds_var_name** (*str*) – SDS Variable Name
- **origin** (*str*) – Origin for the Variable
- **comment** (*str*) – Comment for the Variable
- **active** (*bool*) – Is the Variable Active? - *Rave Specific Attribute*
- **control_type** (*ControlType*) – Control Type for the Variable - *Rave Specific Attribute*
- **acceptable_file_extensions** (*str*) – File extensions for File Upload Control (separated by a comma) - *Rave Specific Attribute*
- **indent_level** (*int*) – Level of indentation of a field from the left-hand page margin. - *Rave Specific Attribute*
- **source_document_verify** (*bool*) – Does this Variable need to be SDV'ed? - *Rave Specific Attribute*
- **default_value** (*str*) – Value entered in this field is displayed as the default value in Rave EDC. - *Rave Specific Attribute*
- **sas_format** (*str*) – SAS variable format of maximum 25 alphanumeric characters.
- **sas_label** (*str*) – SAS label of maximum 256 alphanumeric characters.

- **query_future_date** (*bool*) – Generates a query when the Rave EDC user enters a future date in the field. - *Rave Specific Attribute*
- **visible** (*bool*) – Indicates that the field displays on the form when the version is pushed to Rave EDC. - *Rave Specific Attribute*
- **translation_required** (*bool*) – Enables translation functionality for the selected field. - *Rave Specific Attribute*
- **query_non_conformance** (*bool*) – Generates a query when the Rave EDC user enters data in a format other than what has been defined for the variable. - *Rave Specific Attribute*
- **other_visits** (*bool*) – Display previous visit data - *Rave Specific Attribute*
- **can_set_item_group_date** (*bool*) – If a form contains log fields, this parameter assigns the date entered into the field as the date for the record (log line) - *Rave Specific Attribute*
- **can_set_form_date** (*bool*) – Observation Date of Form assigns the date entered into the field as the date for the form - *Rave Specific Attribute*
- **can_set_study_event_date** (*bool*) – Observation Date of Folder assigns the date entered into the field as the date for the folder - *Rave Specific Attribute*
- **can_set_subject_date** (*bool*) – Observation Date of Subject assigns the date entered into the field as the date for the Subject. - *Rave Specific Attribute*
- **visual_verify** (*bool*) – This parameter sets the field as a Visual Verify field in Rave DDE - *Rave Specific Attribute*
- **does_not_break_signature** (*bool*) – Indicates that the field (both derived and non-derived) does not participate in signature. - *Rave Specific Attribute*
- **date_time_format** (*str*) – Displays text boxes for the user to enter the day, month, and year according to the specified variable format. - *Rave Specific Attribute*
- **field_number** (*str*) – Number that displays to the right of a field label in Rave EDC to create a numbered list on the form. - *Rave Specific Attribute*
- **variable_oid** – OID for Variable - *Rave Specific Attribute*

aliases = None
Collection of *Alias*

codelistref = None
Matching *CodeListRef*

entry_restrictions = None
Collection of *MdsolEntryRestriction*

header_text = None
Matching *MdsolHeaderText*

help_texts = None
Collection of *MdsolHelpText*

measurement_unit_refs = None
Collection of *MeasurementUnitRef*

question = None
Matching *Question*

range_checks = None
Collection of *RangeCheck*

review_groups = None
Collection of *MdsolReviewGroup*

view_restrictions = None
Collection of *MdsolViewRestriction*

```
class rwslib.builders.metadata.ItemGroupDef (oid,          name,          repeating=False,
                                             is_reference_data=False,
                                             sas_dataset_name=None,    domain=None,
                                             origin=None,   role=None,   purpose=None,
                                             comment=None)
```

An ItemGroupDef describes a type of item group that can occur within a Study.

Parameters

- **oid** (*str*) – OID for ItemGroupDef
- **name** (*str*) – Name for ItemGroupDef
- **repeating** (*bool*) – Is this a repeating ItemDef?
- **is_reference_data** (*bool*) – If IsReferenceData is Yes, this type of item group can occur only within a ReferenceData element. If IsReferenceData is No, this type of item group can occur only within a ClinicalData element. The default for this attribute is No.
- **sas_dataset_name** (*str*) – SAS Dataset Name
- **domain** (*str*) – Domain for Items within this ItemGroup
- **origin** – Origin of data (eg CRF, eDT, Derived)
- **role** – Role for the ItemGroup (eg Identifier, Topic, Timing, Qualifiers)
- **purpose** – Purpose (eg Tabulation)
- **comment** – Comment on the ItemGroup Contents

aliases = None
Collection of *Alias*

item_refs = None
Collection of *ItemRef*

label_refs = None
Collection of *MdsolLabelRef*

```
class rwslib.builders.metadata.ItemGroupRef (oid,          order_number=None,    manda-
                                             tory=True)
```

A reference to an ItemGroupDef as it occurs within a specific *FormDef*. The list of ItemGroupRefs identifies the types of item groups that are allowed to occur within this type of form. The ItemGroupRefs within a single FormDef must not have duplicate ItemGroupOIDs nor OrderNumbers.

Parameters

- **oid** (*str*) – OID for the referenced *ItemGroupDef*
- **order_number** (*int*) – OrderNumber for the ItemGroupRef
- **mandatory** (*bool*) – Is this ItemGroupRef required?

```
class rwslib.builders.metadata.ItemRef (oid,          order_number=None,    manda-
                                       tory=False,    key_sequence=None,    imputa-
                                       tion_method_oid=None,    role=None,
                                       role_codelist_oid=None)
```

A reference to an *ItemDef* as it occurs within a specific *ItemGroupDef*. The list of ItemRefs identifies the

types of items that are allowed to occur within this type of item group.

Parameters

- **oid** (*str*) – OID for *ItemDef*
- **order_number** (*int*) – OrderNumber for the ItemRef
- **mandatory** (*bool*) – Is this ItemRef required?
- **key_sequence** (*int*) – The KeySequence (if present) indicates that this item is a key for the enclosing item group. It also provides an ordering for the keys.
- **imputation_method_oid** (*str*) – *DEPRECATED*
- **role** (*str*) – Role name describing the use of this data item.
- **role_codelist_oid** (*str*) – RoleCodeListOID may be used to reference a *CodeList* that defines the full set roles from which the Role attribute value is to be taken.

attributes = None

Collection of *MdsolAttribute*

```
class rwslib.builders.metadata.MdsolAttribute(namespace, name, value, transaction_type='Insert')
```

Rave Web Services element for holding Vendor Attributes

Note: This is Medidata Rave Specific Element

name = None

Name for the Attribute

namespace = None

Namespace for the Attribute

transaction_type = None

TransactionType for the Attribute

value = None

Value for the Attribute

```
class rwslib.builders.metadata.MdsolCheckAction(variable_oid=None, field_oid=None,
                                                form_oid=None, folder_oid=None,
                                                record_position=None,
                                                form_repeat_number=None,
                                                folder_repeat_number=None,
                                                check_action_type=None,
                                                check_string=None,
                                                check_options=None,
                                                check_script=None)
```

Check Action modeled after check action in Architect Loader spreadsheet.

Note:

- Do not use directly, use appropriate sub-class.
 - This is a Medidata Rave Specific Element
-

Parameters

- **variable_oid** (*str*) – OID for the Variable that is the target of the CheckAction
- **field_oid** (*str*) – OID for the Field that is the target of the CheckAction
- **form_oid** (*str*) – OID for the Form that is the target of the CheckAction
- **folder_oid** (*str*) – OID for the Folder that is the target of the CheckAction
- **record_position** (*int*) – Record Position for the CheckAction
- **form_repeat_number** (*int*) – Form Repeat Number for the CheckAction
- **folder_repeat_number** (*int*) – Folder Repeat Number for the CheckAction
- **check_string** (*str*) – CheckAction String
- **check_options** (*str*) – CheckAction Options
- **check_script** (*str*) – CheckAction Script

check_action_type

return the CheckAction Type

```
class rwslib.builders.metadata.MdsolCheckStep (variable_oid=None, data_format=None,
                                              form_oid=None,      folder_oid=None,
                                              field_oid=None,    static_value=None,
                                              function=None,   custom_function=None,
                                              record_position=None,
                                              form_repeat_number=None,
                                              folder_repeat_number=None,      logi-
                                              cal_record_position=None)
```

A check step modeled after the Architect Loader definition.

Note: Do not use directly, use appropriate subclasses.

Note: this is a Medidata Rave Specific Element

Parameters

- **variable_oid** (*str*) – OID for Variable targeted by CheckStep
- **data_format** (*str*) – Format for Value
- **form_oid** (*str*) – OID for Form
- **folder_oid** (*str*) – OID for Folder
- **field_oid** (*str*) – OID for Field
- **custom_function** (*str*) – Name of Custom Function for CheckStep
- **record_position** (*int*) – Record Position - If the field is a standard (non-log) field, enter 0
- **form_repeat_number** (*int*) – Repeat Number for Form for CheckStep
- **folder_repeat_number** (*int*) – Repeat Number for Folder for CheckStep
- **logical_record_position** (*LogicalRecordPositionType*) – LRP value for the CheckStep
- **static_value** (*str*) – Static Value for CheckStep

- **function** (*StepType*) – Check Function for CheckStep

logical_record_position

Get the Logical Record Position :return: the Logical Record Position

class `rwslib.builders.metadata.MdsolConfirmationMessage` (*message, lang=None*)
Form is saved confirmation message

Note: this is a Medidata Rave Specific Element

Parameters

- **message** (*str*) – Content of confirmation message
- **lang** (*str*) – Language declaration for Message

class `rwslib.builders.metadata.MdsolCustomFunctionDef` (*oid, code, language='C#'*)
Extension for Rave Custom functions

Note:

- This is a Medidata Rave Specific Extension
 - VB has been deprecated in later Rave versions.
-

Parameters

- **oid** (*str*) – OID for CustomFunction
- **code** (*str*) – Content for the CustomFunction
- **language** (*str*) – Language for the CustomFunction

class `rwslib.builders.metadata.MdsolDerivationDef` (*oid, active=True, bypass_during_migration=False, needs_retesting=False, variable_oid=None, field_oid=None, form_oid=None, folder_oid=None, record_position=None, form_repeat_number=None, folder_repeat_number=None, logical_record_position=None, all_variables_in_folders=None, all_variables_in_fields=None*)

Extension for Rave derivations

Note: This is a Medidata Rave Specific Extension

Parameters

- **oid** (*str*) – OID for Derivation
- **active** (*bool*) – Is this Derivation Active?
- **bypass_during_migration** (*bool*) – Bypass this Derivation on Study Migration?

- **needs_retesting** (*bool*) – Does this Derivation need retesting?
- **variable_oid** (*str*) – OID for target variable (eg OID for *ItemDef*)
- **field_oid** (*str*) – OID for target field (eg OID for *ItemDef*)
- **form_oid** (*str*) – OID for Form for target of Derivation (eg OID for *FormDef*)
- **folder_oid** (*str*) – OID for Folder for target of Derivation (eg OID for *StudyEventDef*)
- **record_position** (*int*) – Record Position for the Derivation
- **form_repeat_number** (*int*) – Form Repeat Number for the CheckAction
- **folder_repeat_number** (*int*) – Folder Repeat Number for the CheckAction
- **logical_record_position** (*LogicalRecordPositionType*) –
- **all_variables_in_folders** (*bool*) – Evaluates the derivation according to any field using the specified variable within a specific folder.
- **all_variables_in_fields** (*bool*) – Evaluates the derivation according to any field using the specified variable across the whole subject.

derivation_steps = None

Set of *MdsolDerivationStep* for this derivation

logical_record_position

Get the Logical Record Position :return: the Logical Record Position

```
class rwslib.builders.metadata.MdsolDerivationStep (variable_oid=None,
                                                    data_format=None,
                                                    form_oid=None,
                                                    folder_oid=None,
                                                    field_oid=None,      value=None,
                                                    function=None,         cus-
                                                    tom_function=None,
                                                    record_position=None,
                                                    form_repeat_number=None,
                                                    folder_repeat_number=None,
                                                    logical_record_position=None)
```

A derivation step modeled after the Architect Loader definition.

Note: Do not use directly, use appropriate subclasses.

Note: this is a Metadata Rave Specific Element

Parameters

- **variable_oid** (*str*) – OID for Variable targeted by Derivation
- **data_format** (*str*) – Format for Value
- **form_oid** (*str*) – OID for Form
- **folder_oid** (*str*) – OID for Folder
- **field_oid** (*str*) – OID for Field

- **value** (*str*) – Value for DerivationStep
- **custom_function** (*str*) – Name of Custom Function for DerivationStep
- **record_position** (*int*) – Record Position - If the field is a standard (non-log) field, enter 0
- **form_repeat_number** (*int*) – Repeat Number for Form for DerivationStep
- **folder_repeat_number** (*int*) – Repeat Number for Folder for DerivationStep
- **logical_record_position** (*LogicalRecordPositionType*) – LRP value for the DerivationStep

function

Return the Derivation Function :return:

logical_record_position

Get the Logical Record Position :return: the Logical Record Position

```
class rwslib.builders.metadata.MdsolEditCheckDef (oid, active=True, by-  

pass_during_migration=False, needs_retesting=False)
```

Extension for Rave edit checks

Note: This is a Medidata Rave Specific Extension

Parameters

- **oid** (*str*) – EditCheck OID
- **active** (*bool*) – Is this EditCheck active?
- **bypass_during_migration** (*bool*) – Bypass this EditCheck during a Study Migration
- **needs_retesting** (*bool*) – Does this EditCheck need Retesting?

check_actions = None

Set of *MdsolCheckAction* for this EditCheck

check_steps = None

Set of *MdsolCheckStep* for this EditCheck

```
class rwslib.builders.metadata.MdsolEntryRestriction (rolename)  

EntryRestriction for FormDef and ItemDef
```

Note: This is Medidata Rave Specific Element

rolename = None

Name for the role for which the EntryRestriction applies

```
class rwslib.builders.metadata.MdsolHeaderText (content, lang=None)  

Header text for ItemDef when shown in grid
```

Note: this is a Medidata Rave Specific Element

Parameters

- **content** (*str*) – Content for the Header Text
- **lang** (*str*) – Language specification for Header

class rwslib.builders.metadata.**MdsolHelpText** (*lang, content*)
Help element for *FormDef* and *ItemDef*

Note: This is Medidata Rave Specific Element

content = **None**
HelpText content

lang = **None**
Language specification for HelpText

class rwslib.builders.metadata.**MdsolLabelDef** (*oid, name, field_number=None*)
Label definition

Note: This is a Medidata Rave Specific Element

Parameters

- **oid** – OID for the MdsolLabelDef
- **name** – Name for the MdsolLabelDef
- **field_number** (*int*) – FieldNumber for the MdsolLabelDef

help_texts = **None**
Collection of HelpText

translations = **None**
Collection of Translation

view_restrictions = **None**
Collection of ViewRestriction

class rwslib.builders.metadata.**MdsolLabelRef** (*oid, order_number*)
A reference to a label on a form

Note: This is Medidata Rave Specific Element

oid = **None**
OID for the corresponding MdsolLabel

order_number = **None**
OrderNumber for the Label

class rwslib.builders.metadata.**MdsolReviewGroup** (*name*)
Maps to Rave review groups for an *ItemDef*

Note: this is a Medidata Rave Specific Element

Parameters **name** (*str*) – Name for the MdsolReviewGroup

class rwslib.builders.metadata.**MdsolViewRestriction** (*rolename*)
ViewRestriction for *FormDef* and *ItemDef*

Note: This is Metadata Rave Specific Element

rolename = **None**

Name for the role for which the ViewRestriction applies

class rwslib.builders.metadata.**MeasurementUnit** (*oid, name, unit_dictionary_name=None, constant_a=1, constant_b=1, constant_c=0, constant_k=0, standard_unit=False*)

The physical unit of measure for a data item or value. The meaning of a MeasurementUnit is determined by its Name attribute.

Parameters

- **oid** (*str*) – MeasurementUnit OID
- **name** (*str*) – Maps to Coded Unit within unit dictionary entries in Rave.
- **unit_dictionary_name** (*str*) – Maps to unit dictionary Name in Rave. - *Rave specific attribute*
- **constant_a** (*int*) – Maps to the unit dictionary Constant A in Rave. - *Rave specific attribute*
- **constant_b** (*int*) – Maps to the unit dictionary Constant B in Rave. - *Rave specific attribute*
- **constant_c** (*int*) – Maps to the unit dictionary Constant C in Rave. - *Rave specific attribute*
- **constant_k** (*int*) – Maps to the unit dictionary Constant K in Rave. - *Rave specific attribute*
- **standard_unit** (*bool*) – Yes = Standard checked within the unit dictionary entry in Rave. No = Standard unchecked within the unit dictionary entry in Rave. - *Rave specific attribute*

symbols = **None**

Collection of *Symbol* for this MeasurementUnit

class rwslib.builders.metadata.**MeasurementUnitRef** (*oid*)
A reference to a measurement unit definition (*MeasurementUnit*).

Parameters **oid** (*str*) – *MeasurementUnit* OID

class rwslib.builders.metadata.**MetaDataVersion** (*oid, name, description=None, primary_formoid=None, fault_matrix_oid=None, delete_existing=False, signature_prompt=None*)

A metadata version (MDV) defines the types of study events, forms, item groups, and items that form the study data.

Parameters

- **oid** (*str*) – MDV OID

- **name** (*str*) – Name for MDV
- **description** (*str*) – Description for MDV
- **primary_formoid** (*str*) – OID of Primary Form - *Rave Specific Attribute*
- **default_matrix_oid** (*str*) – OID of Default Matrix - *Rave Specific Attribute*
- **delete_existing** (*bool*) – Overwrite the previous version - *Rave Specific Attribute*
- **signature_prompt** (*str*) – Prompt for Signature - *Rave Specific Attribute*

class rwslib.builders.metadata.**Protocol**

The Protocol lists the kinds of study events that can occur within a specific version of a *Study*. All clinical data must occur within one of these study events.

study_event_refs = None
Collection of *StudyEventRef*

class rwslib.builders.metadata.**Question**

A label shown to a human user when prompted to provide data for an item on paper or on a screen.

translations = None
Collection of Translation for the Question

class rwslib.builders.metadata.**RangeCheck** (*comparator, soft_hard*)

Rangecheck in Rave relates to QueryHigh QueryLow and NonConformantHigh and NonConformantLow for other types of RangeCheck, need to use an EditCheck (part of Rave's extensions to ODM)

Parameters

- **comparator** (*str*) – Comparator for RangeCheck (*LT, LE, GT, GE, EQ, NE, IN, NOTIN*)
- **soft_hard** (*str*) – Soft or Hard range check (*Soft, Hard*)

comparator
returns the comparator

soft_hard
returns the Soft or Hard range setting

class rwslib.builders.metadata.**Study** (*oid, project_type=None*)

This element collects static structural information about an individual study.

Parameters

- **oid** (*str*) – Study OID
- **project_type** (*str*) – Type of Project (Project or Global Library) - *Rave Specific Attribute*

studyevent_defs = None
set of *StudyEventDef* for this Study element

class rwslib.builders.metadata.**StudyEventDef** (*oid, name, repeating, event_type, category=None, access_days=None, start_win_days=None, target_days=None, end_win_days=None, over-due_days=None, close_days=None*)

A StudyEventDef packages a set of forms. Scheduled Study Events correspond to sets of forms that are expected to be collected for each subject as part of the planned visit sequence for the study. Unscheduled Study Events are designed to collect data that may or may not occur for any particular subject such as a set of forms that are completed for an early termination due to a serious adverse event.

Parameters

- **oid** (*str*) – OID for StudyEventDef
- **name** (*str*) – Name for StudyEventDef
- **repeating** (*bool*) – Is this a repeating StudyEvent?
- **event_type** (*str*) – Type of StudyEvent (either *Scheduled*, *Unscheduled*, *Common*)
- **category** (*str*) – Category attribute is typically used to indicate the study phase appropriate to this type of study event. Examples might include Screening, PreTreatment, Treatment, and FollowUp.
- **access_days** (*int*) – The number of days before the Target date that the folder may be opened, viewed and edited from the Task List in Rave EDC. - *Rave Specific Attribute*
- **start_win_days** (*int*) – The number of days before the Target date that is considered to be the ideal start-date for use of this folder. - *Rave Specific Attribute*
- **target_days** (*int*) – The ideal number of days between Time Zero and the date of use for the folder. - *Rave Specific Attribute*
- **end_win_days** (*int*) – The number of days after the Target date that is considered to be the ideal end date for use of this folder. - *Rave Specific Attribute*
- **overdue_days** (*int*) – The number of days after the Target date at which point empty data points are marked overdue, and are displayed in the Task Summary in Rave EDC. - *Rave Specific Attribute*
- **close_days** (*int*) – The number of days after the Target date at which point no new data may be entered into the folder. - *Rave Specific Attribute*

class rwslib.builders.metadata.**StudyEventRef** (*oid*, *order_number=None*, *mandatory=False*)

A reference to a StudyEventDef as it occurs within a specific version of a *Study*. The list of *StudyEventRef* identifies the types of study events that are allowed to occur within the study. The *StudyEventRef* within a *Protocol* must not have duplicate StudyEventOIDs nor duplicate OrderNumbers.

Note: In Rave the *OrderNumber* attribute is not reported for the **SUBJECT** Form, for this reason the *order_number* attribute is not mandatory and will default to None

Parameters

- **oid** (*str*) – *StudyEventDef* OID
- **order_number** (*Union(int, None)*) – OrderNumber for the *StudyEventRef* within the *Study*
- **mandatory** (*bool*) – Is this StudyEventDef Mandatory? (True/False)

class rwslib.builders.metadata.**Symbol**
A human-readable name for a *MeasurementUnit*.

translations = None
Collection of *TranslatedText*

class rwslib.builders.metadata.**TranslatedText** (*text*, *lang=None*)
Represents a language and a translated text for that language

Parameters

- **text** (*str*) – Content expressed in language designated by *lang*

- **lang** (*str*) – Language code

13.2 Clinical Data Builders

Note: Any Class with the Prefix **Mdsol** represents a Medidata Rave specific extension

```
class rwslib.builders.clinicaldata.Annotation (annotation_id=None, seqnum=1,  
                                              flags=None, comment=None, trans-  
                                              action_type=None)
```

A general note about clinical data. If an annotation has both a comment and flags, the flags should be related to the comment.

Note: Annotation is not supported by Medidata Rave

Parameters

- **id** (*str or None*) – ID for this Annotation (required if contained within an Annotations element)
- **seqnum** (*int*) – SeqNum for Annotation
- **flags** (*Flag or list(Flag)*) – one or more *Flag* for the Annotation
- **comment** (*Comment*) – one or more *Comment* for the Annotation
- **transaction_type** – TransactionType for Annotation (one of **Insert**, **Update**, **Remove**, **Upsert**, **Context**)

annotation_id
ID for annotation

Note: If an Annotation is contained with an Annotations element, the ID attribute is required.

seqnum

SeqNum attribute (a small positive integer) uniquely identifies the annotation within its parent entity.

```
class rwslib.builders.clinicaldata.Annotations (annotations=[])  
Groups Annotation elements referenced by ItemData[TYPE] elements.
```

```
class rwslib.builders.clinicaldata.AuditRecord (edit_point=None,  
                                              used_imputation_method=None, identi-  
                                              fier=None, include_file_oid=None)
```

An AuditRecord carries information pertaining to the creation, deletion, or modification of clinical data. This information includes who performed that action, and where, when, and why that action was performed.

Note: AuditRecord is supported only by *ItemData* in Rave

Parameters

- **identifier** (*str*) – Audit identifier

- **edit_point** (*str*) – EditPoint attribute identifies the phase of data processing in which action occurred (*Monitoring, DataManagement, DBAudit*)
- **used_imputation_method** (*bool*) – Indicates whether the action involved the use of a Method
- **include_file_oid** (*bool*) – Include the FileOID in the AuditRecord

audit_id

AuditRecord ID

Note: If an AuditRecord is contained within an AuditRecords element, the ID attribute must be provided.

date_time_stamp = None

DateTimeStamp for the AuditRecord

edit_point

EditPoint attribute identifies the phase of data processing in which action occurred (*Monitoring, DataManagement, DBAudit*)

location_ref = None

LocationRef for the AuditRecord

reason_for_change = None

ReasonForChange for the AuditRecord

source_id = None

SourceID for the AuditRecord

user_ref = None

UserRef for the AuditRecord

```
class rwslib.builders.clinicaldata.ClinicalData (projectname, environment, meta-  
data_version_oid='1', annotations=None)
```

Models the ODM ClinicalData object

Parameters

- **projectname** – Name of Project in Medidata Rave
- **environment** – Rave Study Environment
- **metadata_version_oid** – MetadataVersion OID

subject_data = None

collection of *SubjectData* for the ClinicalData Element

```
class rwslib.builders.clinicaldata.Comment (text=None, sponsor_or_site=None)
```

A free-text (uninterpreted) comment about clinical data. The comment may have come from the Sponsor or the clinical Site.

Note: Comment is not supported by Medidata Rave

Parameters

- **text** (*str*) – Text for Comment
- **sponsor_or_site** (*str*) – Originator flag for Comment (either *_Sponsor_* or *_Site_*)

sponsor_or_site

Originator of comment (either Sponsor or Site)

text

Text content of Comment

class `rwslib.builders.clinicaldata.DateTimeStamp` (*date_time*)

The date/time that the data entry, modification, or signature was performed. This applies to the initial occurrence of the action, not to subsequent transfers between computer systems.

date_time = None

specified DateTime for event

class `rwslib.builders.clinicaldata.Flag` (*flag_type=None, flag_value=None*)

A machine-processable annotation on clinical data.

Note: Flag is not supported by Rave

Parameters

- **flag_type** (*FlagType*) – Type for Flag
- **flag_value** (*FlagValue*) – Value for Flag

class `rwslib.builders.clinicaldata.FlagType` (*flag_type, codelist_oid=None*)

The type of flag. This determines the purpose and semantics of the flag. Different applications are expected to be interested in different types of flags. The actual value must be a member of the referenced CodeList.

Note: FlagType is not supported by Rave

Parameters **flag_type** – Type for *Flag*

codelist_oid

Reference to the CodeList for the FlagType

class `rwslib.builders.clinicaldata.FlagValue` (*flag_value, codelist_oid=None*)

The value of the flag. The meaning of this value is typically dependent on the associated FlagType. The actual value must be a member of the referenced CodeList.

Note: FlagValue is not supported by Rave

Parameters **flag_value** – Value for *Flag*

codelist_oid

Reference to the CodeList for the FlagType

class `rwslib.builders.clinicaldata.FormData` (*formoid, transaction_type=None, form_repeat_key=None*)

Models the ODM FormData object

Parameters

- **formoid** (*str*) – FormDef OID
- **transaction_type** (*str*) – Transaction Type for Data (one of **Insert**, **Update**)

- **form_repeat_key** (*str*) – Repeat Key for FormData

annotations = None

Collection of *Annotation* for FormData - *Not supported by Rave*

signature = None

Signature for FormData

```
class rwslib.builders.clinicaldata.ItemData (itemoid, value, specify_value=None, transaction_type=None, lock=None, freeze=None, verify=None)
```

Models the ODM ItemData object

Parameters

- **itemoid** (*str*) – OID for the matching ItemDef
- **value** (*str*) – Value for the the ItemData
- **specify_value** (*str*) – ‘If other, specify’ value - *Rave specific attribute*
- **transaction_type** (*str*) – Transaction type for the data
- **lock** (*bool*) – Lock the DataPoint? - *Rave specific attribute*
- **freeze** (*bool*) – Freeze the DataPoint? - *Rave specific attribute*
- **verify** (*bool*) – Verify the DataPoint? - *Rave specific attribute*

annotations = None

the list of *Annotation* on the DataPoint - *Not supported by Rave*

audit_record = None

the corresponding *AuditRecord* for the DataPoint

deviations = None

the list of *MdsolProtocolDeviation* references on the DataPoint - *Rave Specific Attribute*

measurement_unit_ref = None

the corresponding MeasurementUnitRef for the DataPoint

queries = None

the list of *MdsolQuery* on the DataPoint - *Rave Specific Attribute*

```
class rwslib.builders.clinicaldata.ItemGroupData (itemgroupoid=None, transaction_type=None, item_group_repeat_key=None, whole_item_group=False, annotations=None)
```

Models the ODM ItemGroupData object.

Note: No name for the ItemGroupData element is required. This is built automatically by the form.

Parameters

- **transaction_type** (*str*) – TransactionType for the ItemGroupData
- **item_group_repeat_key** (*int*) – RepeatKey for the ItemGroupData
- **whole_item_group** (*bool*) – Is this the entire ItemGroupData, or just parts? - *Rave specific attribute*

- **annotations** (*list* (*Annotation*) or *Annotation*) – Annotation for the Item-Group - *Not supported by Rave*

signature = *None*

Signature for ItemGroupData

class rwslib.builders.clinicaldata.**LocationRef** (*oid*)

Reference to a Location

Parameters *oid* (*str*) – OID for referenced Location

class rwslib.builders.clinicaldata.**MdsolProtocolDeviation** (*value*, *status*,
repeat_key=1,
code=None,
klass=None, *trans-*
action_type=None)

Extension for Protocol Deviations in Rave

Note:

- This is a Medidata Rave Specific Extension
 - This primarily exists as a mechanism for use by the Clinical Audit Record Service, but it is useful to define for the builders
-

Parameters

- **value** (*str*) – Value for the Protocol Deviation
- **status** (*rwslib.builder_constants.ProtocolDeviationStatus*) –
- **repeat_key** (*int*) – RepeatKey for the Protocol Deviation
- **code** (*basestring*) – Protocol Deviation Code
- **klass** (*basestring*) – Protocol Deviation Class
- **transaction_type** – Transaction Type for the Protocol Deviation

class rwslib.builders.clinicaldata.**MdsolQuery** (*value=None*, *query_repeat_key=None*,
recipient=None, *status=None*, *re-*
quires_response=None, *response=None*)

MdsolQuery extension element for Queries at item level only

Note: This is a Medidata Rave specific extension

Parameters

- **value** (*str*) – Query Value
- **query_repeat_key** (*int*) – Repeat key for Query
- **recipient** (*str*) – Recipient for Query
- **status** (*QueryStatusType*) – Query status
- **requires_response** (*bool*) – Does this Query need a response?
- **response** (*str* or *None*) – Query response (if any)

status

Query Status

class rwslib.builders.clinicaldata.**ReasonForChange** (*reason*)

A user-supplied reason for a data change.

Parameters *reason* (*str*) – Supplied Reason for change**class** rwslib.builders.clinicaldata.**Signature** (*signature_id=None, user_ref=None, location_ref=None, signature_ref=None, date_time_stamp=None*)

An electronic signature applies to a collection of clinical data. This indicates that some user accepts legal responsibility for that data. See 21 CFR Part 11. The signature identifies the person signing, the location of signing, the signature meaning (via the referenced SignatureDef), the date and time of signing, and (in the case of a digital signature) an encrypted hash of the included data.

Parameters

- **user_ref** (*UserRef*) – *UserRef* for User signing Data
- **location_ref** (*LocationRef*) – *LocationRef* for Location of signing
- **signature_ref** (*SignatureRef*) – *SignatureRef* for SignatureDef providing signature meaning
- **date_time_stamp** – *DateTimeStamp* for the time of Signature

signature_id

The ID for the Signature

Note: If a Signature element is contained within a Signatures element, the ID attribute is required.

class rwslib.builders.clinicaldata.**SignatureRef** (*oid*)

Reference to a Signature

Parameters *oid* (*str*) – OID for referenced *Signature***class** rwslib.builders.clinicaldata.**SiteRef** (*oid*)

Reference to a Location The default value is *SiteName*, and the value *SiteUUID* implies that the *LocationOID* .. note:: The *mdsol:LocationOIDType* attribute should be used to indicate the type of *LocationOID*

Parameters *oid* (*str*) – OID for referenced Location**class** rwslib.builders.clinicaldata.**SourceID** (*source_id*)

Information that identifies the source of the data within an originating system. It is only meaningful within the context of that system.

source_id = None

specified DateTime for event

class rwslib.builders.clinicaldata.**StudyEventData** (*study_event_oid, transaction_type='Update', study_event_repeat_key=None*)

Models the ODM StudyEventData object

Parameters

- **study_event_oid** (*str*) – StudyEvent OID
- **transaction_type** (*str*) – Transaction Type for Data (one of **Insert**, **Update**, **Remove**, **Context**)

- **study_event_repeat_key** (*int*) – StudyEventRepeatKey for StudyEventData

annotations = None
Annotation for Study Event Data - *Not Supported by Rave*

forms = None
FormData part of Study Event Data

signature = None
Signature for Study Event Data

class rwslib.builders.clinicaldata.**SubjectData** (*site_location_oid*, *subject_key*, *subject_key_type*=*'SubjectName'*, *transaction_type*=*'Update'*)

Models the ODM SubjectData and ODM SiteRef objects

Parameters

- **site_location_oid** (*str*) – SiteLocation OID
- **subject_key** (*str*) – Value for SubjectKey
- **subject_key_type** (*str*) – Specifier as to the type of SubjectKey (either **SubjectName** or **SubjectUUID**)
- **transaction_type** (*str*) – Transaction Type for Data (one of **Insert**, **Update**, **Upsert**)

annotations = None
collection of *Annotation*

audit_record = None
AuditRecord for SubjectData - *Not Supported By Rave*

signature = None
Signature for SubjectData

siteref = None
SiteRef

study_events = None
collection of *StudyEventData*

class rwslib.builders.clinicaldata.**UserRef** (*oid*)

Reference to a User

Parameters **oid** (*str*) – OID for referenced User

13.3 Medidata Extensions to ODM (MODM) Builders

Note: Any Class with the Prefix **Mdsol** represents a Medidata Rave specific extension

class rwslib.builders.clinicaldata.**Annotation** (*annotation_id*=None, *seqnum*=1, *flags*=None, *comment*=None, *transaction_type*=None)

A general note about clinical data. If an annotation has both a comment and flags, the flags should be related to the comment.

Note: Annotation is not supported by Medidata Rave

Parameters

- **id** (*str* or *None*) – ID for this Annotation (required if contained within an Annotations element)
- **seqnum** (*int*) – SeqNum for Annotation
- **flags** (*Flag* or *list (Flag)*) – one or more *Flag* for the Annotation
- **comment** (*Comment*) – one or more *Comment* for the Annotation
- **transaction_type** – TransactionType for Annotation (one of **Insert**, **Update**, **Remove**, **Upsert**, **Context**)

annotation_id

ID for annotation

Note: If an Annotation is contained with an Annotations element, the ID attribute is required.

seqnum

SeqNum attribute (a small positive integer) uniquely identifies the annotation within its parent entity.

class rwslib.builders.clinicaldata.**Annotations** (*annotations=[]*)

Groups Annotation elements referenced by ItemData[TYPE] elements.

class rwslib.builders.clinicaldata.**AuditRecord** (*edit_point=None*,
used_imputation_method=None, *identifier=None*, *include_file_oid=None*)

An AuditRecord carries information pertaining to the creation, deletion, or modification of clinical data. This information includes who performed that action, and where, when, and why that action was performed.

Note: AuditRecord is supported only by *ItemData* in Rave

Parameters

- **identifier** (*str*) – Audit identifier
- **edit_point** (*str*) – EditPoint attribute identifies the phase of data processing in which action occurred (*Monitoring*, *DataManagement*, *DBAudit*)
- **used_imputation_method** (*bool*) – Indicates whether the action involved the use of a Method
- **include_file_oid** (*bool*) – Include the FileOID in the AuditRecord

audit_id

AuditRecord ID

Note: If an AuditRecord is contained within an AuditRecords element, the ID attribute must be provided.

date_time_stamp = *None*

DateTimeStamp for the AuditRecord**edit_point**

EditPoint attribute identifies the phase of data processing in which action occurred (*Monitoring*, *DataManagement*, *DBAudit*)

location_ref = None
LocationRef for the AuditRecord

reason_for_change = None
ReasonForChange for the AuditRecord

source_id = None
SourceID for the AuditRecord

user_ref = None
UserRef for the AuditRecord

class rwslib.builders.clinicaldata.**ClinicalData** (*projectname, environment, meta-data_version_oid='1', annotations=None*)

Models the ODM ClinicalData object

Parameters

- **projectname** – Name of Project in Medidata Rave
- **environment** – Rave Study Enviroment
- **metadata_version_oid** – MetadataVersion OID

subject_data = None
collection of *SubjectData* for the ClinicalData Element

class rwslib.builders.clinicaldata.**Comment** (*text=None, sponsor_or_site=None*)
A free-text (uninterpreted) comment about clinical data. The comment may have come from the Sponsor or the clinical Site.

Note: Comment is not supported by Medidata Rave

Parameters

- **text** (*str*) – Text for Comment
- **sponsor_or_site** (*str*) – Originator flag for Comment (either *_Sponsor_* or *_Site_*)

sponsor_or_site
Originator of comment (either Sponsor or Site)

text
Text content of Comment

class rwslib.builders.clinicaldata.**DateTimeStamp** (*date_time*)
The date/time that the data entry, modification, or signature was performed. This applies to the initial occurrence of the action, not to subsequent transfers between computer systems.

date_time = None
specified DateTime for event

class rwslib.builders.clinicaldata.**Flag** (*flag_type=None, flag_value=None*)
A machine-processable annotation on clinical data.

Note: Flag is not supported by Rave

Parameters

- **flag_type** ([FlagType](#)) – Type for Flag
- **flag_value** ([FlagValue](#)) – Value for Flag

class `rwslib.builders.clinicaldata.FlagType` (*flag_type, codelist_oid=None*)

The type of flag. This determines the purpose and semantics of the flag. Different applications are expected to be interested in different types of flags. The actual value must be a member of the referenced CodeList.

Note: FlagType is not supported by Rave

Parameters **flag_type** – Type for [Flag](#)

codelist_oid

Reference to the CodeList for the FlagType

class `rwslib.builders.clinicaldata.FlagValue` (*flag_value, codelist_oid=None*)

The value of the flag. The meaning of this value is typically dependent on the associated FlagType. The actual value must be a member of the referenced CodeList.

Note: FlagValue is not supported by Rave

Parameters **flag_value** – Value for [Flag](#)

codelist_oid

Reference to the CodeList for the FlagType

class `rwslib.builders.clinicaldata.FormData` (*formoid, transaction_type=None, form_repeat_key=None*)

Models the ODM FormData object

Parameters

- **formoid** (*str*) – FormDef OID
- **transaction_type** (*str*) – Transaction Type for Data (one of **Insert**, **Update**)
- **form_repeat_key** (*str*) – Repeat Key for FormData

annotations = None

Collection of [Annotation](#) for FormData - *Not supported by Rave*

signature = None

[Signature](#) for FormData

class `rwslib.builders.clinicaldata.ItemData` (*itemoid, value, specify_value=None, transaction_type=None, lock=None, freeze=None, verify=None*)

Models the ODM ItemData object

Parameters

- **itemoid** (*str*) – OID for the matching ItemDef
- **value** (*str*) – Value for the the ItemData
- **specify_value** (*str*) – ‘If other, specify’ value - *Rave specific attribute*
- **transaction_type** (*str*) – Transaction type for the data

- **lock** (*bool*) – Lock the DataPoint? - *Rave specific attribute*
- **freeze** (*bool*) – Freeze the DataPoint? - *Rave specific attribute*
- **verify** (*bool*) – Verify the DataPoint? - *Rave specific attribute*

annotations = **None**

the list of *Annotation* on the DataPoint - *Not supported by Rave*

audit_record = **None**

the corresponding *AuditRecord* for the DataPoint

deviations = **None**

the list of *MdsolProtocolDeviation* references on the DataPoint - *Rave Specific Attribute*

measurement_unit_ref = **None**

the corresponding *MeasurementUnitRef* for the DataPoint

queries = **None**

the list of *MdsolQuery* on the DataPoint - *Rave Specific Attribute*

```
class rwslib.builders.clinicaldata.ItemGroupData (itemgroupoid=None,
                                                    transaction_type=None,
                                                    item_group_repeat_key=None,
                                                    whole_item_group=False,  annota-
                                                    tions=None)
```

Models the ODM ItemGroupData object.

Note: No name for the ItemGroupData element is required. This is built automatically by the form.

Parameters

- **transaction_type** (*str*) – TransactionType for the ItemGroupData
- **item_group_repeat_key** (*int*) – RepeatKey for the ItemGroupData
- **whole_item_group** (*bool*) – Is this the entire ItemGroupData, or just parts? - *Rave specific attribute*
- **annotations** (*list (Annotation) or Annotation*) – Annotation for the Item-Group - *Not supported by Rave*

signature = **None**

Signature for ItemGroupData

```
class rwslib.builders.clinicaldata.LocationRef (oid)
```

Reference to a Location

Parameters **oid** (*str*) – OID for referenced Location

```
class rwslib.builders.clinicaldata.MdsolProtocolDeviation (value,          status,
                                                            repeat_key=1,
                                                            code=None,
                                                            klass=None,      trans-
                                                            action_type=None)
```

Extension for Protocol Deviations in Rave

Note:

- This is a Medidata Rave Specific Extension

- This primarily exists as a mechanism for use by the Clinical Audit Record Service, but it is useful to define for the builders

Parameters

- **value** (*str*) – Value for the Protocol Deviation
- **status** (*rwslib.builder_constants.ProtocolDeviationStatus*) –
- **repeat_key** (*int*) – RepeatKey for the Protocol Deviation
- **code** (*basestring*) – Protocol Deviation Code
- **klass** (*basestring*) – Protocol Deviation Class
- **transaction_type** – Transaction Type for the Protocol Deviation

```
class rwslib.builders.clinicaldata.MdsolQuery (value=None, query_repeat_key=None,
                                             recipient=None, status=None, re-
                                             quires_response=None, response=None)
```

MdsolQuery extension element for Queries at item level only

Note: This is a Medidata Rave specific extension

Parameters

- **value** (*str*) – Query Value
- **query_repeat_key** (*int*) – Repeat key for Query
- **recipient** (*str*) – Recipient for Query
- **status** (*QueryStatusType*) – Query status
- **requires_response** (*bool*) – Does this Query need a response?
- **response** (*str or None*) – Query response (if any)

status

Query Status

```
class rwslib.builders.clinicaldata.ReasonForChange (reason)
```

A user-supplied reason for a data change.

Parameters **reason** (*str*) – Supplied Reason for change

```
class rwslib.builders.clinicaldata.Signature (signature_id=None, user_ref=None, lo-
                                             cation_ref=None, signature_ref=None,
                                             date_time_stamp=None)
```

An electronic signature applies to a collection of clinical data. This indicates that some user accepts legal responsibility for that data. See 21 CFR Part 11. The signature identifies the person signing, the location of signing, the signature meaning (via the referenced SignatureDef), the date and time of signing, and (in the case of a digital signature) an encrypted hash of the included data.

Parameters

- **user_ref** (*UserRef*) – *UserRef* for User signing Data
- **location_ref** (*LocationRef*) – *LocationRef* for Location of signing

- **signature_ref** (*SignatureRef*) – *SignatureRef* for *SignatureDef* providing signature meaning
- **date_time_stamp** – *DateTimeStamp* for the time of Signature

signature_id

The ID for the Signature

Note: If a Signature element is contained within a Signatures element, the ID attribute is required.

class rwslib.builders.clinicaldata.**SignatureRef** (*oid*)

Reference to a Signature

Parameters **oid** (*str*) – OID for referenced *Signature*

class rwslib.builders.clinicaldata.**SiteRef** (*oid*)

Reference to a Location The default value is *SiteName*, and the value *SiteUUID* implies that the *LocationOID* .. note:: The *mdsol:LocationOIDType* attribute should be used to indicate the type of *LocationOID*

Parameters **oid** (*str*) – OID for referenced Location

class rwslib.builders.clinicaldata.**SourceID** (*source_id*)

Information that identifies the source of the data within an originating system. It is only meaningful within the context of that system.

source_id = None

specified DateTime for event

class rwslib.builders.clinicaldata.**StudyEventData** (*study_event_oid*, *transaction_type*, *study_event_repeat_key*)

Models the ODM StudyEventData object

Parameters

- **study_event_oid** (*str*) – StudyEvent OID
- **transaction_type** (*str*) – Transaction Type for Data (one of **Insert**, **Update**, **Remove**, **Context**)
- **study_event_repeat_key** (*int*) – StudyEventRepeatKey for StudyEventData

annotations = None

Annotation for Study Event Data - *Not Supported by Rave*

forms = None

FormData part of Study Event Data

signature = None

Signature for Study Event Data

class rwslib.builders.clinicaldata.**SubjectData** (*site_location_oid*, *subject_key*, *subject_key_type*, *transaction_type*)

Models the ODM SubjectData and ODM SiteRef objects

Parameters

- **site_location_oid** (*str*) – SiteLocation OID
- **subject_key** (*str*) – Value for SubjectKey

- **subject_key_type** (*str*) – Specifier as to the type of SubjectKey (either **Subject-Name** or **SubjectUUID**)
- **transaction_type** (*str*) – Transaction Type for Data (one of **Insert**, **Update**, **Upsert**)

annotations = **None**

collection of *Annotation*

audit_record = **None**

AuditRecord for SubjectData - *Not Supported By Rave*

signature = **None**

Signature for SubjectData

siteref = **None**

SiteRef

study_events = **None**

collection of *StudyEventData*

class rwslib.builders.clinicaldata.**UserRef** (*oid*)

Reference to a User

Parameters **oid** (*str*) – OID for referenced User

13.4 Administrative Data Builders

Note: Any Class with the Prefix **Mdsol** represents a Medidata Rave specific extension

class rwslib.builders.admindata.**Address** (*street_names=None, city=None, state_prov=None, country=None, postal_code=None, other_text=None*)

The user's postal address.

Parameters

- **street_names** (*list* (*Address*)) – User street names
- **city** (*City*) – User City
- **state_prov** (*StateProv*) – User State or Province
- **country** (*Country*) – User City
- **postal_code** (*PostalCode*) – User City
- **other_text** (*OtherText*) – User Other Text

class rwslib.builders.admindata.**AdminData** (*study_oid=None*)

Administrative information about users, locations, and electronic signatures.

Parameters **study_oid** (*str*) – OID pointing to the StudyDef

class rwslib.builders.admindata.**City** (*text*)

The city name part of a user's postal address.

class rwslib.builders.admindata.**Country** (*country_code*)

The country name part of a user's postal address. This must be represented by an ISO 3166 two-letter country code.

class rwslib.builders.admindata.**DisplayName** (*text*)

A short displayable name for the user.

class rwslib.builders.admindata.**Email** (*text*)

The user's email address.

class rwslib.builders.admindata.**FirstName** (*text*)

The user's initial given name or all given names.

class rwslib.builders.admindata.**FullName** (*text*)

The user's full formal name.

class rwslib.builders.admindata.**LastName** (*text*)

The user's surname (family name).

class rwslib.builders.admindata.**Location** (*oid*, *name*, *location_type=None*, *meta-data_versions=None*)

A physical location – typically a clinical research site or a sponsor's office.

Parameters

- **oid** (*str*) – OID for the Location, referenced in `LocationRef`
- **name** (*str*) – Name for the Location
- **location_type** (`rwslib.builder_constants.LocationType`) – Type for this Location
- **metadata_versions** (*list* (`MetaDataVersionRef`)) – The `MetaDataVersionRef` for this Location

class rwslib.builders.admindata.**LoginName** (*text*)

The user's login identification.

class rwslib.builders.admindata.**MetaDataVersionRef** (*study_oid*, *metadata_version_oid*, *effective_date*)

A reference to a MetaDataVersion used at the containing Location. The `EffectiveDate` expresses the fact that the metadata used at a location can vary over time.

Parameters

- **study_oid** (*str*) – References the `Study` that uses this metadata version.
- **metadata_version_oid** (*str*) – References the `rwslib.builders.MetaDataVersion` (within the above `Study`).
- **effective_date** (`datetime.datetime`) – Effective Date for this version and Site

class rwslib.builders.admindata.**Organization** (*text*)

The user's organization.

class rwslib.builders.admindata.**OtherText** (*text*)

Any other text needed as part of a user's postal address.

class rwslib.builders.admindata.**Phone** (*text*)

The user's voice phone number.

class rwslib.builders.admindata.**PostalCode** (*text*)

The postal code part of a user's postal address.

class rwslib.builders.admindata.**SimpleChildElement** (*text*)

Generic Element, for elements we're not ready to flesh out in the builders

class rwslib.builders.admindata.**StateProv** (*text*)

The state or province name part of a user's postal address.

class rwslib.builders.admindata.**StreetName** (*text*)

The street address part of a user's postal address.

class rwslib.builders.admindata.**User** (*oid*, *user_type=None*, *login_name=None*, *display_name=None*, *full_name=None*, *first_name=None*, *last_name=None*, *organisation=None*, *addresses=[]*, *emails=[]*, *phones=[]*, *locations=[]*)

Information about a specific user of a clinical data collection system. This may be an investigator, a CRA, or data management staff. Study subjects are not users in this sense.

Parameters

- **oid** (*str*) –
- **user_type** (*rwslib.builder_constants.UserType*) – User Type
- **login_name** (*LoginName*) – User Login Name - see *LoginName*
- **display_name** (*DisplayName*) – User Display Name - see *DisplayName*
- **full_name** (*FullName*) – User Full Name - see *FullName*
- **first_name** (*FirstName*) – User First Name - see *FirstName*
- **last_name** (*LastName*) – User Last Name - see *LastName*
- **organisation** (*Organisation*) – User Organisation - see *Organisation*
- **addresses** (*list (Address)*) – User Address - see *Address*
- **emails** (*list (Email)*) – User Email - see *Email*
- **phones** (*list (Phone)*) – User Phone - see *Phone*
- **locations** (*list (LocationRef)*) – Locations for User - see *LocationRef*

user_type

User Type :return:

CHAPTER 14

Indices and tables

- `genindex`
- `modindex`
- `search`

r

- `rwslib`, [65](#)
- `rwslib.builders.admindata`, [107](#)
- `rwslib.builders.clinicaldata`, [100](#)
- `rwslib.builders.metadata`, [79](#)
- `rwslib.rws_requests`, [66](#)
- `rwslib.rws_requests.biostats_gateway`,
[75](#)
- `rwslib.rws_requests.odm_adapter`, [78](#)
- `rwslib.rwsobjects`, [71](#)

A

Address (class in *rwslib.builders.admindata*), 107
 AdminData (class in *rwslib.builders.admindata*), 107
 Alias (class in *rwslib.builders.metadata*), 79
 aliases (*rwslib.builders.metadata.CodeList* attribute), 80
 aliases (*rwslib.builders.metadata.CodeListItem* attribute), 80
 aliases (*rwslib.builders.metadata.FormDef* attribute), 81
 aliases (*rwslib.builders.metadata.ItemDef* attribute), 83
 aliases (*rwslib.builders.metadata.ItemGroupDef* attribute), 84
 Annotation (class in *rwslib.builders.clinicaldata*), 94, 100
 annotation_id (*rwslib.builders.clinicaldata.Annotation* attribute), 94, 101
 Annotations (class in *rwslib.builders.clinicaldata*), 94, 101
 annotations (*rwslib.builders.clinicaldata.FormData* attribute), 97, 103
 annotations (*rwslib.builders.clinicaldata.ItemData* attribute), 97, 104
 annotations (*rwslib.builders.clinicaldata.StudyEventData* attribute), 100, 106
 annotations (*rwslib.builders.clinicaldata.SubjectData* attribute), 100, 107
 args () (*rwslib.rws_requests.PostDataRequest* method), 67
 args () (*rwslib.rws_requests.PostMetadataRequest* method), 67
 args () (*rwslib.rws_requests.RWSRequest* method), 68
 attributes (*rwslib.builders.metadata.ItemRef* attribute), 85
 audit_id (*rwslib.builders.clinicaldata.AuditRecord* attribute), 95, 101
 audit_record (*rwslib.builders.clinicaldata.ItemData* attribute), 97, 104

audit_record (*rwslib.builders.clinicaldata.SubjectData* attribute), 100, 107
 AuditRecord (class in *rwslib.builders.clinicaldata*), 94, 101
 AuditRecordsRequest, 57
 AuditRecordsRequest (class in *rwslib.rws_requests.odm_adapter*), 78
 AuthorizationException, 65

B

BasicDefinitions (class in *rwslib.builders.metadata*), 79
 BuildVersionRequest, 9
 BuildVersionRequest (class in *rwslib.rws_requests*), 66

C

CacheFlushRequest, 11
 CacheFlushRequest (class in *rwslib.rws_requests*), 66
 check_action_type (*rwslib.builders.metadata.MdsolCheckAction* attribute), 86
 check_actions (*rwslib.builders.metadata.MdsolEditCheckDef* attribute), 89
 check_dataset_format () (in module *rwslib.rws_requests.biostats_gateway*), 77
 check_dataset_type () (in module *rwslib.rws_requests*), 70
 check_steps (*rwslib.builders.metadata.MdsolEditCheckDef* attribute), 89
 CheckValue (class in *rwslib.builders.metadata*), 79
 City (class in *rwslib.builders.admindata*), 107
 ClinicalData (class in *rwslib.builders.clinicaldata*), 95, 102
 ClinicalStudiesRequest, 13
 ClinicalStudiesRequest (class in *rwslib.rws_requests*), 66
 CodeList (class in *rwslib.builders.metadata*), 79

[codelist_items \(rwslib.builders.metadata.CodeList attribute\), 80](#)
[codelist_oid \(rwslib.builders.clinicaldata.FlagType attribute\), 96, 103](#)
[codelist_oid \(rwslib.builders.clinicaldata.FlagValue attribute\), 96, 103](#)
[CodeListItem \(class in rwslib.builders.metadata\), 80](#)
[CodeListRef \(class in rwslib.builders.metadata\), 80](#)
[codelistref \(rwslib.builders.metadata.ItemDef attribute\), 83](#)
[CodeNameRequest, 10](#)
[CodeNameRequest \(class in rwslib.rws_requests\), 66](#)
[Comment \(class in rwslib.builders.clinicaldata\), 95, 102](#)
[CommentDataRequest, 53](#)
[CommentDataRequest \(class in rwslib.rws_requests.biostats_gateway\), 76](#)
[comparator \(rwslib.builders.metadata.RangeCheck attribute\), 92](#)
[ConfigurableDatasetRequest, 11](#)
[ConfigurableDatasetRequest \(class in rwslib.rws_requests\), 66](#)
[content \(rwslib.builders.metadata.MdsolHelpText attribute\), 90](#)
[Country \(class in rwslib.builders.admindata\), 107](#)
[CVMetaDataRequest, 43](#)
[CVMetaDataRequest \(class in rwslib.rws_requests.biostats_gateway\), 75](#)

D

[DataDictionariesRequest, 55](#)
[DataDictionariesRequest \(class in rwslib.rws_requests.biostats_gateway\), 76](#)
[dataset \(rwslib.rws_requests.ConfigurableDatasetRequest attribute\), 66](#)
[dataset_format_to_extension\(\) \(in module rwslib.rws_requests.biostats_gateway\), 77](#)
[date_time \(rwslib.builders.clinicaldata.DateTimeStamp attribute\), 96, 102](#)
[date_time_stamp \(rwslib.builders.clinicaldata.AuditRecord attribute\), 95, 101](#)
[DateTimeStamp \(class in rwslib.builders.clinicaldata\), 96, 102](#)
[Decode \(class in rwslib.builders.metadata\), 80](#)
[derivation_steps \(rwslib.builders.metadata.MdsolDerivationDef attribute\), 88](#)
[deviations \(rwslib.builders.clinicaldata.ItemData attribute\), 97, 104](#)
[DiagnosticsRequest, 10](#)
[DiagnosticsRequest \(class in rwslib.rws_requests\), 66](#)
[DisplayName \(class in rwslib.builders.admindata\), 107](#)

E

[edit_point \(rwslib.builders.clinicaldata.AuditRecord attribute\), 95, 101](#)
[Email \(class in rwslib.builders.admindata\), 108](#)
[entry_restrictions \(rwslib.builders.metadata.FormDef attribute\), 81](#)
[entry_restrictions \(rwslib.builders.metadata.ItemDef attribute\), 83](#)

F

[FirstName \(class in rwslib.builders.admindata\), 108](#)
[Flag \(class in rwslib.builders.clinicaldata\), 96, 102](#)
[FlagType \(class in rwslib.builders.clinicaldata\), 96, 103](#)
[FlagValue \(class in rwslib.builders.clinicaldata\), 96, 103](#)
[format_date_argument\(\) \(in module rwslib.rws_requests\), 70](#)
[FormData \(class in rwslib.builders.clinicaldata\), 96, 103](#)
[FormDataRequest, 45](#)
[FormDataRequest \(class in rwslib.rws_requests.biostats_gateway\), 76](#)
[FormDef \(class in rwslib.builders.metadata\), 80](#)
[FormRef \(class in rwslib.builders.metadata\), 81](#)
[forms \(rwslib.builders.clinicaldata.StudyEventData attribute\), 100, 106](#)
[fromElement\(\) \(rwslib.rwsobjects.MetaDataVersion class method\), 71](#)
[fromElement\(\) \(rwslib.rwsobjects.RWSSudyListItem class method\), 73](#)
[fromElement\(\) \(rwslib.rwsobjects.RWSSubjectListItem class method\), 74](#)
[FullName \(class in rwslib.builders.admindata\), 108](#)
[function \(rwslib.builders.metadata.MdsolDerivationStep attribute\), 89](#)

G

[getEnvironmentFromNameAndProtocol\(\) \(in module rwslib.rwsobjects\), 75](#)
[GlobalLibrariesRequest, 29](#)
[GlobalLibrariesRequest \(class in rwslib.rws_requests\), 66](#)
[GlobalLibraryDraftsRequest, 29](#)
[GlobalLibraryDraftsRequest \(class in rwslib.rws_requests\), 66](#)
[GlobalLibraryVersionRequest, 30](#)
[GlobalLibraryVersionRequest \(class in rwslib.rws_requests\), 66](#)
[GlobalLibraryVersionsRequest, 30](#)
[GlobalLibraryVersionsRequest \(class in rwslib.rws_requests\), 67](#)

GlobalVariables (class in *rwslib.builders.metadata*), 81

H

header_text (*rwslib.builders.metadata.ItemDef* attribute), 83

help_texts (*rwslib.builders.metadata.ItemDef* attribute), 83

help_texts (*rwslib.builders.metadata.MdsolLabelDef* attribute), 90

helptexts (*rwslib.builders.metadata.FormDef* attribute), 81

I

isProd() (*rwslib.rwsobjects.RWSStudyListItem* method), 73

item_refs (*rwslib.builders.metadata.ItemGroupDef* attribute), 84

ItemData (class in *rwslib.builders.clinicaldata*), 97, 103

ItemDef (class in *rwslib.builders.metadata*), 82

itemgroup_refs (*rwslib.builders.metadata.FormDef* attribute), 81

ItemGroupData (class in *rwslib.builders.clinicaldata*), 97, 104

ItemGroupDef (class in *rwslib.builders.metadata*), 84

ItemGroupRef (class in *rwslib.builders.metadata*), 84

ItemRef (class in *rwslib.builders.metadata*), 84

L

label_refs (*rwslib.builders.metadata.ItemGroupDef* attribute), 84

lang (*rwslib.builders.metadata.MdsolHelpText* attribute), 90

LastName (class in *rwslib.builders.admindata*), 108

Location (class in *rwslib.builders.admindata*), 108

location_ref (*rwslib.builders.clinicaldata.AuditRecord* attribute), 95, 101

LocationRef (class in *rwslib.builders.clinicaldata*), 98, 104

logical_record_position (*rwslib.builders.metadata.MdsolCheckStep* attribute), 87

logical_record_position (*rwslib.builders.metadata.MdsolDerivationDef* attribute), 88

logical_record_position (*rwslib.builders.metadata.MdsolDerivationStep* attribute), 89

LoginName (class in *rwslib.builders.admindata*), 108

M

make_url() (in module *rwslib.rws_requests*), 71

MdsolAttribute (class in *rwslib.builders.metadata*), 85

MdsolCheckAction (class in *rwslib.builders.metadata*), 85

MdsolCheckStep (class in *rwslib.builders.metadata*), 86

MdsolConfirmationMessage (class in *rwslib.builders.metadata*), 87

MdsolCustomFunctionDef (class in *rwslib.builders.metadata*), 87

MdsolDerivationDef (class in *rwslib.builders.metadata*), 87

MdsolDerivationStep (class in *rwslib.builders.metadata*), 88

MdsolEditCheckDef (class in *rwslib.builders.metadata*), 89

MdsolEntryRestriction (class in *rwslib.builders.metadata*), 89

MdsolHeaderText (class in *rwslib.builders.metadata*), 89

MdsolHelpText (class in *rwslib.builders.metadata*), 90

MdsolLabelDef (class in *rwslib.builders.metadata*), 90

MdsolLabelRef (class in *rwslib.builders.metadata*), 90

MdsolProtocolDeviation (class in *rwslib.builders.clinicaldata*), 98, 104

MdsolQuery (class in *rwslib.builders.clinicaldata*), 98, 105

MdsolReviewGroup (class in *rwslib.builders.metadata*), 90

MdsolViewRestriction (class in *rwslib.builders.metadata*), 91

measurement_unit_ref (*rwslib.builders.clinicaldata.ItemData* attribute), 97, 104

measurement_unit_refs (*rwslib.builders.metadata.ItemDef* attribute), 83

measurement_units (*rwslib.builders.metadata.BasicDefinitions* attribute), 79

MeasurementUnit (class in *rwslib.builders.metadata*), 91

MeasurementUnitRef (class in *rwslib.builders.metadata*), 91

MetaDataRequest, 46

MetaDataRequest (class in *rwslib.rws_requests.biostats_gateway*), 77

MetadataStudiesRequest, 23

MetadataStudiesRequest (class in *rwslib.rws_requests*), 67

MetaDataVersion (class in

rwslib.builders.metadata), 91
 MetaDataVersion (class in *rwslib.rwsobjects*), 71
 MetaDataVersionRef (class in *rwslib.builders.admindata*), 108

N

name (*rwslib.builders.metadata.MdsolAttribute* attribute), 85
 namespace (*rwslib.builders.metadata.MdsolAttribute* attribute), 85

O

ODM Clinical Datasets, 16
 ODMDatasetBase (class in *rwslib.rws_requests*), 67
 ODMDoc (class in *rwslib.rwsobjects*), 71
 oid (*rwslib.builders.metadata.MdsolLabelRef* attribute), 90
 order_number (*rwslib.builders.metadata.MdsolLabelRef* attribute), 90
 Organization (class in *rwslib.builders.admindata*), 108
 OtherText (class in *rwslib.builders.admindata*), 108

P

parseXMLString() (in module *rwslib.rwsobjects*), 75
 Phone (class in *rwslib.builders.admindata*), 108
 PostalCode (class in *rwslib.builders.admindata*), 108
 PostDataRequest, 33
 PostDataRequest (class in *rwslib.rws_requests*), 67
 PostMetadataRequest, 26
 PostMetadataRequest (class in *rwslib.rws_requests*), 67
 ProjectMetaDataRequest, 49
 ProjectMetaDataRequest (class in *rwslib.rws_requests.biostats_gateway*), 77
 Protocol (class in *rwslib.builders.metadata*), 92
 ProtocolDeviationsRequest, 54
 ProtocolDeviationsRequest (class in *rwslib.rws_requests.biostats_gateway*), 77

Q

queries (*rwslib.builders.clinicaldata.ItemData* attribute), 97, 104
 QueryOptionGetRequest (class in *rwslib.rws_requests*), 67
 Question (class in *rwslib.builders.metadata*), 92
 question (*rwslib.builders.metadata.ItemDef* attribute), 83

R

range_checks (*rwslib.builders.metadata.ItemDef* attribute), 83

RangeCheck (class in *rwslib.builders.metadata*), 92
 reason_for_change (*rwslib.builders.clinicaldata.AuditRecord* attribute), 95, 102
 ReasonForChange (class in *rwslib.builders.clinicaldata*), 99, 105
 result () (*rwslib.rws_requests.CacheFlushRequest* method), 66
 result () (*rwslib.rws_requests.ClinicalStudiesRequest* method), 66
 result () (*rwslib.rws_requests.GlobalLibrariesRequest* method), 66
 result () (*rwslib.rws_requests.GlobalLibraryDraftsRequest* method), 66
 result () (*rwslib.rws_requests.GlobalLibraryVersionsRequest* method), 67
 result () (*rwslib.rws_requests.MetadataStudiesRequest* method), 67
 result () (*rwslib.rws_requests.PostDataRequest* method), 67
 result () (*rwslib.rws_requests.PostMetadataRequest* method), 67
 result () (*rwslib.rws_requests.RWSRequest* method), 68
 result () (*rwslib.rws_requests.StudyDraftsRequest* method), 68
 result () (*rwslib.rws_requests.StudySubjectsRequest* method), 69
 result () (*rwslib.rws_requests.StudyVersionsRequest* method), 69
 result () (*rwslib.rws_requests.VersionRequestBase* method), 70
 review_groups (*rwslib.builders.metadata.ItemDef* attribute), 83
 rolename (*rwslib.builders.metadata.MdsolEntryRestriction* attribute), 89
 rolename (*rwslib.builders.metadata.MdsolViewRestriction* attribute), 91
 rws_requests, 8
 RWSAuthorizedGetRequest (class in *rwslib.rws_requests*), 67
 RWSAuthorizedPostRequest (class in *rwslib.rws_requests*), 67
 RWSConnection (class in *rwslib*), 65
 RWSError (class in *rwslib.rwsobjects*), 71
 RWSErrorResponse (class in *rwslib.rwsobjects*), 71
 RWSException, 71
 RWSGetRequest (class in *rwslib.rws_requests*), 67
 rwslib (module), 65
 rwslib.builders.admindata (module), 107
 rwslib.builders.clinicaldata (module), 94, 100
 rwslib.builders.metadata (module), 79
 rwslib.rws_requests (module), 66

rwslib.rws_requests.biostats_gateway (module), 75
 rwslib.rws_requests.odm_adapter (module), 78
 rwslib.rwsobjects (module), 71
 RWSPostErrorResponse (class in *rwslib.rwsobjects*), 71
 RWSPostRequest (class in *rwslib.rws_requests*), 67
 RWSPostResponse (class in *rwslib.rwsobjects*), 72
 RWSRequest (class in *rwslib.rws_requests*), 67
 RWSResponse (class in *rwslib.rwsobjects*), 72
 RWSStudies (class in *rwslib.rwsobjects*), 72
 RWSStudyListItem (class in *rwslib.rwsobjects*), 73
 RWSStudyMetadataVersions (class in *rwslib.rwsobjects*), 73
 RWSSubjectListItem (class in *rwslib.rwsobjects*), 73
 RWSSubjects (class in *rwslib.rwsobjects*), 75

S

send_request() (*rwslib.RWSConnection* method), 65
 seqnum (*rwslib.builders.clinicaldata.Annotation* attribute), 94, 101
 Signature (class in *rwslib.builders.clinicaldata*), 99, 105
 signature (*rwslib.builders.clinicaldata.FormData* attribute), 97, 103
 signature (*rwslib.builders.clinicaldata.ItemGroupData* attribute), 98, 104
 signature (*rwslib.builders.clinicaldata.StudyEventData* attribute), 100, 106
 signature (*rwslib.builders.clinicaldata.SubjectData* attribute), 100, 107
 signature_id (*rwslib.builders.clinicaldata.Signature* attribute), 99, 106
 SignatureDefinitionsRequest, 62
 SignatureDefinitionsRequest (class in *rwslib.rws_requests.odm_adapter*), 78
 SignatureRef (class in *rwslib.builders.clinicaldata*), 99, 106
 SimpleChildElement (class in *rwslib.builders.admindata*), 108
 SiteRef (class in *rwslib.builders.clinicaldata*), 99, 106
 siteref (*rwslib.builders.clinicaldata.SubjectData* attribute), 100, 107
 SitesMetadataRequest, 60
 SitesMetadataRequest (class in *rwslib.rws_requests.odm_adapter*), 78
 soft_hard (*rwslib.builders.metadata.RangeCheck* attribute), 92
 source_id (*rwslib.builders.clinicaldata.AuditRecord* attribute), 95, 102
 source_id (*rwslib.builders.clinicaldata.SourceID* attribute), 99, 106
 SourceID (class in *rwslib.builders.clinicaldata*), 99, 106
 sponsor_or_site (*rwslib.builders.clinicaldata.Comment* attribute), 96, 102
 StateProv (class in *rwslib.builders.admindata*), 108
 status (*rwslib.builders.clinicaldata.MdsolQuery* attribute), 98, 105
 StreetName (class in *rwslib.builders.admindata*), 109
 Study (class in *rwslib.builders.metadata*), 92
 study_event_refs (*rwslib.builders.metadata.Protocol* attribute), 92
 study_events (*rwslib.builders.clinicaldata.SubjectData* attribute), 100, 107
 StudyDatasetRequest, 17
 StudyDatasetRequest (class in *rwslib.rws_requests*), 68
 StudyDraftsRequest, 24
 StudyDraftsRequest (class in *rwslib.rws_requests*), 68
 studyevent_defs (*rwslib.builders.metadata.Study* attribute), 92
 StudyEventData (class in *rwslib.builders.clinicaldata*), 99, 106
 StudyEventDef (class in *rwslib.builders.metadata*), 92
 StudyEventRef (class in *rwslib.builders.metadata*), 93
 studyname_environment() (*rwslib.rws_requests.biostats_gateway.CVMetaDataRequest* method), 76
 StudySubjectsRequest, 14
 StudySubjectsRequest (class in *rwslib.rws_requests*), 68
 StudyVersionRequest, 25
 StudyVersionRequest (class in *rwslib.rws_requests*), 69
 StudyVersionsRequest, 24
 StudyVersionsRequest (class in *rwslib.rws_requests*), 69
 subject_data (*rwslib.builders.clinicaldata.ClinicalData* attribute), 95, 102
 subject_name (*rwslib.rwsobjects.RWSSubjectListItem* attribute), 75
 SubjectData (class in *rwslib.builders.clinicaldata*), 100, 106
 SubjectDatasetRequest, 19
 SubjectDatasetRequest (class in *rwslib.rws_requests*), 69
 Symbol (class in *rwslib.builders.metadata*), 93
 symbols (*rwslib.builders.metadata.MeasurementUnit* attribute), 91

T

`text` (*rwslib.builders.clinicaldata.Comment* attribute), 96, 102

`transaction_type` (*rwslib.builders.metadata.MdsolAttribute* attribute), 85

`TranslatedText` (class in *rwslib.builders.metadata*), 93

`translations` (*rwslib.builders.metadata.Decode* attribute), 80

`translations` (*rwslib.builders.metadata.MdsolLabelDef* attribute), 90

`translations` (*rwslib.builders.metadata.Question* attribute), 92

`translations` (*rwslib.builders.metadata.Symbol* attribute), 93

`TwoHundredRequest`, 10

`TwoHundredRequest` (class in *rwslib.rws_requests*), 70

`ViewMetaDataRequest`, 51

`ViewMetaDataRequest` (class in *rwslib.rws_requests.biostats_gateway*), 77

`X-MWS-CV-Last-Updated`, 21

`XMLRepr` (class in *rwslib.rwsobjects*), 75

U

`User` (class in *rwslib.builders.admindata*), 109

`user_ref` (*rwslib.builders.clinicaldata.AuditRecord* attribute), 95, 102

`user_type` (*rwslib.builders.admindata.User* attribute), 109

`UserRef` (class in *rwslib.builders.clinicaldata*), 100, 107

`UsersRequest`, 61

`UsersRequest` (class in *rwslib.rws_requests.odm_adapter*), 78

`using_builders`, 35

V

`value` (*rwslib.builders.metadata.MdsolAttribute* attribute), 85

`VersionDatasetRequest`, 20

`VersionDatasetRequest` (class in *rwslib.rws_requests*), 70

`VersionFoldersRequest`, 58

`VersionFoldersRequest` (class in *rwslib.rws_requests.odm_adapter*), 78

`VersionRequest`, 9

`VersionRequest` (class in *rwslib.rws_requests*), 70

`VersionRequestBase` (class in *rwslib.rws_requests*), 70

`view_restrictions` (*rwslib.builders.metadata.FormDef* attribute), 81

`view_restrictions` (*rwslib.builders.metadata.ItemDef* attribute), 84

`view_restrictions` (*rwslib.builders.metadata.MdsolLabelDef* attribute), 90